

**A Radiation-Hardened Neuromorphic
System-on-Chip on an Open 130 nm PDK**
Design, Hardening, Verification and Layout

Hasan Melih Akbulut

15 September 2026

Abstract

This document describes a radiation-hardened neuromorphic system-on-chip designed for the IHP SG13G2 130 nm open process design kit with open tools only. The part joins a 32-bit RISC-V application core to a spiking neural-network accelerator, protects both with triple-modular redundancy, single-error-correcting double-error-detecting codes and a scrubbing engine, and carries the whole through synthesis, placement, routing and geometric sign-off on eight vendor memory macros.

Two things distinguish the record. The first is that every claim in it is a measurement with a named source: where a number was superseded it is marked and kept rather than rewritten, and where a check did not finish the document says so in the word the tool used, because TIMEOUT, KILLED and ERROR are three different results. The second is that the design's own instrumentation is part of the subject: guards that count redundancy in the netlist rather than in the source, formal property sets that state one-line invariants over all reachable states, and fault-injection campaigns at register-transfer and gate level whose verdicts are cross-checked against those invariants.

No clock frequency is claimed for this design, by a rule the project adopted early and has kept. No silicon of the full system-on-chip exists; a frozen slice of the accelerator was submitted to a multi-project shuttle and is pinned by hash. What exists is a design in register-transfer form, a placed and routed layout that meets hold and is clean where the design drew, and the measurements that say what each of its mechanisms costs.

Contents

1	Introduction	6
1.1	The part	6
1.2	What this is not	8
1.3	The mission case	8
1.4	Why 130 nm, and why this PDK	10
1.5	Two parts: a frozen pilot and a system around it	12
1.6	Where the work is published	13
1.7	The rules this work holds itself to	14
1.8	What this document is	15
1.9	Road map	16
2	Architecture	17
2.1	The management core	17
2.1.1	Why there is no lockstep	19
2.2	The bus fabric	19
2.3	The memory map, and its single source	20
2.4	The memories	20
2.4.1	Why the check bits cost half the RAM	21
2.5	Time, interrupts and supervision	22
2.5.1	No platform interrupt controller	23
2.5.2	The three-stage watchdog	23
2.6	The peripheral bus and what is on it	24
2.7	The boot flow	26
2.8	Telemetry and discovery	27
2.9	What the top level is, and what it is not	27
3	The neuromorphic accelerator	29
3.1	What it computes: leaky integrate-and-fire, in integers	29
3.2	Spikes on a wire: the address-event representation	30
3.3	The crossbar and the weight memory	30
3.4	Networks larger than the fabric	31
3.5	Attaching it to the system-on-chip	32

3.5.1	Two faces, because there are two problems	32
3.5.2	Why the transport is serial	32
3.5.3	The parallel event pins are not symmetric	32
3.5.4	The event engine	33
3.5.5	One interrupt, and a cause register that scales	33
3.5.6	The clock gate and the wake	33
3.6	The golden-model comparison	34
3.7	The pilot chip	35
3.7.1	What is on it	35
3.7.2	The tile budget, which is where the shape came from	36
3.7.3	How the artefacts are sealed	36
3.8	The mesh link that is specified and not built	36
3.9	One line, finally checkable over all reachable states	37
3.10	Where 15,686 cells landed	38
3.11	What is open	39
4	Radiation hardening	40
4.1	The criterion, and why it is not the flip-flop count	40
4.2	Triple modular redundancy, and why one bit cannot have it	40
4.3	The register file	42
4.4	The time base	43
4.5	The memories	43
4.6	Boot	44
4.7	Stopping the clock without losing the report	45
4.8	W9: a correction that cost a reset	45
4.9	The evidence	46
4.10	TMR blindness	48
4.11	What it costs	49
4.12	Four shapes an error takes here	50
4.13	What is still open	51
5	Verification	52
5.1	Five layers, and the failure they all share	52
5.2	Simulation: the cocotb suites	52
5.2.1	What only simulation can see	53
5.3	Guards: the checks that do not simulate	53
5.3.1	The principle: check the property, not the name	54
5.3.2	Two guards that caught something	54
5.3.3	What guards cannot do	55
5.4	The SymbiYosys property sets	55
5.5	riscv-formal on the management core	57

5.5.1	Binding it, which is where the work was	57
5.5.2	The ledger, and the vacuity control	57
5.5.3	What closed, and what did not	58
5.5.4	Five defects, and the one in the specification	58
5.5.5	The M extension, and the register file	59
5.6	Fault injection	60
5.6.1	At register-transfer level	60
5.6.2	At gate level	61
5.6.3	The finding only the gate level could produce	61
5.7	The cross-check, and where the layers meet	62
5.8	What each layer sees, and where the blind spots are	62
6	Physical design	64
6.1	The flow, and what is pinned	64
6.2	The floorplan	65
6.3	Where the logic went	67
6.4	Timing	70
6.4.1	The worst path	72
6.4.2	The macro read return	73
6.4.3	Where the resizer stopped	73
6.5	Hold, antennas, and 29.6 ps	74
6.6	Two placement experiments	74
6.6.1	The CLINT cluster probe	74
6.6.2	The memory read register	75
6.7	Geometric sign-off	76
6.7.1	What the KLayout deck is objecting to	76
6.8	Power	77
6.9	What is open	78
7	The measurements, together	79
7.1	The state of the part, on one page	79
7.2	Four experiments, 14–15 September 2026	79
7.2.1	The event engine’s invariant, proved over all reachable states	79
7.2.2	The register file with its storage and its scrub	81
7.2.3	The multiply and divide checks, bounded rather than stopped	81
7.2.4	The memory read register, measured on the design that exists	82
7.2.5	A run that measured nothing, kept on purpose	82
8	What exists, what does not, and what comes next	83
8.1	What exists	83
8.2	What does not exist, stated plainly	83

8.3	The open items, ranked by what they would settle	84
8.4	What this document is	84

List of Figures

1.1	The system-on-chip, as placed	7
1.2	Floorplan and the eight vendor macros	11
2.1	Where each block landed. Nine small multiples, one per block, each drawing that block's standard cells against a grey die with the eight vendor memory macros outlined. Hierarchy barely survives this flow's synthesis: of the 168,592 instances in the sign-off run's DEF — 168,584 standard cells and the eight macros — only 1,140 still spell a dot in their name, 632 of them the RAM's, and by the script's own prefix test exactly one carries a top-level block name. The rest are attributed by connectivity — nets keep their hierarchical names, each anonymous cell takes the block most of its nets vote for, and two rounds of neighbour propagation follow (<code>thesis/figures/make_layout_figures.py</code>). The method leaves a remainder: 5,916 of the 60,694 logic cells, 9.75 %, are unattributed and are drawn grey rather than hidden (<code>thesis/figures/layout_attribution.json</code>). Nothing here was placement-constrained except the macros, so the clusters are the placer's.	18
3.1	The two large blocks of the placed and routed <code>soc_top</code> : the Ibex core (left) and the neuromorphic accelerator with the frozen die inside it (right), drawn against the same die outline, core box and eight fixed vendor macros. Generated by <code>thesis/figures/make_layout_figures.py</code> from a sign-off run's placement database and netlist, which is the only provenance that script records.	38
4.1	Every standard cell of the placed die, coloured by block. Only 1,140 of the die's instance names carry a dot at all after synthesis, 632 of them the RAM's, and by the script's own prefix test exactly one carries a top-level block name, so attribution is by connectivity — nets keep their names, each anonymous cell takes the block most of its pins' nets vote for, and two rounds of neighbour propagation follow (<code>thesis/figures/make_layout_figures.py</code> , whose docstring counts 168,592 instances where its output file counts 168,584; these figures are the output file's). The remainder, 5,916 cells or 9.75 % of the 60,694 non-fill cells, is drawn grey rather than hidden, and the 107,890 fill, tap, tie, decap and antenna instances are not drawn at all.	41
6.1	The floorplan and the eight macros	66
6.2	The whole die, coloured by block	68
6.3	One panel per block	69
6.4	Standard-cell density over the core	71

Chapter 1

Introduction

1.1 The part

This document describes a fault-tolerant system-on-chip for on-board AI inference in small-satellite missions [20], implemented on a 130 nm technology with an open-source RTL-to-GDS flow (`README.md`; `docs/00-index` section 1). Its architecture class follows the Frontgrade Gaisler GR801 [8] as published in that device’s April 2026 product brief — a RISC-V management processor, an event-driven neuromorphic inference engine, on-chip SRAM and a set of spacecraft interfaces — retargeted from 28 nm FDSOI to 130 nm at a scale an open PDK and one person can carry (`docs/00-reference-brief.md`; `docs/01` section 2).

What exists in the repository is two designs and a corpus of measurements about them. The first design is a frozen accelerator slice submitted to a multi-project shuttle and pinned by hash (`docs/34` section 1). The second is a management subsystem begun after that freeze and kept strictly separate from it: an Ibex RV32IMC core in its `small-pmp` configuration — PMP, no lockstep — on a fabric that speaks Ibex’s own protocol internally and real AMBA 3 APB at the peripheral boundary, with a RISC-V CLINT, a GRLIB-style general purpose timer, a watchdog that escalates through a non-maskable interrupt to a system reset to an external pin, a console UART, a 16-pin GPIO port, a register-mode QSPI flash controller with two chip selects, a boot ROM that loads and verifies an image out of flash, and the frozen accelerator instantiated inside it over the same four serial pins the die will have (`docs/38`, `docs/39`, `docs/40`, `docs/41`, `docs/51`, `docs/65`, `docs/66`, `docs/68`). That subsystem has been synthesised, placed and routed whole, most recently as an eight-macro layout carrying the boot ROM’s error-correcting check macros, with zero detailed-routing violations, a unique Netgen LVS match, and all four geometric deck families run on it (`docs/67` section 11).

[Figure 1.1](#) is the part. It carries 168,592 instances, of which eight are the vendor SRAM macros and 168,584 are standard cells; the two largest attributed blocks are the Ibex core at 23,942 cells and the accelerator at 15,686, and 107,890 of the total are fill, decap and diode cells that belong to no block and are not drawn (`thesis/figures/layout_attribution.json`). Those per-block figures are the output of the attribution method described in the caption, and the method’s own 9.75 % unattributed remainder is the error bar that travels with every one of them. A reader who wants a census of a block should take it from that block’s synthesis record, not from this picture; what the picture is for is the question “where is the processor, where is the memory”, which no table answers as quickly.

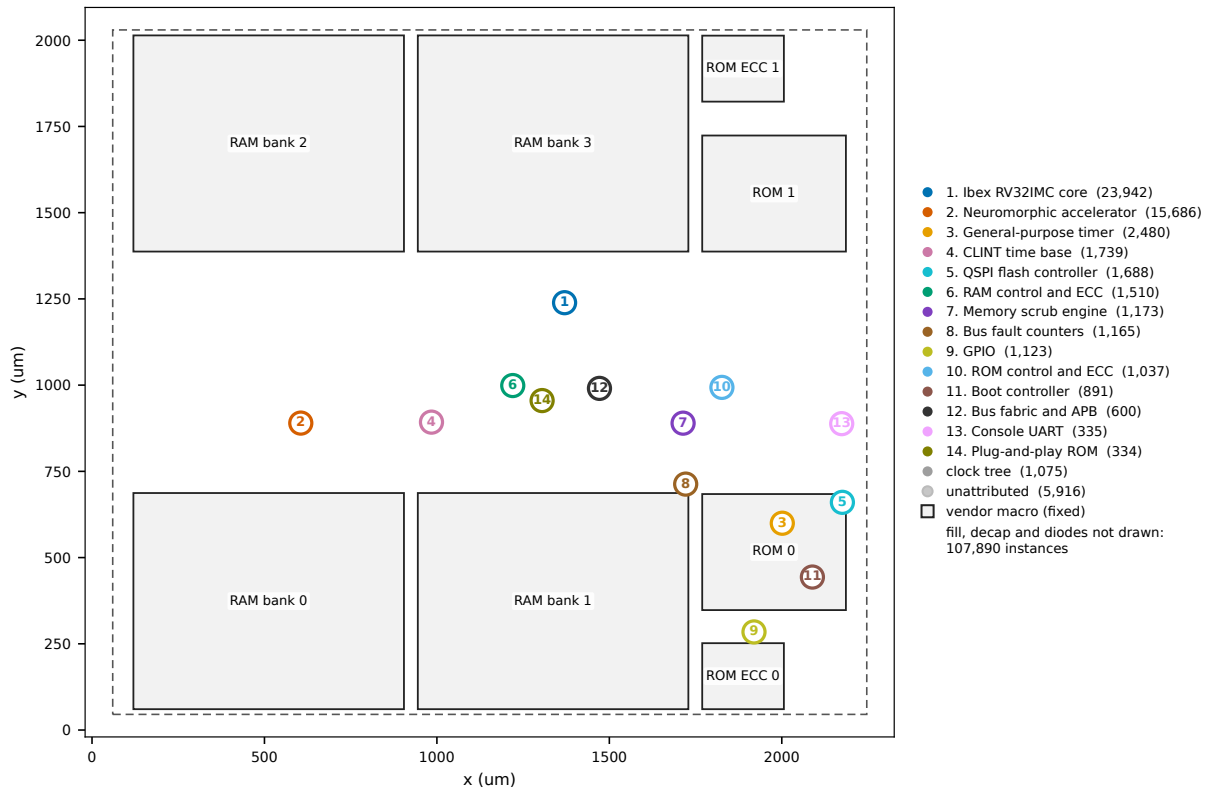


Figure 1.1: The whole die, with every standard cell coloured by the block it belongs to and the eight vendor memory macros drawn and labelled, read out of run `s83romecc5`'s own DEF and netlist — the eight-macro layout of `docs/67` section 11. The die is 2306.40 by 2075.22 μm , that is 4,786,287.408 μm^2 (`docs/59` section 3.1). Apart from the eight macros, which are fixed, nothing here was told where to go: the clusters are the placer's, not the designer's. Only about 7,700 of the die's instances still carry a hierarchical name after synthesis — the generator's `docstring` figure, beside a comment in the same file counting 1,140 instance names with a dot in them — so the block attribution is made by connectivity — nets keep their hierarchical names, each anonymous cell takes the block most of its pins' nets vote for, and two rounds of neighbour propagation follow — and the 9.75 % that never resolves is drawn grey and counted in the legend rather than hidden (`make_layout_figures.py` and `layout_attribution.json`, both in `thesis/figures/`). Read the per-block counts as an attribution with that error bar on it, not as a census.

1.2 What this is not

Four negatives bound everything that follows, and each is a matter of record rather than of modesty.

There is no silicon of this system-on-chip, and no tape-out of it. Nothing in the repository has been fabricated (`docs/00-index` section 2.2). The one design that has been submitted anywhere is the pilot of [Section 1.5](#), and even there the shuttle slot was still an open owner action at the time of writing, with the close at 2026-09-21 20:00 UTC (`ROADMAP.md` section 4 item 1). A full-SoC run is gated on two things the pilot cannot settle: the SRAM macro sign-off blocker of [Section 1.4](#), which is upstream, and a timing constraint the layout does not meet (`ROADMAP.md` section 1).

No clock frequency is claimed for this design. This is a rule, not an omission, and [Section 1.7](#) gives its three reasons. The consequence for the reader is that every timing result in this document is reported as a period, a slack and a corner name, in the words the tool used: the system-on-chip is constrained at a 20 ns period and misses it by -3.2029 ns on 2,003 endpoints at the `nom_slow_1p08V_125C` corner under a real 5 % derate (`docs/47` section 8.2), and on the layout that also contains the accelerator by -3.5789 ns on 2,175 endpoints (`docs/61` section 8.1). Those two numbers differ by 0.3760 ns, which is inside the roughly 0.4 ns of instrument resolution `docs/44` section 5.2 measured, so that difference is not reported as a regression in either direction (`docs/61` section 8.1). Neither of them is the current layout’s figure, and this document does not let them stand in for it: on the eight-macro run of [Section 1.4](#) the same corner reads -7.5758 ns on 3,529 endpoints, and -7.3492 ns on 3,550 after the antenna repair (`docs/67` section 11).

There is no radiation test data. No total-dose, proton or heavy-ion result exists for anything built here; the fault-injection campaigns are simulations of single-bit upsets, and the documents that report them say so in their own limitations sections (`docs/00-index` section 2.2). Every reliability statement in this work is therefore pre-silicon.

There is no flight heritage, and the reference has it. The positioning documents are explicit that the incumbent’s advantage is institutional: Gaisler’s GR712RC and GR740 fly **[fact]**, which makes the GR801 the default choice for a mission that must show qualification paperwork to an agency customer — an inference that document tags **[estimate]** rather than a measured share (`docs/05` section 1.1). This project does not compete with that and the positioning rules forbid implying that it does: the device is described as fault-tolerant by architecture on a commercial bulk 130 nm process, never as radiation-hardened, with a stated LEO class of 10–30 krad(Si) pending test data (`docs/05` section 4, rules 1 and 2).

One further caution belongs here because it is the sharpest disagreement inside the corpus. `docs/00-index` section 1 describes the work as “a clean-room retarget, not a clone”. `docs/14` section 11.2 records that the decision that would license that phrase — `docs/02`’s open question 2, whether Solderpad-licensed reference RTL was reused, used only as a golden model, or not at all — is *still open*, and states the consequence plainly: until it is answered, no public description of this accelerator may claim independent implementation. `docs/78` section 5 repeats it as a live item on the published mirror. This document therefore does not make that claim. It describes what the accelerator does, what it was verified against, and where its architecture came from, and leaves the provenance question where its owner left it: open, with a date on it.

1.3 The mission case

The argument for building this part at all rests on a gap in the market survey and on a workload sizing that was done late and changed the project’s priorities when it arrived.

The gap first. Across three tiers of space edge-AI silicon — qualified rad-hard SoCs, radiation-characterised COTS systems, and flight experiments with commercial neuromorphic parts — docs/05 section 2.3 finds no open-architecture device in any tier: no vendor publishes RTL, verification suites or fault-injection data, so a mission integrator cannot independently audit anyone’s fault-tolerance claims. It also finds no chip-level neuromorphic product below the rad-hard tier, between a qualified SoC in the tens of thousands of euro ([estimate], docs/05 section 1.2) and an unrated commercial part on a custom board. Both findings are recorded as facts by absence in that survey, the second explicitly kept under review, which is the correct strength for them. The demand side is not speculative: NASA flew Intel’s Loihi on the TechEdSat-13 3U CubeSat, and BrainChip’s AKD1000 has flown in low Earth orbit on Optimus-1 (docs/05 sections 2.1 and 2.3).

The workload sizing is docs/53, and it found the project had been asking the wrong question. docs/05 section 3.4 lists four candidate mission profiles, all tagged [estimate], and the only numeral anywhere in those four bullets is a power figure: orbit-average power in the tens of milliwatts (docs/53 section 3). Sized against a flown instrument — Falcon Neuro’s two event cameras on the International Space Station, whose paper reports 16,700 events/s time-weighted and 53,444 events/s recording-weighted with a standard deviation of 134,982 — the heaviest profile costs 10.23 % of the cycle budget at two standard deviations above the mean, the lightest costs 0.0044 %, and the latency profile is met by a factor of 76,551 (docs/53 sections 3.1, 6.1–6.4; every one of those percentages is an [estimate] derived from measured event rates and measured per-event costs). That document tabulates each profile at two candidate periods and the figures quoted here are the conservative column of the two; neither period is repeated in this work, for the reason Section 1.7 gives. The same document measured where the cycles actually go: the serial transport between the processor and the accelerator is 93–98 % of every workload measured, an architectural factor of 27 against a clock factor of 1.16 (docs/53 sections 4.2 and 5).

So the binding requirement is power, not throughput, and the power numbers have themselves been corrected twice. The system-on-chip’s sign-off run carried a `report_power` result with no switching activity annotated at all — 27.774, 34.835 and 46.095 mW at the slow, typical and fast corners (docs/53 section 7.1) — which five documents had described as “no power number” while the file sat in the run directory they were reporting from (docs/64 section 3.4). Annotated with the measured toggle rates of the one program in this repository that has a duty cycle in it — docs/46’s time-triggered supervisor, which idles in WFI between frames, and not the whole-SoC self-test, which is awake for 99.94 % of its cycles — the same layout gives 33.890 mW busy and 5.539 mW idle with the core’s clock gated in WFI, and an orbit average of 5.540 mW at the 0.0044 % duty cycle profile 1 was sized at (docs/57 sections 5.1, 5.2 and 11.1). The self-test is the cross-check rather than the source: its 120-cycle post-WFI tail measures 5.619 mW, 1.4 % from a supervisor window three hundred times longer (docs/57 section 5.2). That layout does not contain the accelerator (docs/57 section 3.2); on layouts that do, the idle figure is 8.287 mW with an orbit average of 8.289 mW (docs/76 section 8), and on the layout after that, 8.315 mW idle and 22.142 mW computing — with no busy figure published as a property of the design, because that document measures the busy total as a placement’s property and not a design’s (docs/77 section 9.5). Every one of those numbers is a pre-silicon flow estimate on a layout that fails setup at the slow corner and fails maximum slew and maximum capacitance, and docs/05 section 4 rule 5 requires that all of that travel with the number or none of it does.

The requirement the design does *not* meet is the fourth profile: long idle mission phases, where an activity-proportional part should idle at microwatts. docs/53 section 7.3 put the gap at four orders of magnitude and said nothing in the design was aimed at it. Both halves of that are superseded, by docs/57, which landed 2026-09-04 and whose section 12 items 2 and 4 name them: measured under a duty cycle the part idles at 4.5 to 7.1 mW across the three corners rather than 27.8 to 46.1, which puts the gap at about 2.4 orders of magnitude from “tens of microwatts”

and about 1.7 from “hundreds” ([**estimate**] for the orders, [**fact**] for the measured column), and there is a mechanism, it works, and it covers 80 % of the flip-flops (docs/57 section 11.2). The requirement is still not met, and it is worth saying here in the introduction because it is still the largest single unmet requirement in the record.

1.4 Why 130 nm, and why this PDK

Two questions, and the answers are different in kind: the node follows from arithmetic, and the particular kit follows from a radiation story and a memory story.

The arithmetic is docs/01 section 3. The reference carries 11.2 MB of on-chip SRAM. The bit-cell ratio between 28 nm and 130 nm is about $20\times$, but open-PDK memory compilers do not reach foundry array efficiency, so the effective memory-density gap between the reference’s platform and an open 130 nm flow is $70\text{--}90\times$ ([**estimate**], docs/01 section 3.2). Memory, not logic, sets the scaling factor: a 25 mm²-class die supports roughly 100–128 KB of SRAM, a 64–128 MAC event-driven fabric, an RV32 management core and the low-speed interface set ([**estimate**], docs/01 section 3.4). That figure has to travel with its PDK, because the document forbids quoting it otherwise: every 130 nm density in docs/01 section 3 is SKY130-derived, and on the SG13G2 kit actually selected the same section’s comparison puts the budget at 256–512 KiB, two to four times as much ([**estimate**], an assumed band pending docs/04 open question 2). A 1:1 retarget is absurd; a proportional one is not, because the architecture’s multi-pass execution is exactly the mechanism that lets a small fabric run networks sized for a larger one (docs/01 section 2.2).

The node is also where the cost argument lives. Standard-rate access to the selected process through Europractice is EUR 7,300/mm² with a 10 mm² minimum, so a minimum-size run is EUR 62k–73k and out of scope; the open-source MPW rate of EUR 1,000–2,800/mm² puts a 12 mm² MVP die at EUR 12k–34k, which brackets the comparable SKY130 commercial shuttle at \$14,950, and the whole MVP envelope including packaging, boards and an optional total-dose campaign lands at roughly EUR 20k–65k (docs/04 sections 5.1 and 5.3; the band on the realised open-source price is the single biggest cost unknown and that document says so).

The particular kit is IHP SG13G2, with SkyWater SKY130 as the fallback, and docs/04 section 2 gives five reasons in order. The first is that *the radiation story is the product*. IHP has published characterisation of its 130 nm family through an ESA paper covering the rad-hard sibling PDKs, and that data is a genuine asset — but the same section carries a correction dated 2026-08-29 that is more instructive than the data: an earlier revision had recommended putting a specific heavy-ion number into the project’s own claim sentence, and it was taken out, because the figure measures sibling-PDK standard-cell test vehicles and a reader of that sentence would reasonably take it as a property of the part being described. The published numbers stay in the sourced-facts list where their attribution travels with them, and the claim sentence says only “130 nm family with published ESCC-track TID and SEL characterization” plus this project’s architectural hardening. Nothing in the repository measures the single-event latch-up behaviour of anything this project has built. The second reason is the European institutional ecosystem; the third is that the open PDK [10] ships foundry compiled SRAM macros with a built-in BIST interface rather than compiler-generated cells (docs/04 section 3); the fourth is the cost argument of the paragraph above; and the fifth is that the fallback is unusually strong, because the flow is already silicon-proven on the fallback node [24], so falling back costs a retarget rather than a flow rebuild ([**estimate**] at 3–6 weeks, docs/04 section 2).

The third reason is also where the project’s largest external blocker lives, and Figure 1.2 is the picture of it. The macros the open PDK ships cannot be signed off by the open decks at the pinned PDK version. A trial harden containing one of them completed every flow stage and ended with 1,106,478 Magic DRC error boxes, 2,316 KLayout DRC errors and 359 Netgen LVS errors — and

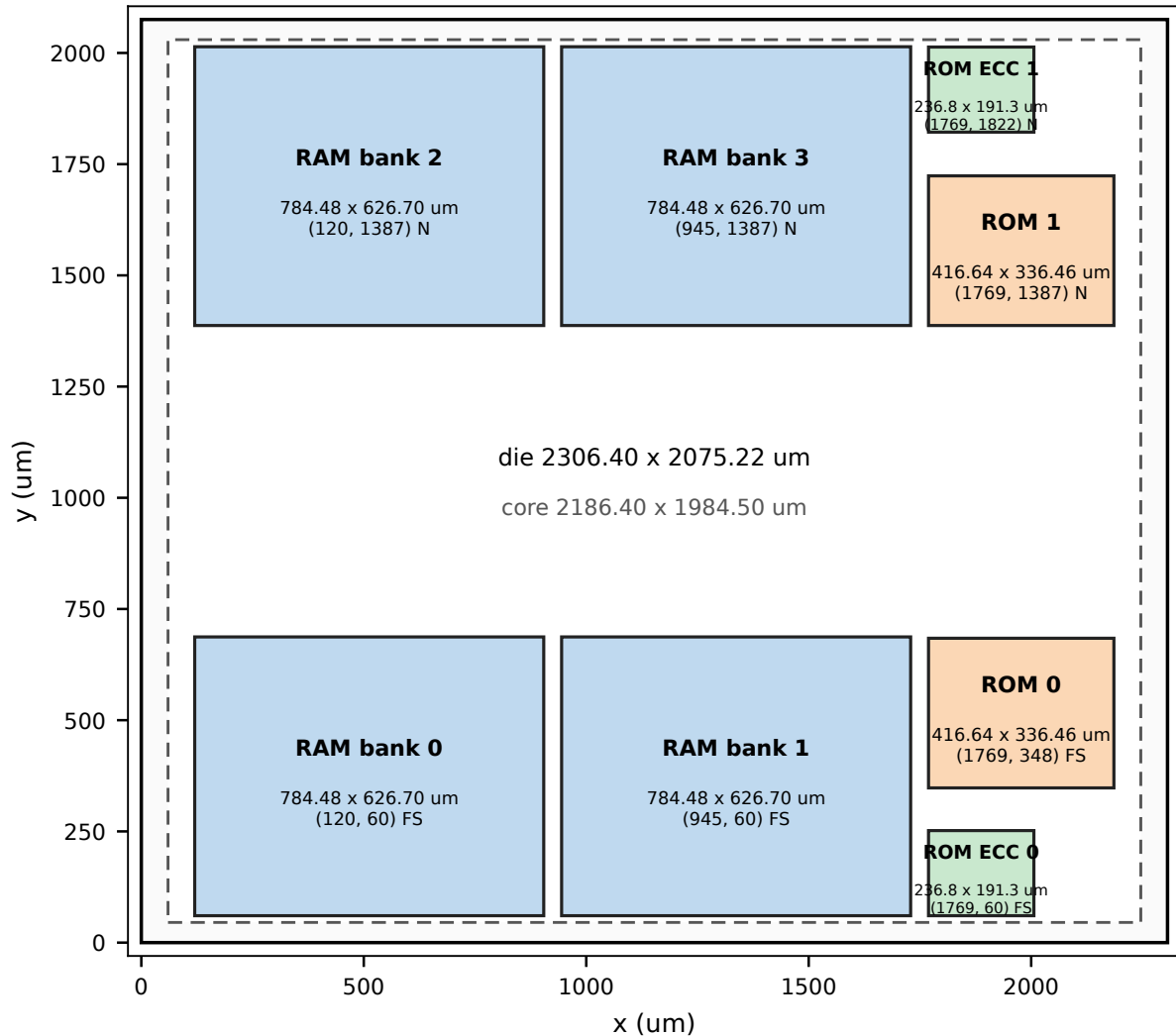


Figure 1.2: The die outline, the core area and the eight vendor SRAM macros with their names, sizes and coordinates, drawn from the sign-off run's own DEF (`thesis/figures/make_layout_figures.py`). Four RM_IHPSG13_1P_2048x64 macros carry the RAM, two 1024x32 macros the boot ROM and two 512x16 macros the ROM's check bits. These are the parts the open decks cannot sign off (`docs/12section 8`), which is why every geometric result in this work is reported as inside the macro hierarchy or outside it rather than as a total.

every one of the Magic boxes, and every KLayout item, was *inside the macro's own footprint*, with the standard-cell area, the power grid and the routing clean (`docs/12` section 7.5). That Magic figure is a count of error boxes and not of distinct violations: the flow's own report prints "Should be divided by 3 or 4" beneath the total, which puts the distinct count at roughly 277,000–369,000 (`[estimate]`, the report gives a range and not a divisor), and `docs/12` section 7.5 quotes the raw box count everywhere with that caveat attached. The conclusion does not depend on which end of the range is right: the smallest reading is still hundreds of thousands of violations against a required zero, and `docs/12` section 8 recorded a NO-GO for this PDK version thirteen days before the gate that asked for it (`ROADMAP.md` section 1). Two of the LVS causes were diagnosed in that document — hierarchy names that disagree between the CDL and the extracted layout, so Netgen flattens the macro, and bus index delimiters that disagree between angle brackets and square brackets, so none of the macro's 188 bus pins match — and the LVS half is an open upstream issue.

That is why the geometric results on the eight-macro layout are stated the way they are. On the `s83romecc5` layout, Netgen reports `Circuits match uniquely` with all seven LVS counters at zero (deck run `s831vsbb2`), read for exactly what it is: the vendor macro is a black box on both sides, so the match signs off the connectivity *into* the macros' pins and says nothing about their internals. The KLayout deck declares 173 rule categories in 47 groups and three fire, for 11,048 markers over 204 distinct cells — 6,584 distinct shapes, because two of the three rules are evaluated over the same derived layer and report an identical set — of which zero are outside the vendor macro hierarchy. The abstracted Magic run reports 170 error boxes on `s83romecc5`, 20 inside a macro footprint and 150 outside, and every one of the 150 lies within 0.500 μm of a macro edge, none in open routing area, because a LEF obstruction blanket is one shape more than 10 μm wide and ordinary routing beside it violates wide-metal spacing rules that the macro's real geometry would not (`docs/67` section 11). `README.md` summarises the run with 182 Magic boxes and 162 outside, which reads like a contradiction and is not one: 182 and 162 are run `s83ant`, the same layout with its two antenna violations repaired, and the 60 diodes that repair inserted moved routing inside the same halo — 20 inside a footprint in both, all 162 still within the same half micron, and the KLayout figures identical marker for marker (`docs/67` section 11). That is the shape [Section 1.7](#) is about, and it had to be applied to this chapter's own reading of its sources before it could be written down: two correct statements about two different inputs, quoted without saying which.

1.5 Two parts: a frozen pilot and a system around it

The work is deliberately split in two, and the strategy is stated in `ROADMAP.md` section 1: silicon-prove the PDK, the flow, a slice of the inference fabric and the hardening demonstrators on a cheap shuttle run before committing a full SoC.

The pilot is `tt_um_melihakbulut_nssoc`, a $6 \times 2 = 12$ -tile design on the TTIHP26b shuttle, die area 0 0 1289.28 313.74 μm , frozen at commit `b6738e5` (`docs/34` section 1). Its sign-off run closes setup at every corner — the slow corner at 1.2262 ns, quoted here to four decimals from a metric that carries more digits (+1.226180081066267) — with hold at 0.1029 ns, 33,564 instances at 48.7516 % utilisation, and zeros for Magic DRC, LVS, detailed-route DRC, antenna nets and pins, and total negative slack at all three corners, under a 5 % derate written into the flow's own constraint file (`docs/34` section 3.3). The Tiny Tapeout precheck passes 10 of 10 (`docs/34` section 7). Its measured upset behaviour is a campaign of 378 injections classified against the golden model: 97 masked (25.7 %), 193 corrected (51.1 %), 82 detected (21.7 %), 6 silent data corruptions (1.6 %), 0 hangs, in 445.7 s (`docs/64` section 3.1). At gate level, 370 of 370 comparable injections classify identically to RTL (`docs/32` section 5.1, quoted by `README.md`).

The point of `docs/34` is not the numbers but the pinning: it answers one question — *is what I*

am submitting the thing that was verified? — and answers it so the answer can be checked by running commands [15]. Ten RTL blobs by git hash, the submission tree by a 31-file manifest whose own SHA-256 is recorded, six flow configurations pinned by hash inside their own generator, and the built netlist, powered netlist and DEF bit-identical across four independently configured hardens of two configurations on two dates (docs/34 sections 2.1, 3.1, 3.2, 4 and 5.1). The GDS files differ between those runs and that is expected, because GDS carries an embedded generation timestamp: the GDS hash is a provenance record and not an equality criterion, and the geometric equality is established by the decks instead (docs/34 section 3.2).

What remains on the pilot is not engineering. The shuttle closes 2026-09-21 at 20:00 UTC, and the slot costs EUR 955 — twelve tiles at EUR 70 is EUR 840, plus a EUR 300 devkit subsidised to EUR 100 while any of the subsidised boards remain, plus EUR 15 of shipping (docs/37 section 5 items 1 and 2, with the price table both were read from in that document’s section 1). Both the date and the price are owner-side facts read from the vendor’s own published records on 2026-08-31, and the close carries a time the project’s earlier documents did not record: 20:00 UTC, not local midnight (docs/37 section 1). Buying the slot is the one item in ROADMAP.md section 4 that a person must still do.

The system-on-chip under `hw/soc` is everything else. It was begun after the pilot froze, it instantiates the frozen pilot rather than copying it — `hw/rtl/pilot_top.v` inside `soc_top.v`, reached over the same four serial pins the die will have — and a program running on Ibex out of the boot ROM configures it, loads its weights, feeds it a six-frame event stream and reads the spikes back against an answer the golden model computed at build time: 27 checks, fail mask 0 (docs/51). The transport is serial because that is what the die has, which costs 176 clock cycles per register access (docs/53 section 4.1) and buys exactly one implementation of the die’s host protocol, the one in the system’s own regression.

The freeze rule is what keeps the two apart. docs/34 section 9 states what does and does not void it: documents, tests and analysis over existing run trees do not; any non-comment change to `hw/rtl`, to the submission sources or to a pinned configuration forces a full re-harden and a gate-level re-run. That rule has been exercised twice — once when two checkers were bound to all corners, which forced a re-harden that reproduced the superseded runs on 196 of 196 and 194 of 194 metrics with the netlist bit-identical, and once when the licensing decision added SPDX headers to every file in `hw/rtl`, where all ten blobs moved and the netlist did not (docs/34 sections 1, 2.1 and 3.3). Both times the record was amended in place with every superseded hash retained, because a freeze record that can only be checked against *now* is not a freeze record.

1.6 Where the work is published

The development repository is private and the published tree is a generated mirror of it. docs/78 section 1 takes that decision and records what it costs: `scripts/gen_public_mirror.py` writes the published subset into a directory and makes one commit naming the source revision, it does not push and it does not create a repository, so publication stays an act of the owner. The alternative — making the development repository public — was declined for one honest reason: three hundred-odd commit messages written over three weeks have not been audited, and one missed message is not recoverable after a push (docs/78 section 1.1). The mirror’s cost is stated in its own README.md rather than left for a reader to notice: you cannot see how the design got here from the published repository, only where it arrived. What makes that cost bearable is the corpus, which records the withdrawn claims and the re-attributed measurements that a commit log would not hold — the history is published, it is just not in `git log`.

Three licences apply, one per kind of thing, decided in docs/14 and signed on 2026-09-09 (docs/14 section 11): CERN-OHL-W-2.0 for RTL, testbenches, constraints, formal jobs and flow

configuration; Apache-2.0 for generators, golden models, harnesses, flow drivers, analysis and CI; CC-BY-4.0 for the documents in `docs/` and the measurement data. The reasoning is in `docs/14` sections 6.1–6.3 and the load-bearing part of it is that the outbound choice was *unconstrained*: every inbound licence that touches a tracked file is permissive, because third-party trees are fetched into gitignored directories rather than vendored, so inbound obligations attach in only two tracked places — the Apache-2.0 Tiny Tapeout template scaffolding under `tt/`, and two corrected copies of riscv-formal’s own instruction models, which carry upstream’s ISC notice inline (`docs/14` sections 2.1, 2.2 and 5.0). The reciprocal licence for hardware was chosen against the permissive one for a stated reason: the project’s claimed differentiator is auditability, and a licence that lets a derivative be closed undercuts the claim (`docs/14` section 6.1).

Three things are held back from the mirror, all commercial and none of them technical: one section of the positioning document, the funding and shuttle document, and the drafted text of an unsubmitted application. Nothing technical is held — no negative result, no withdrawn claim, no measurement, no failing check — and `docs/78` section 2 states why that is a condition rather than a courtesy: a record that publishes only what worked is a brochure. Each held file is replaced by a stub that keeps the filename and says what was held, so that the corpus’s own cross-reference test still passes in the generated tree, which it does: 90 of 90 when that section was written (`docs/78` section 2.1), and 95 by the time `docs/78` section 7 re-ran it a day later, because it is a count of documents and not of findings. The mirror was first published on 2026-09-10 at 518 files from source revision 93757c9, and republished on 2026-09-11 at 529 files from 26f4494 (`docs/78` sections 5 and 7).

1.7 The rules this work holds itself to

A reader who takes nothing else from this chapter should take these four. Each exists because the project broke it once and wrote down what that cost.

A superseded measurement is reported as superseded, with its date, and is not rewritten. This is `docs/64` section 2’s method: the original sentence stays where it was, struck through where a short phrase is wrong or left whole and followed by a dated note, and the note says what the value is now, why the old one was written, and which file settles it. The document that established it was reconciling seven figures the corpus stated two ways, and its first verdict is the reason the rule is worth keeping: *none* of the seven was a transcription error. Every disagreement was two correct statements about two different inputs — an earlier run, a different source list, a different content — quoted without saying so (`docs/64` section 1). The pilot’s fault-injection campaign is the clearest case: 335, 366 and 378 are all real logs of the same file at three commits, and the history of that one file settles which is the campaign of record without making the other two wrong (`docs/64` section 3.1). Where this document reports a figure that a later measurement has moved, it reports both and says which is which; where two documents in the repository still disagree, as with the Magic box count in [Section 1.4](#), it says so and cites both.

No clock frequency is published for this design. `docs/53` section 9.2 refuses it in three ranked reasons. The first is that the only available number is the frequency a layout happened to reach, and a requirement written in the units of the answer is not a requirement — the derived requirement in that document is not a frequency at all, it is a count of serial frames per second, and at the geometry that exists the fabric caps that count long before the clock does (`docs/53` section 6.5). The second is `docs/05` section 4 rule 5: any figure published externally must be traceable to measurement or clearly labelled a design target, and the available figure is neither, because it is the derived maximum frequency of a run that fails setup on 2,003 endpoints, fails maximum slew at the slow corner and fails maximum capacitance at all three. Half of that second reason has since been superseded and the half that governs has not: the same sentence in

`docs/53` section 9.2 also said the layout had never seen Magic DRC, KLayout DRC or Netgen LVS, and all four deck families have run on `soc_top`'s own geometry since 2026-09-11 (`README.md`, citing `docs/77` and `docs/79`); the slew and capacitance failures are unchanged, and they are the half the rule now rests on. The third reason is that two parts in one project would carry two clocks and one of them is being fabricated, and a reader offered both would ask which one the project's clock is. What may be said instead is the stronger claim anyway: the mission profiles have been sized, three of four are met with two to five orders of magnitude of margin, and the fourth is a power requirement the design does not meet with the mechanism named (`docs/53` section 9.4).

timeout, killed and error are three different words. `docs/63` section 16 spends thirty lines on the distinction after a formal job was reported as a timeout and its own log said otherwise: the job had been sent a signal from outside the run at 1 h 41 m and the tool recorded `DONE (ERROR, rc=16)`, while no timeout had been configured for it to reach. The reading each word licenses is different. `TIMEOUT` means a bound was set, the job reached it, and the measurement is complete — *this engine did not close in that much time* — and the jobs entitled to that word wrote `rc=124` into their wall-time files, which is the `timeout` utility saying it sent the signal itself. An error from an outside kill means no bound was set and no measurement was completed, and calling it a timeout would silently promote an interrupted run into a finished experiment. The corpus has made that mistake in both directions, and the fix is the same either way: read the runner's return code before choosing the noun. The general form — “nothing came” and “I arrived late” are not the same result — is applied to a testbench monitor in `docs/81` section 4.1, which attributes the rule to `docs/64` where the passage itself is in `docs/63`; both are cited here because they disagree about the citation and not about the rule.

A check must test the property, not the name. The sharpest statement of this is `docs/78` section 3, and it is sharp because both of its instances were in the project's own instruments rather than in the design. The licence checker searched the first 4,000 characters of a file for an identifier string in any context, so it read its own docstring as its own licence header and reported zero missing while carrying no header at all; it now counts a tag only on a comment line in the first twelve lines, which is where a header is and where prose about headers is not. The mirror generator carried a guard meant to refuse to run if a workflow file no longer contained a sentence it rewrites — written as “if the file is in the set”, so when the file was *deleted* the guard did not fire, it was skipped, and the generator wrote 511 files as though nothing were owed. `docs/78` section 3.2 records the conclusion in one line: a check that cannot fail when the thing it checks is absent is not a check. That was the third instance of the shape in this repository's own instruments and the first found in a guard written to prevent it. The question is never “did the check pass”, it is “what could the check have seen”. The corresponding rule when a check is found to be narrow is `docs/50` section 8.1's: *extend, do not relax*.

Underneath all four sits a labelling convention used in every document and carried into this one. **[fact]** means measured in this environment or read out of a file; **[estimate]** means derived or judged; **[planned]** means an intention with a date attached and nothing behind it yet (`docs/00-index`, and the same three words at the head of most documents in the corpus). Where a source marks a statement with one of these, this document carries the mark rather than flattening it.

1.8 What this document is

This is a record of measurements taken with open tools on a design that exists in register-transfer and layout form. It is not a qualification report and nothing in it should be read as one.

Concretely: there is no beam test, no lot, no screening and no qualification body [6]; the fault-injection results are simulated single-bit upsets, not measured event rates [21]; the timing, power

and area figures are pre-silicon flow results on layouts that are explicitly named where they fail; and the memory macros the design depends on cannot be signed off by the open decks at the pinned PDK version, so the die that carries them cannot be either ([docs/67](#) section 11). The claims this work does make are of a different kind — that a given mechanism is present in the shipped netlist rather than only in the source, that a given property holds over all reachable states, that a given campaign classified a given number of injections a given way, that a given deck found a given number of markers in a given place — and each of them is quoted here against the document that measured it, which in turn names the command and the run tree it was read from.

1.9 Road map

The remaining chapters are organised by what they measure rather than by the order in which the work happened.

The architecture chapter describes the system as built: the bus, the memory map frozen as a single generated source, the interrupt and timing subsystem, the watchdog and its escalation ladder, the peripherals, and the boot flow that initialises every word of an error-correcting RAM before anything reads it — because a memory whose rows carry check bits powers up with an uncorrectable word everywhere, not a zero. Its sources are [docs/38](#) through [docs/41](#), [docs/65](#), [docs/66](#) and [docs/68](#).

The accelerator chapter describes the event-driven inference engine, the register map generated from a single source, the serial transport to it and what that transport costs, drawing on [docs/02](#), [docs/10](#), [docs/51](#) and [docs/53](#).

The hardening chapter prices every protection mechanism separately, because the project's discipline is that nothing is protected before it is measured and every protection reports what it cost: triple-modular redundancy on the state that cannot recover, SECDDED on the memories and the architectural register file, a scrub engine, and fault counters an operator can read. Its sources are [docs/43](#), [docs/44](#), [docs/55](#)–[docs/58](#), [docs/67](#) and [docs/69](#).

The verification chapter covers the five layers and what each can and cannot see: the cocotb regressions, the *guards* that read the tree without simulating it, the SymbiYosys property sets, the riscv-formal bring-up against the ISA, and the seeded fault-injection campaigns at register-transfer and gate level — every one of them judged against the bit-exact golden models that are the specification ([docs/09](#), [docs/11](#), [docs/16](#), [docs/32](#), [docs/42](#), [docs/52](#), [docs/63](#), [docs/74](#), [docs/82](#)).

The physical chapter takes the design through the flow: the floorplan and the decision behind it, the macro placement, synthesis, place and route, the geometric decks and their macro-scoped results, and the reproducibility record ([docs/12](#), [docs/45](#), [docs/47](#), [docs/48](#), [docs/59](#), [docs/61](#), [docs/71](#), [docs/77](#), [docs/79](#)). The layout figures introduced here are read in detail there.

The results chapter puts the measurements side by side: what closes, what does not, what each mechanism cost in area and in slack, and what the power figures say once real switching activity is annotated ([docs/57](#), [docs/60](#), [docs/76](#), [docs/77](#)).

The concluding chapter states what remains open, in the terms the record uses: the upstream macro blocker, the unmet idle-power requirement, the serial transport that [docs/53](#) section 9.3 ranks above timing, the provenance question of [Section 1.2](#), and the one measurement nothing here substitutes for, which is silicon.

Chapter 2

Architecture

The part is `soc_top`: a 32-bit RISC-V management core, a system fabric, a memory subsystem with a code over every stored bit, a core-local interruptor, a general-purpose timer carrying a three-stage watchdog, a boot controller, two telemetry blocks, a console UART, a GPIO port, a QSPI flash controller, two plug-and-play device tables and a spiking-neural-network node built around a frozen shuttle die (`docs/60`, revision 0.2, 14 September 2026). Its datasheet gives its status in three words: preliminary, pre-silicon, pre-sign-off.

One sentence travels with the rest of this chapter. **This design publishes no clock frequency**, by a rule adopted in `docs/53` section 9.2 and restated in `docs/60` section 0: the 20 ns period below is the constraint the synthesis and place-and-route flows were run against, and every slack here is measured against it. It is not converted into a rating and no rating is implied.

There is no block diagram in this chapter: [fig. 2.1](#) shows where the placer actually put each block, and the floorplan and the density map belong to the physical-design chapter.

2.1 The management core

The core is lowRISC Ibex in the `small-pmp` configuration: RV32IMC with physical memory protection at four regions, a two-stage pipeline, a flip-flop register file, no instruction cache, no branch-target ALU, no writeback stage, no branch predictor. `hw/soc/rtl/soc_top.v` sets `BaseIsa` = 0 (RV32I), `RV32M` = 2 (RV32MFast), `RV32B` = 0, `RV32ZC` = 0 (Zca), `RegFile` = 0 (flip-flop), `PMPEnable` = 1, `PMPNumRegions` = 4, `PMPGranularity` = 0, `MHPMCounterNum` = 0 and `SecureIbex` = 0.

The core is vendored pristine at commit `34b0705760ef3dfa00e99637432473d2be8f22f3`, Apache-2.0, and converted to Verilog-2005 by `sv2v`: 30 of 30 RTL modules and 8 of 8 vendored primitives, 15,873 lines, zero errors and **zero patches to Ibex** (`docs/38` sections 2, 4.2). **No equivalence between the SystemVerilog and the converted Verilog was proven**, because upstream's own equivalence script drives a proprietary tool this project does not have. Two corroborations were run instead — two independent SystemVerilog front-ends landing within **0.89 % of cell area and two flip-flops**, and a self-checking program executing in Icarus — and `docs/38` section 5 refuses to call either a proof.

PMP was not taken for free. `docs/09` part B track 3 had already chosen a software architecture whose isolation mechanism is PMP, so a core without it does not implement the design already decided. It costs **43,634 μm^2** , +18.8 % and **1.21 ns** of pre-layout closing period, because the address-match logic is replicated per region and sits in the load/store path (`docs/38` sections 8.4, 8.6).

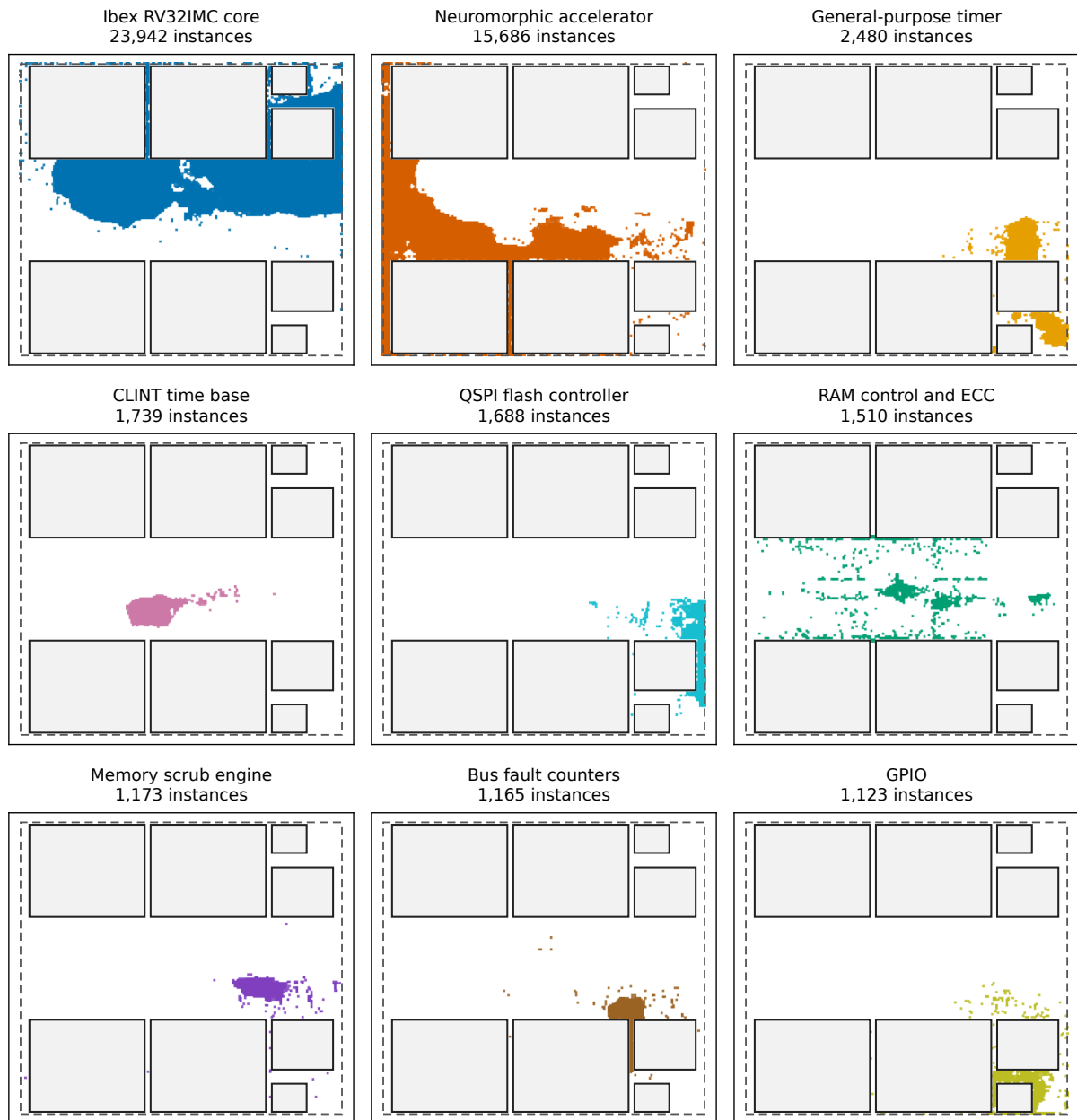


Figure 2.1: Where each block landed. Nine small multiples, one per block, each drawing that block’s standard cells against a grey die with the eight vendor memory macros outlined. Hierarchy barely survives this flow’s synthesis: of the 168,592 instances in the sign-off run’s DEF — 168,584 standard cells and the eight macros — only 1,140 still spell a dot in their name, 632 of them the RAM’s, and by the script’s own prefix test exactly one carries a top-level block name. The rest are attributed by connectivity — nets keep their hierarchical names, each anonymous cell takes the block most of its nets vote for, and two rounds of neighbour propagation follow (`thesis/figures/make_layout_figures.py`). The method leaves a remainder: **5,916 of the 60,694 logic cells, 9.75%, are unattributed** and are drawn grey rather than hidden (`thesis/figures/layout_attribution.json`). Nothing here was placement-constrained except the macros, so the clusters are the placer’s.

2.1.1 Why there is no lockstep

SecureIbex is 0: no shadow core, no output comparison, no dummy instructions, no reset-all flops, no memory-interface ECC. In this Ibex those are not separable. `ibex_top.sv` lines 212–216 make Lockstep, ResetAll, DummyInstructions and RegFileLockstepECC localparams of SecureIbex, and MemECC defaults to it at line 41 (docs/38 section 3.3). It is one switch, and it was priced before it was thrown — each configuration synthesised at its own closing period (docs/38 section 8.4):

Configuration	Cells	Flip-flops	Area, μm^2	kGE
small, no PMP	15,736	1,950	232,049	31.97
small-pmp, chosen	18,921	2,106	275,683	37.99
small-pmp-sec	40,236	4,975	585,233	80.64

Lockstep is $+309,550 \mu\text{m}^2$, $+112\%$ over the chosen core and costs almost nothing in closing period, 0.27 ns, because the shadow core is parallel rather than in series (docs/38 sections 8.5, 8.6).

Area is not what decided it. Lockstep *detects* where this project’s voters *mask* (this Ibex’s lockstep is one shadow core and a comparison; a triple core lock-step votes and therefore masks as well [11]); the two cover disjoint structures, so their costs add rather than share; and the one real overlap, ECC over the register file, duplicates a codec this project already owns. MemECC follows SecureIbex, so the instruction and data buses would become 39 bits wide all the way to memory with the check bits *stored* rather than regenerated: a 22 % SRAM overhead on every byte the CPU touches. **The owner chose small-pmp on 2026-08-31** (docs/38 section 10 item 4), and the same section records what the choice takes on: the management processor becomes the least hardened block in the design, its own fault-injection campaign stops being deferred work, a recovery policy becomes required rather than optional, and the decision is cheap to reverse only before floorplanning.

What has happened since: the campaign was run, the register file gained a (39,32) SECDED codec with a background scrub at $+38,050.5762 \mu\text{m}^2$ and $+222$ flip-flops (docs/43), and the watchdog is the recovery policy. The core as shipped is **22,338 cells, 2,328 flip-flops, $316,051.6590 \mu\text{m}^2$** (docs/45 section 6.2); everything in Ibex except the register file remains unprotected, by decision (docs/60 section 9.8).

2.2 The bus fabric

Ibex presents two masters, an instruction port and a data port, each speaking a simple request/-grant/response protocol; it has no AHB, no AXI and no burst. The interconnect takes that protocol as the internal fabric and puts *real* AMBA 3 APB at the peripheral boundary. **There is no AHB in this SoC and none is claimed** (docs/39 sections 3, 4, 7).

The obvious argument, area, is the one the record refuses. An honest AHB-Lite cost probe was built and synthesised at **$17,154.32 \mu\text{m}^2$ against $14,807.32$** , $+15.9\%$ on the fabric and $+0.85\%$ of the CPU. Nine tenths of one per cent is not a reason to reject a bus; what the measurement is good for is excluding the opposite mistake, that the decision was made by picking the smaller thing (docs/39 section 6.3).

Five arguments decided it. docs/08 section 3 defines proximity to the reference family at the software-visible layer — addresses, register maps, slot geometry, discovery tables — and the wires between core and RAM are not in it. Not one peripheral core the IP survey selected is AHB-native, so every one needs a wrapper whichever bus is chosen, and an APB wrapper is the smaller one. docs/08 row 1’s “single AHB-lite layer” with three masters is a category error, because **AHB-Lite has exactly one master by definition** and Ibex alone supplies two. And AHB folds “your request was accepted” and “your response is here” onto one wire, so an arbiter that

back-pressures a master simultaneously tells it its outstanding data phase has not completed; the probe parks the response instead, and **67 of the arbiter’s 71 flip-flops are parking registers**. Evidence settles the rest: nothing in the pinned toolchain is an AHB protocol checker, while APB’s eight-odd rules are proved by k-induction against ghost registers independent of the bridge’s own storage (docs/39 section 5.3).

soc_bus.v carries two masters onto **six slave ports** — ram, rom, clint, npu, apb, pnp — plus an internal error slave (docs/60 section 5.2). Three of its properties are decisions. It allows **two outstanding requests per master**, because Ibex’s prefetch buffer issues two and one would halve instruction-fetch bandwidth. **All of a master’s outstanding requests must go to one slave**, because Ibex requires in-order responses and slaves differ in latency; the alternative was a reorder buffer that must store a whole response, and the restriction costs one dead cycle when a master changes region. And arbitration is **round-robin**, which makes no-starvation a property of this file rather than an argument about the core: the measured worst gap between grants is 2 cycles against an asserted bound of 8.

An unmapped access is a **bus error, never a read of zero**: the error slave is inside the fabric, always ready, and Ibex turns its response into an access fault. Properties F1–F9 prove one-hot decode against the generated map constants, the outstanding limit and that no response is lost or duplicated. **Neither formal job proves liveness**: nothing says a transfer ever completes (docs/39 sections 5.3, 5.5).

On the one recipe every block here is measured with — Yosys stat -liberty on ihp-sg13g2 typical, abc at a 20 ns target, one gate equivalent = $7.2576 \mu\text{m}^2$ (docs/45 section 6) — the fabric is 868 cells, 55 flip-flops, **10,928.8872 μm^2** , 3.96 % of the unprotected core, and the APB bridge 201 cells, 94 flip-flops, $6,437.4912 \mu\text{m}^2$ (docs/60 section 5.2).

2.3 The memory map, and its single source

regmap/memmap.yaml is the memory map. It is not a table in prose: regmap/generate_memmap.py emits the reader’s document, the Python constants the test suites use, the RTL’s address decoder, both device tables’ contents, the C header the bare-metal programs include and the linker’s MEMORY block, and sw/tests/test_memmap.py fails if any is stale (docs/39 section 2.1).

Three rules are enforced by the generator rather than commented. Ibex resets to {boot_addr_i[31:8], 8’h80}, so the generator derives the reset vector and refuses a boot region that does not contain it or cannot execute it. Ibex’s mtvec is 256-byte aligned and vectored-only, so **every region base must be 256-byte aligned** and the map is never itself why a trap vector lands elsewhere. And every region is a naturally aligned power of two with no overlaps, so decode is a prefix compare rather than two magnitude comparisons in the address path of every access.

Table 2.1 is the map, and every base is taken unchanged from docs/08 section 3.1, which took them from the reference family — except the NPU window, the one row that document marks project-specific. The RAM is **32 KiB, not 64**, since the memory-protection work: the region shrank rather than the macros growing, and section 2.4 is why. docs/60 section 5.3 carries 64 KiB in its revision-0.1 body with a dated correction beside it; the map is the normative source.

2.4 The memories

There are two implementations of one module name, and which one a reader is looking at decides what the numbers mean. soc_mem.v is behavioural and is what every simulation and formal job reads; soc_mem_sram.v is a drop-in with the same module name and port list built on real vendor macros, and **no simulation ever reads it**.

Table 2.1: The system-bus address map, from `regmap/memmap.yaml` and its generated form `docs/memmap-soc.md`. Ten regions, six implemented. Everything outside every region reaches the error slave.

Base	Size	Name	Port	Status	What it is
0x00000000	32 KiB	RAM	<code>ram</code>	implemented	System SRAM, SECDED per byte lane, scrubbed
0x10000000	256 MiB	NPU	<code>npu</code>	implemented	Accelerator window; sixteen 4 KiB node windows
0xC0000000	8 KiB	ROM	<code>rom</code>	implemented	Boot ROM; reset vector at 0xC0000080
0xD0000000	32 MiB	QSPI3	—	reserved	Flash execute-in-place, 3-byte addresses
0xD8000000	128 MiB	QSPI4	—	reserved	Flash execute-in-place, 4-byte addresses
0xE0000000	64 KiB	CLINT	<code>clint</code>	implemented	<code>msip</code> , <code>mtimecmp</code> , <code>mtime</code>
0xF8000000	4 MiB	PLIC	—	reserved	Interrupt controller; section 2.5.1
0xFE000000	16 MiB	DEBUG	—	reserved	RISC-V debug module; there is none
0xFF900000	1 MiB	APB	<code>apb</code>	implemented	Peripheral bridge, sixteen 4 KiB slots
0xFFFFF000	4 KiB	PNP	<code>pnp</code>	implemented	System-bus device table

The macro arrangement, confirmed from `soc_mem_sram.v` and `docs/67` sections 3, 8, is **eight macros**: four `RM_IHPSG13_1P_2048x64_c2_bm_bist` for the RAM, the smallest byte-writable 64 KiB build the PDK [10] admits (the denser 8192x32 part has no byte-mask port, and Ibex emits `sb` and `sh`); two `RM_IHPSG13_1P_1024x32_c2_bm_bist` for the ROM, because there is no 2048x32 part; and two `RM_IHPSG13_1P_512x16_c2_bm_bist`, one per ROM bank, holding the (39,32) word code’s seven check bits at $+90,618.62 \mu\text{m}^2$. The check pair was designed unplaced and was **placed and routed on 2026-09-13** in run `s83romecc5`, with zero detailed-routing DRC errors and a die that does not grow by one database unit. The estimate that the die would have to grow by about $200 \mu\text{m}$ in x — made about the single `RM_IHPSG13_1P_1024x16` that was the alternative, $236.80 \mu\text{m}$ wide against the $60.48 \mu\text{m}$ `docs/61`’s floorplan leaves beside each ROM macro — was **superseded on 2026-09-13** and is kept rather than deleted: the space the check macros needed was vertical, in the pocket the shorter ROM macro leaves in its own row, so the estimate had measured the wrong axis. The pair was never its subject; `docs/67` section 8 costed the pair into that pocket from the start.

2.4.1 Why the check bits cost half the RAM

A 2048x64 row is 64 bits and holds two 32-bit words. The project’s SECDED code is (72,64) — eight bits *wider* than the row — and a (39,32) word code needs fourteen spare bits per row. **There is no bit-widening in a hard macro**, so the check field goes into another macro or into the space a narrower word leaves. `docs/67` section 3 costs six options from the vendor LEF, and the three that keep 64 KiB all buy it with a read-modify-write: a seventh 2048x64 for the (39,32) word code at $+491,633.62 \mu\text{m}^2$, $+21.88\%$ of the six-macro area, with one on every `sb` and `sh`; one 4096x16 for a (72,64) code over the whole row at $+257,608.51 \mu\text{m}^2$, $+11.46\%$, with one on *every* store; and four more 2048x64 at $+1,966,534.46 \mu\text{m}^2$ to avoid it. That is a design change and not a wrapper: a second port cycle per store, a hazard against the pipelined request the fabric permits, and a grant the memory would have to withhold.

What was built holds **one 32-bit word per 64-bit row as four (16,8) byte codewords**, which is exactly 64 bits. Each is the frozen (72,64) Hsiao code [9] with data bits 8–63 tied to

zero; three of the eight check rows are then identically zero, so what is stored is a (13,8) code plus three constant bits, and (13,8) is the minimal SECDED width for a byte. A byte write lands through the macro's own per-bit write mask with its check bits beside it: no seventh macro, no die growth, no read-modify-write, and the response stays one cycle. What the 32 KiB that went were holding was measured rather than assumed: the 28-check bring-up program uses **192 bytes** of static data plus a stack, and the RTL keeps the two-words-per-row arm for the day a sized workload wants 64 KiB back (docs/67 section 3.4).

Every read decodes the row on the way out: a single-bit error in any lane is corrected before the read data and counted, and an uncorrectable syndrome answers with a bus error that Ibex takes as an access fault, so **the recovery is a trap and not a wrong word**. A scrubber [22] sits behind the row port rather than on the fabric — a third bus master would have been a re-proof of the arbiter and its ownership queues — reading one row in an idle cycle, examining it in the next, and writing back only the lanes the decoder corrected. **The bus always wins**: if the bus asks in the examine cycle the scrub result is dropped, so a bus write between a scrub read and its write-back cannot be overwritten.

The protection costs $+31,391.3124 \mu\text{m}^2$, $+4.783\%$ on the whole SoC at the SRAM boundary, taking it from 43,884 cells and 5,599 flip-flops to 45,952 and 5,833 (docs/67 section 5.1). Routed, the read-return group's worst slack at `nom_slow_1p08V_125C` falls from -2.7112 to -4.0021 ns, and it is still not the binding path. Two limits close the section: the row address into either memory is **unprotected**, since a corrupted index reads the wrong row, which is a valid codeword; and on this PDK the boot ROM is an SRAM that powers up undefined and nothing in the design can write it, so check bits on a ROM nobody can write are check bits nobody can initialise (docs/67 sections 8, 9).

2.5 Time, interrupts and supervision

The CLINT and the general-purpose timer are both timers and are not duplication. **The CLINT owns time**: `mtime` is a free-running 64-bit up-counter that is never reloaded and never restarted, and it is the only time base with an architectural meaning. **The GPTIMER owns intervals**: its timers count down from a reload value and restart, which is what periodic work and timeouts want, and its last timer is the watchdog. Neither is a convenient way to do the other's job, and the two have independent time bases that nothing synchronises (docs/40 section 4).

The CLINT sits on the system bus rather than behind the APB bridge, which costs $+1,421.92 \mu\text{m}^2$, $+0.52\%$ of the core — the difference of two measurements, and tagged an estimate where it is published (docs/40 section 9) — and buys a one-cycle `mtime` read instead of the bridge's three-cycle minimum on the one register software polls in a loop. It has the standard layout — `msip` at `0x0000`, `mtimecmp` at `0x4000`, `mtime` at `0xBFF8` — one hart, and a bus error at every other offset. One behaviour is deliberately not papered over: the naive two-store 64-bit `mtimecmp` update glitches, and the cocotb suite *asserts* that a spurious interrupt occurs on it rather than only checking the safe sequence.

`mtime` is the one block here whose hardening is a code rather than a copy. docs/58 makes its 64 stored bits the data field of a (72,64) SECDED codeword, decoded and corrected on every tick, at **eight added flip-flops** and $+7,026.8310 \mu\text{m}^2$. It corrects: 144 of 144 single-bit injections into the 72 stored bits came back corrected, against an unprotected design that displaced the clock in 95 of 128 draws and announced none. That document's own negative result sits beside it — a residue-mod-3 detector measures **98.694% of the corrector's area** and corrects nothing — and `mtimecmp` gets nothing, because 128 injections into it displaced the clock zero times.

The CLINT's area is published twice and both numbers are right: docs/40 section 9 reports 1,038 cells and $16,683.48 \mu\text{m}^2$, docs/45 section 6.2 reports 1,043 and $16,712.8164$ for the same unchanged

file, and the difference is the source list the script read on the day. The second is the one to quote for the SoC as it stands.

2.5.1 No platform interrupt controller

Thirteen interrupt sources are defined and Ibex has fifteen dedicated, individually vectored, individually maskable, priority-ordered fast local interrupt inputs, each with its own vector-table entry. That is an interrupt controller, in the core, at no cost. **No PLIC is built:** the region stays reserved and faults, `irq_external_i` is tied low and left free so that adding a controller later is a wiring change, and **lines 13 and 14 are unassigned** (`docs/40` section 3). The condition that reverses the decision is mechanical: the generator refuses a line outside 0–14 and refuses two sources on one line, so the first peripheral for which no free line remains is a build failure naming the decision, and `sw/tests/test_memmap.py` recomputes the two spare lines from the map rather than asserting them.

The map keeps **two interrupt namespaces** that must not be conflated: `irq` is the plug-and-play source number in the device table’s identification word, running to 24, and `line` is the Ibex wire index, running to 12. Four core inputs are architectural rather than per-peripheral: MSOFT (ID 3) and MTIMER (ID 7) from the CLINT, MEXT (ID 11) unconnected, and **NMI (ID 31) from the watchdog**.

2.5.2 The three-stage watchdog

The watchdog is not a timer with a reset on the end. It carries the weight the SecureIbex decision put on it — it is the only backstop a silently corrupted processor has — and its specification is numbered properties in its own file header, against which the formal properties and the cocotb suite are written (`docs/40` section 5, `docs/60` section 5.6). There are nine of them: `docs/60` section 5.6 tabulates eight, and `soc_wdog.v` carries a ninth, W9, added when `docs/75` found the reset request reaching an asynchronous reset through a combinational decode of the voted word.

W1: armed at reset, not disableable by software. The enable reads 1 out of power-on reset and a write of 0 is accepted by the bus and *ignored*, so a driver that tried can discover that it failed. Only a bootstrap pin sampled once at power-on reset release can stop it, and the status register reports what was sampled. **W2: the time base is not software-programmable**, because the attack on W1 is not to disable the watchdog but to slow it: on a stock GPTIMER it shares a writable prescaler. Here it has its own fixed divider that no register reaches, and the longest timeout it can be talked into is a constant of the netlist, $2^{16} \times 16 = 1,048,576$ clocks. There is deliberately no clamp register: **a bound enforced by the width of a counter cannot be misconfigured and one enforced by a comparator can**, and the first version’s comparator turned out to be dead logic pretending to be a defence.

W3: three stages. Stage 1 raises a non-maskable interrupt and reloads — non-maskable because the core clears `mstatus.MIE` on entry to every trap handler, so a core stuck inside a handler would never take a maskable warning in exactly the case it exists for. Stage 2, on a second expiry with stage 1 unacknowledged, asserts the system reset, which the core cannot prevent. Stage 3, after two watchdog resets, latches an external pin, because resetting has demonstrably not worked and the decision belongs outside the chip. **W4: the state survives the reset it causes**, so the whole block is in the power-on domain and the SoC has two reset domains with a synchroniser. **W5: keyed writes**, upper half-word 0xA51F, on every register and not just the kick, because an unkeyed acknowledge would let a core spraying stores hold the machine in stage 1 for ever. **W6:** the persistent, silent state is bundled into one word, tripled and voted every cycle [12], while the flip-flops the block rewrites every tick or kick are left alone. **W7 and W8**, a windowed kick mode and a kick budget, are both measured; W7 is **recommended for removal**. The word was

22 bits when docs/41 measured it, against 36 unprotected; W7's and W8's four fields joined it by the same criterion, so the shipped block protects **29 bits** against 44 (hw/soc/rtl/soc_wdog.v, docs/75 section 3.5).

“Armed at reset” was right and not sufficient, and running it is what showed that. The reload register is in the power-on domain by W4, so a short timeout software installed before it went wrong *survived the reset that going wrong caused*, and the SoC looped with the console never reaching its first character; a stage-2 reset now restores the reload to the maximum. **Putting state outside the reset domain is what makes the record survive and also what makes a stale configuration survive** (docs/40 section 7.2). The redundancy itself was counted rather than assumed, after stripping every protective attribute [5] from the source text: **102 mapped flip-flops** on the build docs/41 measured, 36 unprotected plus three replicas of 22, at +5,091.06 μm^2 and **1.6 % of what lockstep would have cost**; **132** on the block as it stands, $44 + 1 + 3 \times 29$, the one being W9's registered request (docs/75 section 3.5).

2.6 The peripheral bus and what is on it

Sixteen 4 KiB slots sit inside the 1 MiB APB window. The bridge **does not decode which peripheral is selected**: it drives one “some peripheral” select and soc_top.v turns that into per-slot selects from the generated constants, because a bridge that decoded slots would be a second copy of the memory map, and the second copy is where drift lives (docs/39 section 4).

Table 2.2 is the current map: **nine slots implemented and seven reserved**. docs/60 section 4.1 counts nine reserved, which was true on its own date of 2026-09-05; the SCRUB and BOOTREG slots were implemented afterwards by docs/67 and docs/68, and revision 0.2 did not recount them. The map is the normative source (docs/60 section 0.2).

The **console UART** mirrors the reference family's register map at four offsets and is a real serialiser, not a simulation print hook: the testbench decodes its line rather than snooping the register write, so the baud divider and the shift register are inside what the run proves. It is **transmit only**: a driver that waits for the data-ready bit waits for ever, which is correct for a part with no receiver and better than a receiver that appears to exist.

The **GPIO** port was the first peripheral promoted from reserved to implemented, and it was chosen first for a process reason rather than a mission one: no block had ever walked the sequence from map promotion through top-level decode and source lists to a bring-up check and a proof, and walking it on this project's own RTL means that when the second block fails a step the failure is in the block and not in the path (docs/65 section 2). It is 16 pins on one shared interrupt line, and **its data register reads the pad and never the output register**: a block that folded one into the other would pass a write-then-read-back test with no pin present at all.

The **QSPI flash controller** is register-mode with two chip selects. The alternative, wrapping OpenTitan's spi_host, was measured rather than dismissed: it synthesises to **65.051 kGE**, **171 % of the Ibex core**, with about four fifths of its 5,404 flip-flops FIFO storage at depths fixed in a generated package — the area a fact, the share an estimate (docs/65 section 8.4). What was written instead is 19,690.6248 μm^2 , **4.2 % of the size** of the block it declined. Its data phase **pauses at every word**, so a transfer of any length needs one word of storage and no FIFO — “the storage is where software is, not where the block is” (docs/66 section 3.1).

Execute-in-place is **not** built and the two windows stay reserved with the reason in the map beside them: it would make the controller a fabric slave as well as an APB one — a seventh port and a re-proof of soc_bus.v — and a quad read of one word is about 56 system clocks (docs/66 section 2.1) on a core where one extra cycle of memory latency was measured at 25.23 % of run time (docs/50 section 5.2). The controller is checked against a behavioural W25Q128JV

Table 2.2: The peripheral slots, from `regmap/memmap.yaml`. Areas are the same recipe as every other block here, rounded here to two decimals, and they come from different sessions: UART0, TIMER0, BUSSTAT and APBPNP from `docs/45` section 6.2, GPIO from `docs/65` section 7, QSPI from `docs/66` section 9, SCRUB from `docs/67` section 5.1 and BOOTREG — the protected build, which is what the design instantiates — from `docs/69` section 7. A block re-measured in a different source list moves by of order one per cent (`docs/45` section 4.3), and the BUSSTAT row is the file `docs/44` measured: `docs/55` later added the NPU fault lines at a further $+4,796.1396 \mu\text{m}^2$, which `docs/60` section 5.7 declines to fold in because no run has reported the sum. The SCRUB row counts more registers than `docs/60` section 5.7 does, and both are right on their own dates: that document’s seven-register table is revision 0.1’s, and `docs/55` and `docs/58` later added CNT_NPUCOR, CNT_NPUDET, CNT_NPUTMR and CNT_MTECC above CLR, which keeps its offset of 0x018.

Slot	Address	Block	Registers	IRQ	Line	Area, μm^2
0x000	0xFF900000	UART0, console	4: data, status, ctrl, scaler	2	0	4,359.78
0x001	0xFF901000	UART1	<i>reserved</i>	3	1	—
0x002	0xFF902000	GPIO, 16 pins	10, plus 9 logical aliases	4	2	13,162.87
0x008	0xFF908000	TIMER0 and watchdog	scaler, reload, config, latch, 3 timers, WDOGSTAT	8	3	33,528.07
0x009	0xFF909000	TIMER1	<i>reserved</i>	12	4	—
0x00D	0xFF90D000	SpaceWire	<i>reserved</i>	16	5	—
0x011	0xFF911000	CAN	<i>reserved</i>	18	6	—
0x012	0xFF912000	SPI	<i>reserved</i>	19	7	—
0x013	0xFF913000	I2C	<i>reserved</i>	20	8	—
0x014	0xFF914000	QSPI flash	7, at 0x00–0x18	21	9	19,690.62
0x015	0xFF915000	BUSSTAT	11, at 0x00–0x28	22	10	7,060.55
0x016	0xFF916000	SCRUB	6 counters, 6 stickies, 2 addresses	23	11	16,710.96
0x017	0xFF917000	BOOTREG	4: straps, status, report, epoch	—	—	12,495.89
0x018	0xFF918000	CLKGATE	<i>reserved</i>	—	—	—
0x019	0xFF919000	NPUCFG	12, at 0x000–0x02C	24	12	in soc_npu
0x0FF	0xFF9FF000	APBPNP	two words per slot	—	—	905.27

written from the part’s datasheet, with every departure from its AC table counted and asserted zero — after which docs/66 section 12 says the sentence it most needs to: **a behavioural model is not a flash**.

2.7 The boot flow

Until docs/68 the boot ROM held the bring-up program, and that had stopped working: the 28-check program was 7,794 of the 8,064 bytes the map leaves above the reset vector when docs/66 measured it, and 7,826 after the memory-protection work. Four questions were open, in four documents — how the ROM gets contents, who writes the loader, what the bootstrap pin and the boot report are, and what happens when boot fails — and they are one design, because each answer is the next one’s mechanism.

The ROM holds a loader. Four candidates were costed against one criterion: do the contents exist before the core leaves reset without needing something else with the same problem? A mask-programmed array is the only terminal answer — its contents exist because the mask exists, and a bit that is a via has no stored-upset mechanism, so it needs no check field. An SRAM loaded through the vendor macros’ test port relocates the question: **the loader then has a loader**. This PDK ships no ROM generator [10], so the two 1024x32 macros are a **stand-in**, and in silicon on this PDK the ROM’s contents are unprotected and undefined. The loader is 3,104 of the 8,064 bytes available.

An uninitialised protected word is an uncorrectable, not a zero. The RAM’s check bits do not exist until something writes them, so the loader sweeps every word before it reads any, before the stack pointer is set — the only order that works, because its own stack, data and BSS are inside what it is initialising — and it writes *whole words*, because a byte store makes one of a row’s four codewords valid and leaves three as the SRAM powered up. **The image checksum is then computed from the RAM, not from the flash:** a second pass reads the copied words back out and sums them, so a word that arrived intact and landed on a row whose codeword did not take is caught, and that pass is also where an uncorrectable in the loaded image is detected. The sweep costs 6.003 cycles per word, the copy 44.06 and the verify 17.01.

The escalation is the watchdog’s; there is no second ladder. Within a boot, the primary image’s magic, header checksum, geometry, body checksum or ECC each send the loader to a secondary image elsewhere in the device. Both failing means the loader shortens the watchdog to its minimum and stops kicking — safe only because a stage-2 reset restores the reload to the maximum. The attempt limit is **three**, tied to the watchdog’s escalation count of two by an argument rather than by taste: the external pin is asserted on the second watchdog reset, which is the reset that begins boot number 2, and the loader’s last attempt is boot number 2, so **software gives up after the hardware has told the platform, never before**. There is no “wait for an upload” state, because the console is transmit-only. The watchdog is kicked **on progress, not on a schedule**, from inside the copy loop gated on the word index.

soc_boot.v is the register block and its design rule is one sentence: **the one field with authority is the one software cannot write**. The boot counter is maintained by hardware on each release of the system reset, saturating, and no APB write moves it; the report and epoch words are software’s and are *evidence*. The block therefore needs no write key — a field hardware maintains cannot be forged — and its only outputs are the APB read path, so it can shorten no budget and reach no timer.

Hardening it (docs/69) produced a ranking rather than a count. Of 92 flip-flops **eighteen are protected** — the boot counter’s eight, the four sampled straps, the watchdog-pin readback, two flags, a two-bit sampling delay and the system-reset sample — bundled into one word with a five-bit mismatch report and tripled under the frozen voter, while **74 are deliberately not**, 64

of them the report and epoch words, 69.6 % of the block’s state with zero boot consequence [19]. The campaign moved one bit into the protected set: the system-reset sample is rewritten every clock, which normally means a register sheds an upset on its own, and that is true of the register and false of the consequence — an upset there manufactures a release edge and the saturating counter never takes it back. Protection costs +5,255.9010 μm^2 and +51 flip-flops; placed and routed (docs/70) it costs **no die area, no hold margin and no routing quality** — and the layout it landed in reports **4.30 ns less setup slack** at nom_slow_1p08V_125C than the layout placed beside it, on a design that closes neither way, which that document attributes to the flow’s timing repair rather than to the block: u_boot appears on no violating path in either run.

Boot end to end, through the real fabric from a modelled flash with the RAM powered up undefined, is **195,068 cycles**; the program that follows is **220,256**, for **415,324** in the run. The same program fetched from the boot ROM is 217,630, so the move costs +1.207 %, and all of that is the fabric rather than the memory: with the program in ROM the two masters never contend, and with it in RAM both go to one slave port and one waits (docs/68 sections 8, 8.1).

2.8 Telemetry and discovery

Two blocks exist because a SECDDED register file corrects silently, and in silicon a corrected upset is indistinguishable from no upset at all [23]. **BUSSTAT** carries four saturating counters and four sticky bits for register-file corrections, corrected read cycles, uncorrectable syndromes and watchdog voter mismatches. The counters are different quantities and the header insists they not be folded: the correction counter counts an upset the program does not overwrite exactly once, while the corrected-read counter is an upper bound, because one corrupt word read in a loop raises it every iteration. The record lives in the power-on domain and survives the reset the watchdog causes, while the interrupt enable is deliberately in the system domain, so an armed interrupt cannot fire on a fresh boot before software has configured it (docs/60 section 5.7). **SCRUB** is the memories’ equivalent and is a separate block for a blunt reason — **BUSSTAT** is full — carrying the same three quantities per memory, both scrubbers’ enables, the interval, and the byte offset of the last uncorrectable row. Both scrubbers ship *walking*, by the rule that a mechanism which ships disabled and has never caught anything is a liability.

Two **plug-and-play device tables** are read-only in the reference family’s record layout: one for the system bus at 0xFFFFF000, eight words per slave record, one for the peripheral bus at the bridge base + 0xFF000, two words per slot, both generated from the same source as the map. The layout is mirrored because a well-known layout is what makes a table readable by someone other than its author, and two claims are explicitly not made: it is not an AMBA AHB plug-and-play table, because there is no AHB, and **the device IDs are assigned by this project and are not registered** with the vendor whose “various contributions” code they sit under, so a family-aware tool shows a known vendor and an unrecognised device, which is the truthful outcome. One field cannot express this map exactly — the bank address register compares addr[31:20] and four regions are smaller than 1 MiB — so the exact byte size of every region is carried in user-defined word 1.

2.9 What the top level is, and what it is not

There is no pad ring and no package, so soc_top’s port list is what a floorplan or a future pad ring would have to serve. Its functional pins are the clock, the power-on reset, the watchdog bootstrap pin, four bootstrap straps, the UART transmit line, the watchdog’s stage-3 output, sixteen GPIO pins as three wires each because there is no pad, and the flash controller’s clock, two chip selects and four lanes as three wires each. Everything else is an observation output for a

testbench or a scope. docs/60 section 6.2 says the design has three functional pins; that was true of revision 0.1, before the GPIO, flash and strap pins.

The design has **one clock and one mode**: no clock-domain crossing, no retention, no power domains, and a reserved address where the clock-gate block would be. docs/60 section 4.3 calls the consequence its one measured requirement gap, four orders of magnitude wide: on the telemetry mission profile's duty cycle the part is 0.0044 % busy — an estimate, on docs/53 sections 6.2, 7.3 — and 100 % powered. That second half is revision 0.1's. Since docs/76 the netlist carries **three integrated clock gates** where it carried one, on the core's, the fabric's and the accelerator's clocks, the two new ones driven by their own block's enable and the CLK_GATE slot still reserved: the share of flip-flops behind a gate goes from 39.5 % to **76.0 %** and idle power at the typical corner from 18.745 to **8.287 mW**, a factor of 2.26, carried through docs/60 section 11.3's correction of 2026-09-12 as an orbit average of 8.289 mW. The gap is narrower and still orders of magnitude wide.

Three absences belong beside table 2.2, which a reader could otherwise take at face value. There is **no DMA, no descriptor ring and no third bus master**: a ring is a bus master, the fabric has two hardcoded master ports and a fairness property written for two, so a third is a fabric change plus a re-proof — and at this scale it would save nothing, because the transport and not the CPU is the bottleneck (docs/51 section 12). There is **no debug module and no JTAG**. And there are **no spacecraft link interfaces** — SpaceWire, CAN, SPI and I2C are frozen addresses with nothing behind them, though all four candidate cores have been fetched, pinned and measured and an APB-to-Wishbone bridge for two of them is proved but instantiated nowhere (docs/65 sections 9.4, 9.5).

The whole design at the SRAM boundary, synthesised as one netlist at the 20 ns target with both memories protected, is **45,952 cells, 5,833 flip-flops, 687,776.3676 μm^2** of standard cells plus the eight vendor macros (docs/67 section 5.1). That row is from before the boot block existed — soc_boot.v does not appear anywhere in docs/67 — and the netlist the placed design of record was built from is a different configuration, hardened boot block and ROM_HARDEN = 0: **45,546 cells, 5,873 flip-flops, 685,153.2744 μm^2** (docs/69 section 7.4, docs/60 section 9.10.1). **No document measures both together**, so no combined figure is given here. Neither number is a frequency, and none is claimed.

Chapter 3

The neuromorphic accelerator

This block exists in two sizes at once. There is a full-scale specification — 512 neurons, 512 axons, a quarter of a million synapses in an error-corrected SRAM — written as `docs/10`, dated 24 August 2026, with a bit-exact Python golden model as its normative executable form. And there is a slice of it, elaborated at eight neurons and eight axons with its synapses in flip-flops, carried through synthesis, placement, routing and geometric sign-off and submitted to a Tiny Tapeout shuttle, where `docs/34` pins every byte of it by hash. Everything measured here was measured on that slice or on the system-on-chip that instantiates it.

3.1 What it computes: leaky integrate-and-fire, in integers

The neuron model is classical digital leaky integrate-and-fire [7]. Each neuron carries a 24-bit state word: a signed 16-bit membrane potential V , an unsigned 4-bit refractory countdown R , and four reserved bits `docs/10` section 3 holds open as a per-neuron parity hook. Threshold, reset potential, leak shift, synaptic shift and refractory period are core-global configuration loaded once per pass, not per-neuron state. `docs/10` section 4 states the update as ten numbered equations (table 3.1), and the discipline that matters is that they are exact integer arithmetic with explicit saturation — no floating point anywhere in the evaluation path, by construction and by test.

Three of the ten carry a decision rather than a definition.

E3 saturates and must never wrap, because a two’s-complement wraparound of a positive overflow would silently *lose* a spike rather than produce a wrong one, and a lost spike is invisible.

E4 is checked after the update, on every non-gated event, including events whose contribution is zero. `docs/10` section 4.1 calls checking before the update, or one event late, a specification violation, for a stated reason: this exact off-by-one was a verified bug class in the developer’s prior integrate-and-fire silicon work.

E6’s leak has a minimum step of one, so every non-zero potential reaches zero within $|V|$ ticks rather than being stranded once the shift rounds down. That bound licenses the project’s position on unprotected neuron state, which `docs/10` section 11.3 states as a bet rather than a result: no error-correcting code on the membrane potential in v0.1, because an upset in V costs at worst one spurious or missed spike and then decays back toward rest — on the order of 100 ticks at the reset leak shift of 3 from full scale — and an upset in R costs at most $T_{\text{REFR}} \leq 15$ ticks of wrong gating. The four reserved bits are the hook if a campaign shows the bet is wrong. The pilot did not wait for that: `lif_core.v` holds the 20-bit $\{R, V\}$ word under one SECDED (26, 20) codeword, six check bits per neuron, so the elaborated slice corrects what the specification leaves bare (`docs/16` section 2.1).

E8 is what makes the rest possible: no arbitration, no clock-dependent reordering, no dropped

Table 3.1: The normative update equations of docs/10section 4, as implemented in `hw/rtl/lif_core.v` and in `sw/golden/lif_core.py`. The test `sw/tests/test_traceability.py` parses the specification for tags and fails if one is uncovered, and, per a corrective note dated 2026-09-11, if the covering test asserts nothing.

Tag	Contract
E1	$w = \text{sext4}(W[a][j])$, a 4-bit signed weight, $w \in [-8, +7]$
E2	$c = w \cdot 2^{S_{\text{SYN}}}$, exact, $S_{\text{SYN}} \in [0, 7]$, so $c \in [-1024, +896]$
E3	$V' = \text{sat16}(V + c)$, summed at full 18-bit signed width, then clamped
E4	$\text{spike} \iff V' \geq \Theta$, signed, checked <i>after</i> the update
E5	on spike: $V \leftarrow V_{\text{RESET}}$, $R \leftarrow T_{\text{REFR}}$, emit id <code>TILE_OFF + j</code>
E6	leak toward zero by $\max(V \gg S_{\text{LEAK}}, 1)$; the sign never flips
E7	$R > 0$ gates E1–E5 for that (event, neuron); a tick decrements R
E8	determinism: arrival order, ascending scan, spikes emitted in scan order
E9	multi-pass neuron-tiling equivalence
E10	an uncorrectable weight word substitutes $c = 0$; processing continues

event on the processing path, so the core is a deterministic function of state, configuration, weights and event order. That is what lets a golden model be bit-exact against hardware and a fault-injection verdict be attributable to the bit that was flipped.

3.2 Spikes on a wire: the address-event representation

Events are 16-bit words and docs/10section 7.1 freezes the layout: two bits of `TYPE`, four reserved zero bits, and a 10-bit `ID` that is an axon identifier inbound and a neuron identifier outbound. `TYPE 00` is a `SPIKE`, `01` a `TICK` — a timestep boundary, which runs the leak and the refractory countdown and emits nothing — `10` a `SYNC`, and `11` is reserved, dropped and counted. The core consumes time and does not create it: tick generation is external in v0.1.

`SYNC` is the primitive the sequencing rests on: when one is consumed, every prior event has been fully processed and the barrier is echoed downstream, so a host injects one after a frame and reads until it returns.

The queues are `hw/rtl/aer_fifo.v`, 64 entries each in the specification, and their contract is more interesting than their depth. A write while full is dropped rather than allowed to corrupt stored events or pointers, and counted in a sticky saturating counter (`CNT_EVQ_OVF`); a link-side producer instead gets lossless backpressure by gating its write with `!full`. Both pointers are triple-redundant and voted. Every stored entry carries an even-parity bit, *detected and not corrected* — a read whose word fails the check discards the entry and holds `rd_valid` low — and that signal is itself dual-rail and likewise detected, so a rail disagreement reads low and the answer is missed rather than fabricated. A full output queue stalls the update pipeline rather than dropping spikes, and deadlock freedom across a mesh is assigned to a future router.

3.3 The crossbar and the weight memory

Logically the synapses are a crossbar: $W[a][j]$ for axon a and neuron j , 4-bit signed, at bit address $(a \cdot \text{N_NEURONS} + j) \cdot 4$ — axon-major, so one event’s weight column is a contiguous burst. Physically, at full scale, they are an SRAM with a single update pipeline time-multiplexed across all neurons [18]: docs/02section 3’s Candidate B, chosen because flip-flop weights do not scale.

The physical word is 64 data bits, exactly 16 weights, plus 8 check bits of a single-error-correcting double-error-detecting code [9]: a 72-bit macro word at 12.5 % overhead. At the 512-by-512 default that is 16,384 words — 128 KB of data and 16 KB of check bits, the dominant area of the block (docs/10section 5) — and one synaptic event reads 32 of them, the pipeline applying E1–E5 to 16 neurons per word.

That codeword is not the one the architecture study assumed, and the correction is on the record rather than edited into it. docs/02section 2 carries a note dated 2026-09-09 saying its own sizing used 32 data bits plus 7 check bits, and measuring the cost: every figure derived from that document’s 156 KB is high by 8.33 % of the 144 KB basis. Nothing was renumbered, by the rule that a superseded figure stays visible with a dated note saying what replaced it (docs/64section 2). What the note could not fix is that docs/10section 5 calls its 144 KB “in line with the docs/02 estimate” — a claim of agreement across exactly that 8.33 %, in a document the note does not own. Both stand, and a reader should treat them as disagreeing by that amount.

Single-bit errors are corrected inline and counted in CNT_SEC. A double-bit error takes E10: zero is substituted for every weight in the affected word, CNT_DED increments, FAULT_ADDR latches the word index, and processing continues. The core is deliberately fail-operational — a dead weight word degrades the network rather than stopping the node — and a background scrubber walks the array while the input queue is empty, so single-bit errors do not accumulate into double-bit ones [22].

What the pilot stores is flip-flops, and that is a fallback that was taken rather than a simplification. hw/rtl/lif_core.v’s header states it: the synapse array is a flip-flop file with a one-weight-per-cycle load port and no SRAM macro — docs/02section 3’s Candidate A storage class, reached by that document’s fallback trigger F1. The trigger fired for a measured reason: docs/12sections 7.5 and 8 hardened one RM_IHPSG13 macro [10] and returned a NO-GO for this shuttle, because the macro places, routes and closes timing and then fails Magic design-rule checking, KLayout design-rule checking and Netgen layout-versus-schematic on causes entirely inside the vendor views — 1,106,478, 2,316 and 359 errors, as docs/61section 14 quotes them. Dropping to flip-flops did not drop the code with it, so E10 is implemented in the pilot at exactly the 16-weight granularity the golden model’s word index defines. The cost of time-multiplexing is in the same header: one event or tick takes $N_NEURONS + 1$ cycles at full output bandwidth, independent of sparsity — precisely the trade that buys the memory scaling.

3.4 Networks larger than the fabric

The physical core holds one weight slice. Larger models live in flash as a sequence of pass images: load the slice, load the configuration and tile offset, replay the input event stream, collect output events until the barrier returns. Sequencing is owned by the management core through the register map in v0.1 (docs/10section 9).

Output-neuron tiling is E9: partition a layer’s output neurons into ordered contiguous tiles, run one pass per tile with the matching weight columns and the tile base in PASS_TILE_OFF, replaying the identical input stream each time; concatenating the tiles’ emissions in tile order, per input event, gives a stream bit-identical to a core wide enough for the whole layer. It holds because neuron trajectories are mutually independent in v0.1 — no lateral coupling. The axon dimension is *not* splittable, and the specification says why rather than merely forbidding it: axon-split passes would interleave saturation and threshold crossings in a different order and break bit-exactness. Tiling does not lift the identifier bound either, since the ID field and PASS_TILE_OFF are both ten bits, so every tile must satisfy $TILE_OFF + \text{tile width} \leq 1024$ — a bound the golden model enforces at construction, refusing wider layers rather than approximating them.

3.5 Attaching it to the system-on-chip

Until `docs/51` the repository held a RISC-V system-on-chip and a separate accelerator that had never been in the same simulation. The connection is built the hard way: `hw/rtl/pilot_top.v`, the frozen die’s own top level, pinned by blob hash and unmodified, is *instantiated* inside `hw/soc/rtl/soc_npu.v` and reached over the same four serial pins the die will have. No copy exists under `hw/soc/`, and a guard suite fails if any of the ten files `docs/34` section 2.1 pins is edited against the index.

3.5.1 Two faces, because there are two problems

The node register window sits on the system bus: one 4 KiB window per mesh node, at the node index times `0x1000` — the per-node base address `docs/10` section 8 freezes — reached with ordinary loads and stores. The event port sits on the peripheral bus in a 4 KiB slot called `NPUCFG`: two 16-bit event queues with a hardware engine between them and the die, plus the block’s interrupt.

Nothing in the memory map moved to accommodate this, and everything outside a live node’s register block faults rather than aliasing: an address above the sixteen windows, an uninstantiated node index, an offset above `0x1FC` where the die’s seven-bit register address runs out, and a sub-word write — because the serial frame carries no byte enable, so a slave that accepted a byte store would write three bytes of a register the master never named. Check 25 of the bring-up program requires a fault for every one of them — a load-access fault at each of its five bad addresses, a store-access fault for the byte store.

3.5.2 Why the transport is serial

The die’s host port is serial because it was designed against a Tiny Tapeout pin budget; inside a system-on-chip there is none, so a parallel register port on a copy of the pilot was a real option. `docs/51` section 3 argues the choice and rejects the obvious reason — “the die will only have a serial port” — because a bench harness for the die needs a serial host either way. What decides it is that the serial transport leaves exactly *one* implementation of the die’s host protocol, the one in the system-on-chip’s own regression, so the three host obligations the die numbers H1, H2 and H3 are implemented once and proved once. What was built is a level above a memory-mapped serial master — the address is the interface, the transport is underneath it — so a future node with the native 32-bit configuration bus `docs/10` section 1 specifies is a change of transport under an unchanged driver.

The cost is stated rather than waved at. A node-window register access costs **176 clock cycles** from request to response, of which 171 is the serial frame; the protocol’s arithmetic floor — 40 serial-clock cycles plus one period of select setup and one of inter-frame gap, at two clock cycles per half period — is 168, and the `cocotb` test asserts the measurement lies between the floor and the floor plus 24, so the number is an observation and not a design intent (`docs/51` section 8.1). At full scale the same transport would need 32,768 register writes for one weight image, which is why `docs/10` section 9 puts bulk loading on a flash path.

3.5.3 The parallel event pins are not symmetric

The die’s event pins are used inbound and cannot be used outbound, and both facts come from the pin contract. Inbound, the pin word’s type field comes from a single tick pin, so it is 00 or 01 and never 10: **the barrier cannot be injected over the pins at all**, and goes over the serial input-queue register instead, once per frame rather than once per event. Outbound, the

four-bit identifier pins carry no type, so a barrier echo and a spike from neuron 0 are the same four bits — conflating exactly the event the sequencer depends on. So the output-valid pin is a *notification*, the word is read from the die’s output-queue register, and the acknowledge pin is tied low because the serial read is itself the pop. The result is an asymmetry of about 44 times — four clock cycles inbound, a frame outbound — which docs/51 section 4 attributes to the tile’s pin budget rather than to a design choice.

3.5.4 The event engine

Between the two software-facing queues and the die sits one finite-state machine with ten states, declared in `hw/soc/rtl/soc_npu.v`: `E_IDLE`, `E_FETCH`, `E_DECIDE`, three for the pin path (`E_PIN_A`, `E_PIN_S`, `E_PIN_G`), two for a serial injection (`E_SER_RQ`, `E_SER_W`) and two for a drain (`E_DRN_RQ`, `E_DRN_W`). Its datapath is a 16-bit event word in flight, a wait counter, a guard counter, and the pin registers.

Three of its rules carry reasons. **Drain outranks inject**, because a full output queue on the die stalls its update pipeline, so injecting into a stalled core achieves nothing while draining unblocks it. **The type field chooses the transport**: 00 and 01 take the pins, 10 and 11 the serial register, and the reserved code 11 is deliberately not filtered, because filtering would hide the die’s documented behaviour for it. And **issue and wait are separate states**: the transport drops its busy flag and raises its done flag on the same edge, so a single state that both started a frame and watched for its completion starts a second on the way out — a read of the output queue, which pops, reaching an engine that has moved on, and it was measured duplicating one barrier echo in a twelve-event stream. `E_DECIDE` would otherwise hold for as long as the die’s input queue stays full, so it carries both an escape and a bound: the escape detours to a drain and comes back to the same event word, losing nothing, and the bound gives up after `DECIDE_MAX` cycles, discards the event and raises a sticky cause bit — bounded, reported loss, which the file argues is better than an engine that stops.

3.5.5 One interrupt, and a cause register that scales

The block spends nothing that was spare: the peripheral slot had already been assigned source 24 on fast local line 12 before the block existed, and two fast lines remain unassigned, derived from the generated map by a test rather than asserted (docs/51 section 9). One line serves any number of nodes, because the cause register says what happened.

docs/51 shipped seven cause bits: five levels — the capture queue is not empty, and the die’s four fault pins — and two stickies, a refused injection and an expired fetch. The distinction is a decision. A level has state behind it, so the way to clear it is to fix the state; a refused write has none and must be latched or nobody can observe it. Writing one to a level bit is accepted and does nothing, and a test asserts exactly that, because the opposite convention produces a driver that loses events. `hw/soc/rtl/soc_npu.v` today declares fifteen cause bits and a 27-bit protected word; docs/56 section 6.3 took the count from 12 to 14 at a cost of 12 flip-flops, 80 % of that wave’s flip-flop cost, paid because a bound that fires and tells nobody is a defect already recorded once.

3.5.6 The clock gate and the wake

The accelerator has no clock gate of its own and could not be given one, because the die inside it is frozen; the only thing a frozen block’s clock allows is to stop it from outside, which is what docs/76 does. Three integrated clock gates replace one, the accelerator and the fabric joining the core: flip-flops on the ungated clock fall from 3,551 to 1,404, 2,089 of them moving to the

accelerator’s gated net and 55 to the fabric’s, and the share of the design behind a gate rises from 39.5 % to 76.0 %. Idle power of the whole chip at the typical corner falls from 18.745 mW to 8.287 mW, a factor of 2.26, on two runs one parameter apart whose resolved configurations differ in no key — and docs/76 section 8.7 lets that figure travel only with the rest of its sentence, which is that both layouts fail setup at the slow corner, fail maximum slew and maximum capacitance, and have never been through design-rule or layout-versus-schematic checking. The accelerator’s share had been measured before the gate existed: it multiplied the chip’s idle power by 5.588 over 2,152 ungated flip-flops (docs/57 section 9), a population docs/61 section 7.3 counts again in the clock tree as 2,178 with docs/58’s 26 added, every one of them on the ungated net. It is one domain because the serial master divides the block’s clock to make the die’s serial clock, so two would put a clock-domain crossing on the serial port of a frozen block.

Nothing wakes it, because it is never asleep when it is needed. The enable is a combinational function of the cycle’s own inputs, and an integrated clock gate latches its enable on the falling edge, so an enable that rises in cycle N produces the edge that ends cycle N : wake latency is zero by construction. A four-cycle hold counter keeps the clock running after the last activity, as margin for a settling chain inside the frozen die rather than as a correctness term. The gate closes on 96.61 % of the whole-chip self-test, a program that performs an inference, and 99.96 % of the supervisor workload.

docs/77 then split the enable by *arrival*: the two bus-side inputs stay combinational, being the only terms that can rise and fall while the clock is stopped, and the nine fault terms that are functions of the block’s own registers move behind one flip-flop on the ungated clock. Nine and not all ten, because C_INJ_OVF carries the peripheral select and is therefore already covered by the combinational half — which is what STICKY_FROZEN masks out. docs/77 section 5.2 counted eight of nine; the tenth line, C_EVT_T0, was added on 2026-09-11. Such a term cannot pulse and vanish — it rises when the upset lands and stays up until the block is clocked — so a wake one cycle later loses nothing. Both halves are proved by k -induction in hw/soc/formal/clkgate_wake.sby, and the side condition is discharged by a fan-in cone census in Yosys cut at every sequential cell, which found a fault term nobody’s reading of the file had noticed.

The accelerator’s clock-gating check does not close at the slow corner — 5.0198 ns short at docs/76’s sign-off, 4.2111 ns after docs/77’s change — and the two documents disagree about the cause. docs/76 blamed the block’s fault lines; docs/77 cut launch domains one at a time on that same netlist and measured them worth at most 1.3129 ns, the 5.0198 ns path launching instead from a bit of the CPU’s register-file read address.

3.6 The golden-model comparison

docs/51 section 7 is the demonstration, and its structure is what makes it evidence. The generator `gen_npu_vectors.py` runs on every build of the boot image, *before* the compiler: it runs the golden model `sw/golden/lif_core.py` over the stimulus and emits the node register map in C plus the stimulus and the expected output stream. The program compares against those constants and never asks the hardware what the answer is — the difference between a check and a tautology. The stimulus is the pilot’s own testbench’s, unchanged, so a disagreement between the two runs is attributable.

In the loop are Ibex, the boot ROM, the fabric, the peripheral bridge, the serial transport, the frozen die’s register bank, weight loader, datapath and event queues, the parallel pins inbound, the serial output register outbound, and the system-on-chip’s own two queues. The sequence clears neuron state, writes eight configuration registers and *reads one back* — because a refused configuration write is otherwise invisible until the arithmetic comes out wrong and then looks like an arithmetic defect — loads four error-checked weight words, checks that the correction and

Table 3.2: The pilot die’s content and configuration, from docs/15 section 1.

Block	Source, under hw/rtl/	Configuration
Integrate-and-fire datapath	<code>lif_core.v</code>	8 neurons $\times$ 8 axons, 4-bit signed weights in flip-flops
Input event queue	<code>aer_fifo.v</code>	16 bits $\times$ 4 entries
Output event queue	<code>aer_fifo.v</code>	16 bits $\times$ 4 entries
Configuration voter	<code>tmr_voter.v</code>	55-bit voted word
Weight-word codec	<code>secded_enc.v</code> , <code>secded_dec.v</code>	(72, 64) Hsiao
Registers, serial host port, dispatcher, weight loader, fault counters	<code>pilot_top.v</code>	31 registers of the specification map, plus pilot-only ones
Shuttle wrapper	<code>tt_um_melihakbulut_nssoc.v</code>	pin map, pin directions, reset synchroniser

detection counters are zero, and enables the core. Then six frames, each injected as its spikes, a tick and a barrier, read back until that frame’s barrier returns.

What is checked is stronger than a spike count: twenty-two events in and twelve out — six spikes and six barrier echoes — matched event for event in stream order; then the *whole neuron state file*, potential and refractory counter for all eight neurons, which catches an error that happened to cancel; then that nothing was lost, with the die’s drop counters, the injection-drop counter and the cause register’s fault bits all zero; and finally that the hardware’s own count of events delivered and drained, 22 and 12, agrees with the stream by an independent path.

The whole-chip self-test result at that document was 27 checks, fail mask zero, 213,971 cycles. That invariant has moved twice since, and the earlier figures are superseded rather than overwritten: docs/55 measured 215,428, all 1,457 added cycles being software; docs/76 reports 220,256 in the image and 415,324 for the run, at 28 of 28 checks and fail mask zero.

3.7 The pilot chip

3.7.1 What is on it

docs/15 section 1 says the pilot adds glue, not arithmetic: every block was verified before it was integrated, and table 3.2 is its content. At eight by eight the die stores 256 synapse bits, 160 neuron-state bits, 128 event-queue bits and a 72-bit error-protected weight staging word; the wrapper declares 1,241 flip-flops and 1,235 map through the flow.

Two fault-tolerance demonstrators sit on the real datapath rather than beside it. The 64-bit weight word software writes *is* the data field of a stored (72, 64) codeword: on commit an armed injection pattern is exclusive-ored into it, the decoder runs, and the loader writes sixteen weights into the datapath *from the decoder’s output*. And all 55 configuration bits are held in three replicas and voted before they leave the register block; with one replica held wrong, software still reads the voted value, the inference still produces exactly the same spike stream, and the counter and the fault pin still record it. Four of the die’s eight dedicated outputs are fault pins, so every hardening event is visible on an oscilloscope with no host software. The stated limit: the injector drives the replica’s read path, not its storage flop, so the pilot implements no replica resynchronisation.

The pilot deliberately carries no SRAM macro, no management core — it is driven from the

host serial port as a bring-up board would be — and no runtime active-neuron count, because implementing one would be register-transfer logic with no golden reference.

3.7.2 The tile budget, which is where the shape came from

The shape was measured into place rather than chosen. `docs/15` records that the default four-tile block offers 126,685 square micrometres of placement rows — measured from the template the flow uses — against a placed design of 181,043, which is **142.9%**: not a tight fit but an impossible one. The floor is firmer still, because the pre-placement netlist alone is 116.5% of that block before a single timing-repair buffer exists. No four-tile run was attempted; the number applies a measured placed area to a measured row count, and the error runs the wrong way for the small block, since a denser floorplan makes the resizer work harder. The eight-tile shape was then found 2.13% *over* the project’s 70% planning criterion, and that finding is what forced the move to twelve tiles; the choice of the six-by-two arrangement over the three-by-four was made on routing convergence, timing and clock skew *rather than* on area, both twelve-tile shapes sitting far inside the criterion (`docs/23`).

3.7.3 How the artefacts are sealed

`docs/34` answers one question, written so the answer can be checked by running commands: *is what I am submitting the thing that was verified?* It pins the ten files of the frozen directory by git blob hash, the submission tree by a manifest of 31 files whose own hash is recorded, six flow configurations by hash inside the generator, and the toolchain and process-kit versions [15].

The tightest statement it makes is an equality of hashes. Four hardens — two configurations, two dates — produce a **bit-identical netlist, powered netlist and placement database**, the same three values in all four runs. Only the stream-out file and the resolved configuration differ: the first because it carries a generation timestamp, so two streamouts of identical geometry never hash equal. The freeze therefore says the design the sign-off decks were read against and the design the submission path builds are the same bytes, not two designs that agree numerically.

That set has moved twice — a checker closure that forced a re-harden reproducing 196 of 196 and 194 of 194 metrics with zero differences, and a licensing change that moved all ten blob hashes without moving the netlist — so the rule now states: amend in place while the built artefacts are unchanged and every superseded value is retained; supersede the record the moment a netlist hash moves.

Both runs carry zero setup and zero hold total negative slack at all three corners, worst setup slack +1.226180081066267 ns at `nom_slow_1p08V_125C` against a 20 ns period with a 5% derate written into the flow’s own constraint file, worst hold slack +0.102949152117101 ns, zero Magic and routing design-rule errors, zero layout-versus-schematic errors, zero antenna violations, 33,564 instances and 48.7516% instance utilisation. No frequency is published for this design, here or anywhere: `docs/53` refuses it on three grounds and `docs/05` rule 5 binds, so what is reported is periods, slacks and corner names.

3.8 The mesh link that is specified and not built

`docs/10` section 8 freezes five things at v0.1 so that a mesh is a scale-out rather than a redesign, and they may be extended and not changed: the 16-bit local event word; **the mesh link word, 24 bits**, `{DST__NODE[3:0], SRC__NODE[3:0], EVENT[15:0]}`, for up to sixteen nodes, where a node forwards words whose destination differs from its own node-identifier register and the 8-bit

routing prefix is tied off in single-node builds; the link handshake, valid/ready and backpressure-based, with no drops on the processing path; the barrier’s semantics as the frame and pass handshake; and the register map layout including the node identifier and the per-node fault counter block, one identical register-map instance per node. Adding a router, more nodes, or a convolution-capable datapath must not change any of the five.

No register-transfer logic implements any of it. The frozen interface shaped what was built — the system-on-chip’s event port is that link minus its routing prefix, the node window decodes all sixteen identifiers, and each node’s register block carries its own counters — but the second node does not exist, and fifteen of the sixteen decoded indices are a bus error (*docs/51* section 14). The shape costs four comparator bits that do nothing; it buys a second node that is a parameter and a wiring change rather than a redesign of the software interface. The routing prefix is specification and reservation, and a reader should not count it as hardware; the node identifier and the fault counters are implemented registers in the frozen die — `NODE_ID` at `0x9C`, `CNT_SEC` at `0x70` and `CNT_DED` at `0x74`, three of the 31 the pilot implements (*docs/15* section 3) — and what does not exist is the second node they would address.

3.9 One line, finally checkable over all reachable states

The connection has been fault-injected and hardened twice, and the second wave is where one flip-flop turned out to carry a block’s whole failure rate. *docs/56* split the event engine into five sub-strata and drew 100 times into each [16], and the result did not match the ranking by size: the event word in flight, 64 bits and 45.7% of the engine, produced 0 silent wrong inferences in 100, as did the 32 counter bits, while the rate lived in the sequencer’s 19 bits (13 of 100), the adapter’s 19 (6 of 100) and the six pin bits (18 of 100). Per site, 96% of the engine’s rate sat in nine bits, and one of them — `aer_in_stb`, the strobe that presents an event to the die — produced **18 silent wrong inferences in 18 draws**: an upset in it strobed a phantom spike or tick into the frozen die with whatever the address and type pins held, and the die accepted it.

The answer costs no flip-flop. The flag is set on the edge out of `E_PIN_A`, cleared on the edge out of `E_PIN_S`, and the flush path clears it in the same statement that returns the machine to idle. So

$$\text{aer_in_stb} = (\text{ev_state} = \text{E_PIN_S}) \quad (3.1)$$

is an invariant of the machine, and the flag is a redundant encoding of a state the module already holds — redundancy that was in the netlist and was not being used, because the flag drove the die’s pin alone. The pin is now the conjunction of the two and a disagreement raises a sticky cause bit, so a spurious strobe needs two coincident upsets where it needed one. Re-synthesising with the gate removed gives an identical flip-flop count and 22 fewer cells: the mechanism is a comparator and an AND gate, measured at 159.0246 square micrometres and no state.

It masks and does not correct, and *docs/56* section 6.2 names the price rather than netting it off: holding the pin quiet is right when the flag was corrupted, 18 of 18 of the measured draws, and it *loses a real event* when the state was corrupted instead, which the module cannot distinguish. That direction is a failure mode the work introduced.

The one-line claim was checked by one cocotb test and one textual guard rather than over all reachable states. *docs/55* section 9.3 named the gap underneath it — the module had no property set at all — and declined to close it; *docs/56* section 9.3 declined a second time and said so, on the ground that declining twice is a decision. It is now closed, and that is the newest result in this chapter: `hw/soc/formal/soc_npu.sby` is the first property set the largest register-transfer file in the system-on-chip has ever had, and [eq. \(3.1\)](#) is its content. Bounded

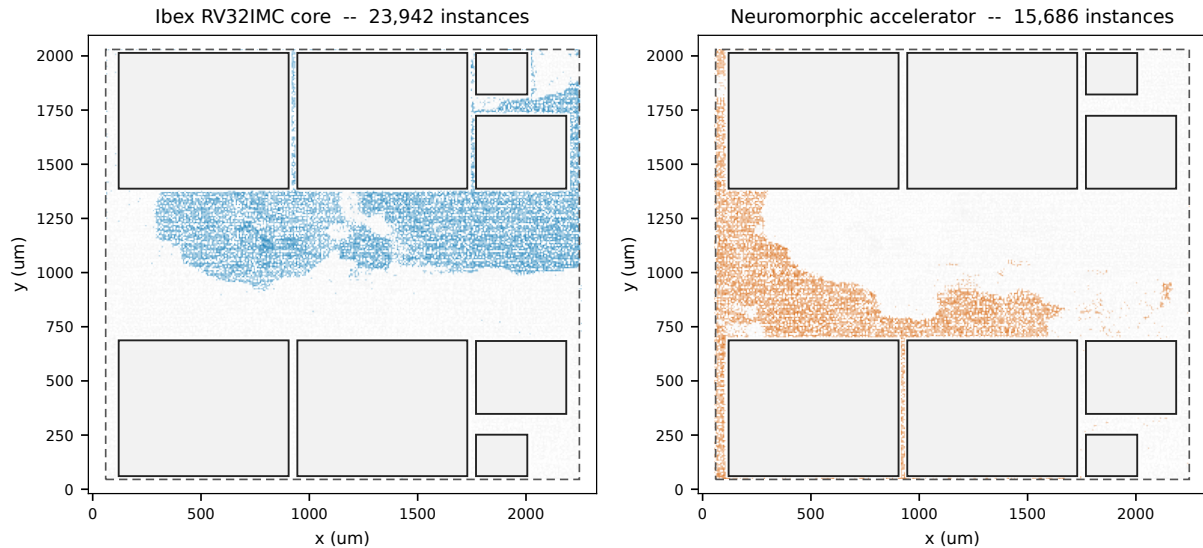


Figure 3.1: The two large blocks of the placed and routed `soc_top`: the Ibex core (left) and the neuromorphic accelerator with the frozen die inside it (right), drawn against the same die outline, core box and eight fixed vendor macros. Generated by `thesis/figures/make_layout_figures.py` from a sign-off run’s placement database and netlist, which is the only provenance that script records.

model checking to depth 30 passes, k -induction at depth 16 passes, and a cover of `E_PIN_S` passes at step 9 — the last because a proof of an invariant over an unreachable state is vacuous. `docs/56` section 10 records the closure as of 2026-09-14.

The property file records what the first runs taught, and it is about the model rather than the design: the properties must be *included* into the module under a preprocessor guard, and the submodules read *without* the formal flag so that their own assumptions stay out of this model. The machinery belongs to the verification chapter; what belongs here is that the claim the hardening rests on is now a theorem about this design rather than an argument about it.

3.10 Where 15,686 cells landed

Figure 3.1 answers “where is the accelerator” on a placed and routed layout of the whole system-on-chip, and it must be read with its method in mind. The die carries 168,592 instances — the attribution file the same script writes totals 168,584, a difference of eight — and after this flow’s synthesis only 1,140 of those instance names carry a dot at all, 632 of them the RAM’s, with exactly one passing the script’s own top-level-block prefix test. A picture that coloured only the named instances would colour one cell and answer the question wrongly. Everything else is attributed by connectivity: nets keep the hierarchical names Yosys did not create, so every net that still carries a top-level prefix votes for that block; each anonymous cell takes the block most of its pins’ nets vote for; and two rounds of neighbour propagation resolve the rest.

The error bar is published with the picture and is 9.75 %. Of the 60,694 instances that are not fill, decap, tie or antenna cells, 5,916 could not be attributed and are drawn in grey rather than hidden; 107,890 fill, decap and diode instances are not drawn at all (`thesis/figures/layout_attribution.json`). The accelerator’s share is **15,686 instances** against the Ibex core’s 23,942 — 25.8 % and 39.4 % of those 60,694.

The picture shows a block the placer spread rather than clustered: the accelerator’s cells occupy

the lower half of the central channel between the two macro rows, with the core taking the upper half, plus a column running the full height of the die down its left edge. Apart from the eight fixed macros, nothing here was told where to go.

It is also the physical statement of what the area measurements said. At docs/51 section 11, on Yosys statistics against the typical-corner library, the connection alone with the die black-boxed was 2,723 cells and 57,805.4988 square micrometres and the whole subsystem including the die was 12,421 cells and 209,739.0834 — **76.08 %** of the Ibex reference of 275,682.6198 by cell area. Two hardening waves superseded those rows: the same recipe in docs/56 section 7 gives **3,416 cells and 67,338.0540** for the connection and **13,148 cells, 2,092 flip-flops and 219,294.8478** for the whole subsystem. More than half of the connection is its two event queues — 53.5 % by area at the earlier measurement — expensive because they carry entry parity, pointer voting and drop counting.

3.11 What is open

There is no second node and no router, so the mesh link is a frozen interface with no implementation, and there are no descriptor rings — a ring would be a third bus master on a fabric whose fairness property is written for two. Multi-pass sequencing is exercised by the pilot's own suite and by nothing on the system-on-chip, where the tile offset is written once, to zero. The weight path still enters the node one register at a time, because that is the die's only weight port; the flash controller changed where the words come from, not how they enter (docs/51 section 14, corrected 2026-09-05).

Most of the connection's flip-flops are unprotected by decision — the transport's shift register, the engine's state, the window's captured request and the frame guards — and the queues' stored words carry aer_fifo's detect-only entry parity and nothing beyond it. In both cases the reason is a measurement rather than a budget. That queue storage is 37.6 % of the connection's flip-flops now that the block has grown, was 41.2 % when the campaign measured it, and is 53.5 % of the area docs/51 section 11 reports; it produced zero silent wrong inferences in 100 draws, because occupancy is low and most of those bits are storage nothing will read: a wave that started with the biggest structure would have bought nothing. The largest site now left is the engine's state register, at 7 of 20, which eq. (3.1) has made load-bearing for a pin it was only implicitly related to before.

One limit applies to everything measured here: every campaign in this chapter is register-transfer, single-bit and flip-flop-only. No back-annotated timing, no multi-bit strike, and no single-event transient in combinational logic [25] — which matters because the strobe gate *is* combinational logic, so a transient on it is a failure mode no instrument here can see. A gate-level netlist carrying the connection has since been injected into — docs/82 draws into the accelerator's gated domain on a placed layout — but at zero cell delay and under a workload that never touches the accelerator (docs/82 section 8).

Chapter 4

Radiation hardening

4.1 The criterion, and why it is not the flip-flop count

Nothing here is protected because it sounds important. The ranking was written once, in docs/41 section 3.1, and every wave since has applied it unchanged. **What rewrites this register?** One the block overwrites every tick sheds an upset on its own; one written once in a mission keeps it for the mission. **Does its corruption announce itself?** A part that resets too early is wrong and loud; a watchdog that has quietly stopped being a watchdog is wrong and silent, and the system believes it still has one. Everything persistent *and* silent is protected; everything the block rewrites is not.

That is close to the inverse of the flip-flop count, and the inversion has been measured three times. In the watchdog, counter, reload and pre are 36 flip-flops against the protected word's 22 and get nothing (docs/41 section 3.3). In the boot block, BRPT and EPOCH are 64 of 92 flip-flops with zero boot consequence (docs/69 section 1). And in the accelerator's connection the two queue strata are 41.2 % and 9.2 % of the flip-flops and contribute *zero* to the measured corruption rate (docs/52 section 6.3).

In build order: triple modular redundancy over bundled decision state; a single-error-correcting, double-error-detecting (SEDED) code over the register file, over the time base, and, shortened, over both memories, each with a scrub; redundancy over the boot decision word; and clock gates designed so that a sleeping block can still report an upset. Each was measured against the design without it [3], and three measurements came back negative (docs/43 section 8.4; docs/46 section 9; docs/58 section 4.3). Figure 4.1 puts that list on the die: the five blocks the protection built or rebuilt — the two memory ECC wrappers, the scrub engine, the fault counters, and the boot controller docs/68 built and docs/69 rebuilt around a tripled word — are 5,776 of the 60,694 non-fill standard cells, about 9.5 %, and an upper bound, since each also carries logic that would exist unprotected [estimate, arithmetic on the figure's legend].

4.2 Triple modular redundancy, and why one bit cannot have it

Three copies of a flag cannot be held apart. hw/rtl/pilot_top.v section 8.2 proves it: over one bit there are exactly two storage functions, x and $\neg x$, so a third replica is bit-for-bit identical to one of the others and Yosys's opt_merge hashes it away, and three bits is the first width at which a third replica has a function left to take (docs/30 section 3.3; docs/41 section 4.1). Five of the watchdog's eight protected fields are one bit wide — hw/soc/rtl/soc_wdog.v's field offsets and docs/41 section 3.2's own table, whose widths sum to the 22 bits quoted below, while that document's section 4.2 says six — and replicating one naively yields a single flip-flop with a voter

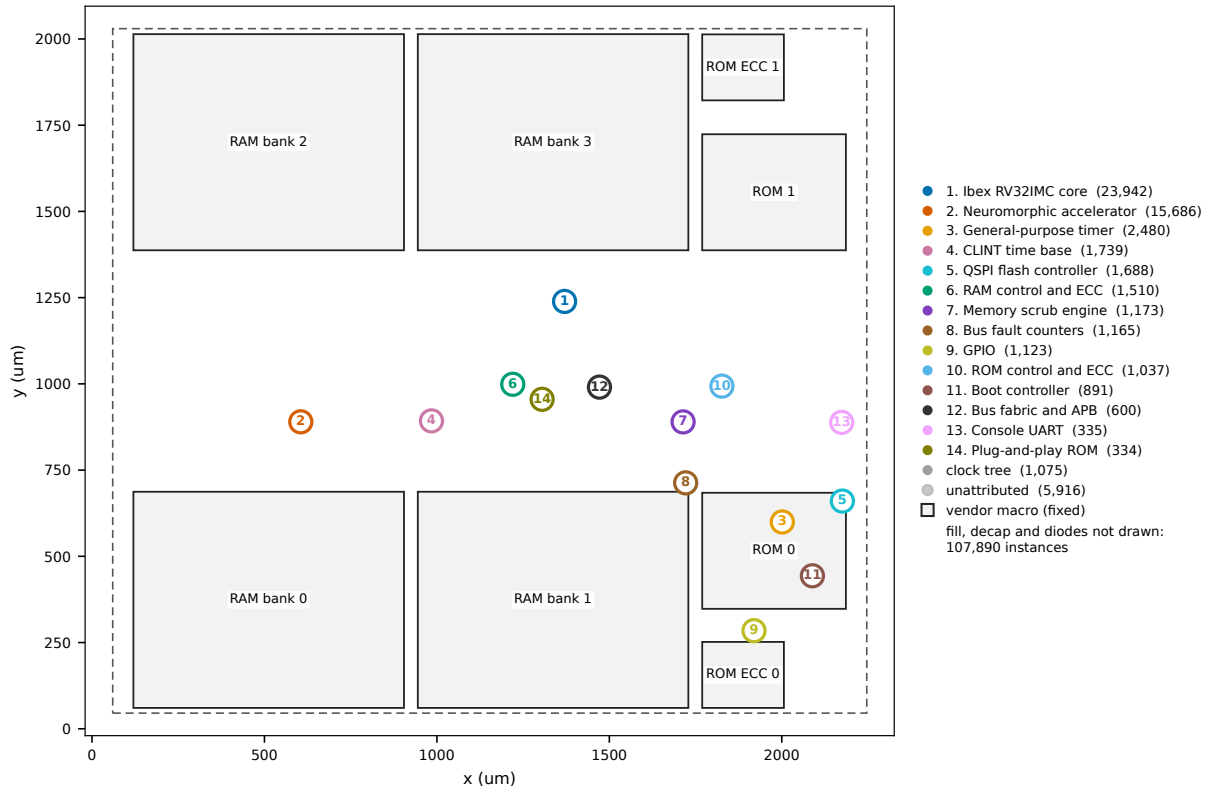


Figure 4.1: Every standard cell of the placed die, coloured by block. Only 1,140 of the die’s instance names carry a dot at all after synthesis, 632 of them the RAM’s, and by the script’s own prefix test exactly one carries a top-level block name, so attribution is by connectivity — nets keep their names, each anonymous cell takes the block most of its pins’ nets vote for, and two rounds of neighbour propagation follow (`thesis/figures/make_layout_figures.py`, whose docstring counts 168,592 instances where its output file counts 168,584; these figures are the output file’s). The remainder, 5,916 cells or 9.75 % of the 60,694 non-fill cells, is drawn grey rather than hidden, and the 107,890 fill, tap, tie, decap and antenna instances are not drawn at all.

voting it against itself — while **every functional test and every proof here would still pass**. Polarity cannot rescue it either (docs/33 section 5).

So the bits are concatenated into one word and the word is replicated, replica A storing it plainly while B and C store mixed images under complementary polarity masks that no polarity of A can equal (docs/41 sections 4.2 and 4.3). The voter is `hw/rtl/tmr_voter.v`, read in place and never copied, so the SoC’s majority gate is the one `formal/tmr_voter.sby` proves exhaustively.

The bank has no write enable, and that is the scrub. Everything in the watchdog is in the power-on reset domain by requirement W4, so “written once” means for the mission, and a bank that held its value would mask the first upset and then wait for a second it could not correct. The whole word is therefore written from the voted value on every clock edge [12]: **the voter is a continuous scrubber, and exposure to a second independent upset is one clock cycle rather than the rest of the mission** (docs/41 section 5.1). That bounds accumulation over *time* and nothing else: the header carries a dated correction saying so, because docs/79 section 1 measured replicas of this shape sharing rows and abutting.

The watchdog protects eight fields, 22 bits, later 29 when docs/43 added a windowed mode and a kick budget. Worst is `dis_q`, the bootstrap latch whose corruption sets `armed` low so the block stops for ever; then `nmi_pend`, whose loss means stage 1 never becomes stage 2, so the watchdog warns for ever without acting (docs/41 section 3.2). The mismatch report is *inside* the word, because docs/16 section 5.8 had measured this repository’s own safety-net report being the single point of failure. The ladder the word governs escalates in three stages (`hw/soc/rtl/soc_wdog.v`, W3): a non-maskable interrupt, with one full timeout for software to notice; then `rst_req_o` for `RST_CYCLES`, which the core cannot prevent; then, after `ESCALATE` resets since power-on, an external pin latched until power-on reset, because resetting has demonstrably not worked and the decision belongs outside the chip.

The identical bank protects three structures — the watchdog’s word at 29 bits, the boot block’s at 23, the accelerator’s cause bank at 25 — and docs/75 section 3.3 re-measured all nine replicas by the voter’s fan-in cone, in the block synthesis and in **every one of the 65 whole-SoC netlists the working tree has kept that contains them** — 64, 10 and 35 of the 65 — and found 29/29/29, 23/23/23 and 25/25/25, every set of three disjoint.

4.3 The register file

The core’s campaign ranked its structures by what their corruption costs [19] and the answer was unambiguous: the register file is 992 of the core’s 2,198 injectable RTL bits — 45 % — and carries 3.2 of the 4.9 design-weighted percentage points of wrong-or-dead outcome, more than everything else in the core combined (`hw/soc/rtl/ibex_regfile_secdded.v` header, citing docs/42). That file carries upstream’s module name and port list and is given to the build instead of upstream’s own, so the Ibex checkout stays a pristine clone at the pinned commit — at the recorded cost that an upstream port whose *meaning* changes silently is caught by nothing.

The codec is `hw/rtl/secdded_enc.v` and `secdded_dec.v`, read in place and fixed at 72 bits over 64 — *one* codec on the die, which is the objection docs/38 raised against lockstep’s [11] second one. One codeword per register with the upper 32 data bits tied off gives the (40, 32) shortening; one check row is then identically zero, so the optimiser deletes one flip-flop per register and what remains is (39, 32), **the minimal SECDED width for 32 data bits** [9]. The netlist census counts 217 check flip-flops and would fail at 248 (docs/43 section 6.3).

Correction on read alone leaves the corruption in the flip-flop, and a register held across a long call keeps it as long as it holds its value — **the second upset in that word is uncorrectable**. So a pointer walks `x1...x31`, decoding and re-encoding one register per idle cycle; when the core

writes, the scrub stalls and the pointer holds rather than skipping. The period is **not proved bounded**: 31 cycles when the core is idle, unbounded in the limit of a program that writes the file every cycle for ever (docs/43 section 6.4).

At first nothing could read the module’s correction counters, because adding an error output is a patch to `ibex_top.sv`. So `opt_clean` deleted them and the report cost zero area, with a test asserting that it did — *a test whose failure would be the good news* — and in silicon a corrected upset and no upset were the same event (docs/43 section 6.5). docs/44 closed that with three anchored hunks applied to a *generated* copy of `ibex_top.v`, never committed, routing the line to `soc_busstat.v` [23] and demonstrating it with a load instruction rather than a hierarchical read. It protects neither the addresses — a corrupted `raddr` reaches the wrong register with a perfectly valid codeword — nor the other 55 % of the core, which carries the highest per-bit rates in the design.

4.4 The time base

`mtime` was ranked and deferred three times — first among what docs/41 section 7.4 left unprotected, second by docs/43 section 12, first by docs/44 section 3 — and protected by none of them, for the same reason each time: an upset in it is persistent and silent — it displaces the architectural clock for ever — but it cannot disarm anything, so the backstop went first (docs/58 section 1).

The answer is not replication, because a counter has structure a general register does not. `mtime` is the 64-bit data field of a (72, 64) codeword whose eight check bits are the only added state. Every tick the stored word is decoded, the corrected value is what increments and what a bus read returns, and **the check bits are re-encoded over the value that goes back in, not over the value that was stored**. That clause is the whole design: encoding over the stored value builds an ECC counter that silently does nothing, the code following the corruption into consistency on the next edge (docs/58 section 3.1). Since the word is rewritten every clock the scrub period is one cycle, so there is no scrubber to build and no accumulation to bound.

Six designs were synthesised against one baseline (docs/58 section 4) and two results are negative. **The residue-mod-3 detector that docs/41 section 7.4 proposed as the cheap answer costs 98.694 % of what the corrector costs and corrects nothing**: the cost of any check over a 64-bit counter is the reduction tree that recomputes it, not the shadow register, so the flip-flop count, 2 against 8, is not where the money is. And docs/41’s unsynthesised estimate for replicating `mtime` and `mtimecmp`, “roughly 27,000 μm^2 ... 256 more flip-flops”, measured at 25,792.0740 μm^2 and *exactly* 256 flip-flops — a good estimate, recorded as one, against a code that corrects the same 64 bits for 50.224 % of the area.

Two decisions follow the same criterion. That 11 of the 64 bits are architecturally dead over a five-year mission is an argument *for* covering them, since deadness bounds the fault-free value and not the faulty one — truncation saves 1,438.1766 μm^2 and is “not a protection; a cheaper failure” (docs/58 section 4.5). And `mtimecmp` gets nothing: software rewrites it every deadline, and its worst direction is a missed deadline, the exact failure the watchdog backs up. The cost that is not area is the proof: the CLINT’s bounded check went from 6 s to 18 min 19 s and its *k*-induction proof from 2 s to 5 min 57 s, because eight XOR reductions of weight 26 over a 64-bit counter is a hard SMT problem in a way a 22-bit voted word is not (docs/58 section 9.3).

4.5 The memories

72 KiB of RAM and ROM was long the largest storage with nothing checking a word read out of it. docs/58 expected an easy fit, because the codec is (72, 64) and the RAM macro’s row is 64 bits.

That coincidence is the problem: a (72,64) codeword is eight bits *wider* than the row, and a hard macro has no bit-widening. Six options were costed from the vendor LEF; keeping 64 KiB needs a seventh macro at +491,633.62 μm^2 or +257,608.51 μm^2 *and* a read-modify-write on every sub-word store, which is a change to the bus protocol rather than a wrapper (docs/67 section 3).

What was built puts the check bits in the row and halves the RAM. A 32-bit word is stored as **four (16,8) byte codewords**, each the same Hsiao code [9] with 56 data bits tied off, which stores (13,8), the minimal SECDED width for a byte, beside three check rows identically zero over one byte (docs/67 section 3.3); four of them is exactly 64 bits, so there is no seventh macro, no die growth and no read-modify-write, a byte write landing through the macro's own per-bit write mask with its check bits beside it. **The RAM is 32 KiB, not 64 KiB**, a halving the bring-up program does not notice because it uses **192 bytes** of static data plus a stack, and an uncorrectable syndrome answers the read with a bus error, so **the recovery is a trap, not a wrong word**.

A word the program never touches is rewritten never, so the memories need a scrubber, and a bus master would have been a re-proof of a fabric whose arbiter is written for two. This one lives behind the row port: in an idle cycle it reads a row, in the next it writes back only the lanes the decoder corrected, and if the bus asks in between the bus takes the port. **Nothing is ever withheld from the bus**, which is proved, and a lane reported uncorrectable is *never* written back, since re-encoding a wrong byte would launder a detected double error into a valid codeword over a wrong value (docs/67 section 4.3).

The interval ships at 255 idle cycles, walking the 8192-row RAM in about 2.1 million idle cycles [estimate]. The campaign found the bound that matters and it is not the expected one: at interval 0 the scrubber repaired 135 of 360 injections against 36 at the shipped setting — *and missed what the slower walk caught*, because its pointer had left the program's low rows within the first hundred cycles and did not return inside the run. **A scrubber's coverage of a row is the interval between two visits to it** [22], which is why the interval is a register (docs/67 section 7.3).

Placed on the same die the codec costs 3.01 % of standard-cell area on a die that does not grow. The timing is the honest half: pre-layout the decoder fit on the read return with +2.0352 ns of slack, **post-layout it does not, and that margin is what layout spent**. The routed path from a macro's A_DOUT to its capturing flip-flop grows from 12.7286 ns to 14.0728 ns and that group's violating population from **68 endpoints to 1,080**, because a load's data now crosses the memory decoder *and* the register file's own SECDED encoder in one cycle — **two independent codes over one word**, which this project would not have chosen deliberately, and which is still not the binding path (docs/67 section 5.3).

4.6 Boot

Protecting a memory creates a problem at power-on. An SRAM powers up undefined and **so does its check field**, so a word nothing has written is 32 random data bits beside 32 random check bits, and the codec's answer is an uncorrectable, answered with a bus error (docs/68 section 4.1).

`boot_crt0.S` sweeps the whole RAM before the stack pointer is set, because the loader's own stack, `.data` and `.bss` are inside what it is initialising. And it writes **whole words**: a byte store makes one of a row's four codewords valid and leaves three as the SRAM powered up, so "initialise the RAM" is not the requirement — "write whole codewords" is. Both counterfactuals time out at 900,095 cycles with a double fault: the loader faults on the first RAM word nobody wrote, and *the trap handler reads a word nobody wrote too*. The byte sweep is the instructive one, leaving 1,648 uncorrectable rows against no sweep's 1,706 while looking initialised (docs/68 section 4.4). The sweep costs 49,178 cycles, **6.003 per word**; the image checksum is then computed *from the RAM* rather than the flash, so a word that arrived intact and landed on a row whose codeword

did not take is caught.

The boot block is 92 flip-flops and docs/69 protects **eighteen** — the boot counter, the sampled straps, the watchdog-pin readback, two validity bits, the sampling delay, and one the campaign added. That one is the finding. `sys_q` was ranked unprotected because it is rewritten from `rst_ni` every clock — true of the register, false of the consequence, since an upset there manufactures a release edge, the boot counter increments, and **the counter is saturating and power-on-only, so nothing ever takes it back**. Three of four draws came back as silent corruption and one put the part past its attempt limit for the power cycle, so it moved into the protected word at a cost of two flip-flops (docs/69 section 8.4).

4.7 Stopping the clock without losing the report

Power, and not frequency, is the requirement this design is written against. docs/57 measured 33.890 mW computing against 5.539 mW waiting at the typical corner, but on full13, a layout docs/57 section 3.2 itself found does not contain the accelerator; docs/76 section 2 could not reproduce that pair on the design that exists and replaced it with its own baseline, so the figures the gates were measured against are 33.359 mW computing and 18.745 mW waiting on the ungated control, becoming 45.199 mW and 8.287 mW with the gates in (docs/76 section 8.3). docs/76 added two clock gates, and the interesting part is what gating a fault-tolerant block gets wrong. **A clock-gated flip-flop still holds state and can still be upset; what it cannot do is report the upset**, because the voter's correction, the queue's parity discard and the cause bank's mismatch are all recorded by registers that need a clock. A sleeping accelerator would have absorbed the upset and told nobody, which is the defect docs/52 section 10 named and docs/55 closed. So all nine of the accelerator's cause lines are terms of its clock-gate enable: **an upset inside a sleeping block wakes it up** — one cycle later, not none, because a term that rises while the clock is stopped cannot be seen until the block is clocked again. docs/76 section 3.2 counts eleven such lines and puts the wake latency at zero; docs/77 section 14, items 3 and 5, corrects both.

The negative result is that neither clock-gating check closes at the slow corner — -0.7495 ns for the fabric's gate and -5.0198 ns for the accelerator's — and a violated setup check at a clock gate is a possible glitch on a clock net, a width this project has no instrument to measure. docs/76 blamed those fault lines and ranked their removal first; docs/77 cut the launch domains one at a time and found everything inside the accelerator worth at most -1.3129 ns, the -5.0198 ns launching instead from **the register-file read address that gives the whole design its worst slack**. The repair was built anyway, for a better reason: the enable is split by *arrival*, and a term that is a function of registers the clock has stopped cannot pulse and vanish, so the cost is **one cycle of wake latency on a fault and one flip-flop**, proved rather than hoped (docs/77 sections 1, 3 and 6).

4.8 W9: a correction that cost a reset

The sharpest result in the corpus came from measuring the netlist instead of the RTL. In a gate-level campaign on the watchdog's three replica banks, **174 of 174 injected upsets classified CORRECTED — and every one of them reset the SoC one clock later** (docs/74 section 10.2).

The mechanism is a decode. `rst_req_o` was `(rst_hold != 0)`, an OR-reduce of five bits of the *voted* word, and two of the three replica outputs are the mixed decode of their stored bits — an XOR tree over all 29 flip-flops of the bank. `soc_top.v` then hung the SoC's reset synchroniser off

Table 4.1: Resets per corrected upset: 174 injections into the same 87 bits at the same 174 cycles, on four netlists mapped on four separate occasions (docs/75 section 8.5). The row is the design and the column is nothing. In all four the vote behaved identically: 174 CORRECTED, one mismatch cycle each, and even *which* twenty replicas held their corrupted value rather than being rewritten — 4, 8 and 8 by bank — are the same.

	unrepaired	repaired
placed and routed layout	174/174 = 1.0000	0/174 = 0.0000
synthesis netlist	174/174 = 1.0000	0/174 = 0.0000

it *asynchronously*, under a comment reading “rst_req is a registered signal in this clock domain”. It was not. **The consumer documented the assumption and the producer did not meet it**, and nothing checked it (docs/75 section 5.1). That this is a hazard rather than an intended decode needed no simulation: of the five distinct asynchronous-reset source sets in the signed-off netlist, four are a port or one or two *registered* signals — and a flip-flop’s output does not glitch — while the fifth, rst_raw_n, has a combinational fan-in of **25 flip-flops of the three replica banks and nothing else** (docs/75 section 6).

The fix is one flip-flop. in_reset_q is the same predicate computed from the *pre-storage* word rather than the decode, so the two are equal on every cycle — proved by *k*-induction as W9a — and rst_req_o = in_reset && in_reset_q. The masking is logical rather than structural, so the mapper may factor it freely; that it survived mapping is proved by SAT on the mapped cells, with two mutations that make the proof fail, one of which keeps the flip-flop and the cell count and gets the logic wrong (docs/75 section 7.4). It cost **one flip-flop** and +643.0536 μm^2 on the whole SoC, with no die area, no hold margin at any corner and no DRC; a block-level area figure was published and then **withdrawn** on 2026-09-09 because it did not reproduce under any of nine recipes, and it is not quoted here.

4.9 The evidence

Every campaign classifies each injection into exactly one of five classes [4], decided in this order (docs/16 section 1.6): HANG, the run did not complete inside its cycle budget *and* nothing was flagged; DETECTED, a design fault flag or counter moved; SDC, the output differs from the golden model and nothing was flagged; CORRECTED, the output matches the golden model *and* a correction counter moved; MASKED, the output matches and nothing moved. **The golden model decides right from wrong, always**; the design’s own flags only decide whether a wrong answer was announced.

CORRECTED, rather than “never SDC”, is therefore the acceptance criterion for a protected bank, because **an injection that never reached a flip-flop comes back masked, and masked is exactly the quiet nothing a mis-targeted injector produces** (docs/41 section 8.2). But it cannot be copied: the register file’s campaign demanded it of every injection and failed at 104 of 124 *on a design that was right*, because a register the program overwrites before the scrub reaches it has shed the upset without the codec seeing it. The criterion there is now that nothing is SDC or DETECTED and every MASKED record is *explained* (docs/43 section 9.2).

Two entries in table 4.2 measure one mechanism and both are negative results, published as such. The windowed watchdog was built on a recommendation, proved, measured, and **its measured value on the first workload was zero**: 0 of 30 dead machines caught, 0 of 4 on the records replayed verbatim, seven spurious escalations bought. The recommendation was aimed at the wrong failure — a window fires on a kick that is too *early*, and the runaway that motivated it

Table 4.2: Fault-injection campaigns bearing on the hardening, with their paired counterfactuals. Every injection is one single-bit flip at a seeded cycle inside a measured window, into one flip-flop everywhere except the two memory rows: there the deposit is one bit exclusive-ored into a data or check word of one memory, 1,000 draws by region of the program’s own link map ([docs/67](#) section 7.1), and [docs/60](#) section 9.10 is explicit that a memory word is not a flip-flop of this netlist.

Population (source)	<i>n</i>	Result
Watchdog’s own state, RTL (docs/41 8.2)	162	126 of 126 into the protected word CORRECTED; 0 SDC, 0 HANG, 0 disarmed; later, 168 of 168 at 29 bits
counterfactual, HARDEN = 0	32	27 SDC, 1 HANG, 2 left the watchdog reading disarmed ; 6 of 32 kept the escalation order
Ibex core, campaign A (docs/43 8.2)	1,400	Register-file strata 0 SDC and 0 HANG in 200, against 3 and 4 before, and 0 uncorrectable syndromes; design-weighted SDC 2.8 % $\rightarrow$ 1.3 %
Ibex core, campaign AW (docs/43 8.4)	1,400	The window rejected a kick as early on 11 draws, 0 on dead machines, and produced 7 spurious escalations in 1,232 survivable upsets
Supervisor workload (docs/46 8)	812	32 of 32 dead machines caught, all by ordinary expiry; the window converted 2 silent corruptions into announced failures and caused 2 of 4 spurious escalations
Register file alone (docs/43 9.2)	124	107 CORRECTED, 17 MASKED, 0 unexplained; without the scrub, 6 unexplained — upsets still in storage that nothing had seen
CLINT, hardened (docs/58 8.3)	342	<code>mtime</code> 128 of 128 and <code>mtime_chk</code> 16 of 16 CORRECTED; clock displaced 0 times; all 144 announced
counterfactual, HARDEN = 0	326	95 of 128 displaced the clock, worst 9,223,372,036,854,775,808 ticks (5,845 years), 0 announced
Memories, two builds (docs/67 7.2–7.3)	720	0 SDC, 0 traps, 0 hangs ; 104 corrected at the shipped interval, 168 at interval 0
counterfactual, no check bits	280	17 silent wrong answers, 17 announced failures, 1 hang; 15 of the 17 from one constant the program reads every round
Boot block (docs/69 8.3)	572	276 of 276 into the protected word CORRECTED; 0 SDC, 0 spurious give-up
counterfactual, HARDEN = 0	368	261 SDC; 38 corrupted the boot counter, worst by 128 boots , 11 past the attempt limit, 8 never reaching it
NPU connection (docs/52 1)	700	4.2 % design-weighted silent corruption; 7 of 600 killed the machine, and the watchdog caught 7 of 7
Core at gate level (docs/74 1)	1,424	1,158 of 1,175 comparable records classify as at RTL; the register file corrected 171 of 180 <i>in the gates</i>
Watchdog banks, gate level (docs/75 8)	174	174 CORRECTED on each of four netlists — and table 4.1

was petting the watchdog on schedule (docs/43section 5.1). What catches that class is a per-phase kick budget, 4 of 4 on the same records. The window also bounds the jitter ratio of the program’s kick cadence: the original workload measures **57.2**, which fits inside no window at all (docs/43section 4.3), and the supervisor written to be what the software will be measures 1.412 time-triggered and 2.104 work-triggered — **at which no timeout satisfies both bounds**. docs/46section 9 therefore recommends removing it, which has not been done.

4.10 TMR blindness

A synthesiser is built to delete deliberately redundant logic [5], and this project has been burned by exactly that: `pilot_top.v` section 9 records the configuration TMR merging away entirely. The project’s own paper [1] states the general form: a redundancy claim “was checked at three levels and was wrong at two of them”. At the register-transfer level it was true; in the mapped netlist it was false, structural hashing having collapsed three declared replicas into one physical bank — **and the RTL fault-injection campaign built to detect exactly that produced bit-identical output on the defective and the repaired design**, because it deposits into signals the netlist no longer contains (`paper/main.tex`, `abstract`).

Every functional test here would pass on a netlist with one replica instead of three, and so would every proof: of five mutations against the composition proof, the last is the honest negative, because one bank answering three times satisfies *every* property (docs/41section 9.4). **The only honest place to count replicas is the netlist**. The census in `sw/tests/test_soc_synthesis_guards.py` is built to be able to fail: with `keep` and `keep_hierarchy` deleted from the *text* of a scratch copy and the flatten forced, the count does not move on either of two technology mappers — while the mutations it does fail on are invisible to everything else, `.MIX(0)` on replica C costing 11 flip-flops despite being functionally identical RTL (docs/41sections 6.2 and 6.3).

Counting is necessary and not sufficient, and docs/75 found out how. Three instruments answer “which flip-flops belong to this replica” and they disagree. By **instance path**, what every guard uses, the watchdog’s banks are 29/29/29; by **Q-net name** they are 29/14/15 on the shipped netlist and 0/14/15 on the block synthesis, because one of B and C resets to one on every bit, `dfflibmap` legalises that as a reset-to-zero flop behind an inverter, and *the resulting cell is not the cell the name was attached to*; by the voter’s **fan-in cone** they are 29/29/29 again (docs/75sections 3.1–3.4). A name is a hypothesis.

And a count of the right size can be a count of the wrong thing. Changing one argument so the voter reads replica A twice produces a design that passes *every* check in the repository — per replica, total, attribute-free, both mappers, every test, every proof — and whose vote is a majority over two distinct values, so an upset in replica A is not masked at all. The union of the three cones falls from 87 to 58; **the cone census catches it and nothing else does**, and only if it anchors on the voter’s own input pin (docs/75section 4.3). Two rungs followed: that census run over every whole-SoC netlist the tree has kept; and `formal/eqy/`, which checks that the netlist computes the RTL’s *function* for `tmr_voter` — the one module a flip-flop census cannot check, having no flip-flops — with two negative controls whose passing the Makefile treats as an error (docs/79section 2).

The last rung is the die, and there the claim does not survive intact. On the frozen pilot’s sign-off layout the banks are not placement-separated: minimum cross-replica separation is 3.78 μm centre-to-centre, the floor for two cells in adjacent rows, and 49 pairs abut. Against a null model shuffling only the replica labels over the same placed positions the banks are indistinguishable from an arbitrary labelling — 110.73 ± 8.26 expected, 103 observed — so the finding is not that the placer drew them together but that **it did not keep them apart**. The mechanism shows as a **4.12-fold concentration on corresponding bits**, median 27.52 μm against 113.47 μm over

Table 4.3: Measured area of each mechanism, against a baseline measured in the same session with only the thing being priced turned off. Gate equivalent is `sg13g2_nand2_1` = 7.2576 μm^2 ; the Ibex reference is 275,682.6198 μm^2 . Deltas are arithmetic on two measurements [estimate]; every measured row is [fact]. Rows marked † are deltas on the whole SoC rather than on one block.

Mechanism	Δ flops	Δ area (μm^2)	of Ibex
Watchdog TMR, W6 (<code>docs/41</code> 6.5)	+49	+5,091.0552	+1.85 %
Windowed mode and kick budget (<code>docs/43</code> 7.3)	+29	+4,904.0964	+1.78 %
SECDED register file (<code>docs/43</code> 7.1)	+222	+38,050.5762	+13.80 %
of which the scrub, module alone (<code>docs/43</code> 7.2)	+5	+9,963.5130	—
<code>mtime</code> SECDED, H6 (<code>docs/58</code> 7)	+8	+7,026.8310	+2.549 %
its report in BUSSTAT	+18	+1,917.4050	+0.695 %
Boot decision word, B1 (<code>docs/69</code> 1)	+51	+5,255.9010	+1.9065 %
Memory ECC and scrub engine† (<code>docs/67</code> 5.1)	+234	+31,391.3124	—
W9, registered reset request† (<code>docs/75</code> 1)	+1	+643.0536	—
Registered clock-gate wake† (<code>docs/77</code> 1)	+1	+829.4454	—
<i>Declined:</i> lockstep, for <i>detection</i> (<code>docs/38</code> 8.5)	—	+309,550	+112 %
<i>Declined:</i> TMR over <code>mtime</code> , <code>mtimecmp</code> (<code>docs/58</code> 4)	+256	+25,792.0740	+9.356 %
<i>Declined:</i> residue-mod-3 detector (<code>docs/58</code> 4)	+2	+6,935.0904	—

all cross-replica pairs, which is the shape a shared voter’s wirelength would leave; and the same picture appears on `soc_top` across all three of its TMR structures (`docs/79` sections 1.2–1.6).

The limit is carried here unchanged from `docs/79` section 3. **A distance is not a safety argument in either direction.** No single-event cross-section or LET threshold for this process was measured and none is available; the measurement establishes that this layout *permits* a common-mode upset over much of the protected word [17] and says nothing about whether one occurs at any fluence. “Because the replicas are a few micrometres apart, one ion track defeats the vote” is therefore forbidden: it silently supplies a conversion this work does not have. What it does change is the prose — every unqualified sentence saying the configuration is protected by triple modular redundancy must read *triplicated in the netlist*. The measurement does not make those sentences false; it removes their evidence.

4.11 What it costs

Two habits sit behind [table 4.3](#), both learned by nearly getting a number wrong. **The baseline must be the same source measured now, with only the priced thing turned off**, never a figure a previous document published: three waves measured that gap at 58.17 μm^2 , 25.4394 μm^2 and 10.9242 μm^2 , the last in the direction that would have *overstated* the hardening (`docs/41` section 6.5; `docs/58` section 7; `docs/69` section 7.1). And **a module measured alone understates what it costs in place**: the register file’s standalone delta is +31,709.55 μm^2 against a whole-core +38,050.58 μm^2 , 20 % larger, because `abc` re-optimises across a boundary the flattened core does not have (`docs/43` section 7.2). One row breaks the first habit: the memories’ delta is against HEAD’s 64 KiB design, not against a control with only the code off, which `docs/67` section 5.1 puts at +26,932.9914 μm^2 and +211 flip-flops.

The costs that are not area. Every RTL hardening wave costs *zero cycles*: each reproduced the whole-SoC self-test’s cycle count exactly — 185,443 (`docs/41` section 9.1), then 215,428 (`docs/58` section 8.5), then 217,682 (`docs/67` section 7.4), then 220,256 (`docs/69` section 10) — because in a fault-free machine every decoder is the identity, proved by induction, so the invariant moves when the design gains content and never when a protection is added. The last move is the boot flow’s, and is not a protection either: a program run from RAM rather than the ROM

costs 1.207 % for sharing one slave port (docs/68 section 8). The scrubs cost a period rather than cycles: one clock for `mtime`, 31 for the register file when the core is idle and unbounded in the limit, one walk of 8,192 rows per 2,100,000 idle cycles for the memory. The boot sweep costs 49,178 cycles, the window costs software a jitter bound, and the clock gate one cycle of wake latency on a fault. The codecs cost 2.7928 ns on the register file's read path, of which 0.2593 ns was recovered for +1,201.5486 μm^2 (docs/44 5.4), and +1.3442 ns on the memory's routed read return.

One of those timing numbers deserves its own sentence, because the first version of it was wrong. docs/43 section 7.4 attributed a setup-slack loss, and a closure period moving from 15.98 ns to 17.62 ns, to the register file's decoder. docs/44 looked at the paths. **All ten worst setup paths at the slow corner, in both configurations, start at `cheriot_enable_i[0]` — a configuration input `soc_top.v` drives with a constant**, which a register-file decoder cannot be on; and the binding check in the hardened build is the *asynchronous reset recovery check* on an unbuffered 2,328-fanout net, bit-identical between the two hardened builds. What the protection cost there is 222 more flip-flops on an unbuffered reset net, a place-and-route obligation and not a logic cost. **The delta was real and the attribution was not** (docs/44 sections 5.1 and 5.3). Consistently with docs/53 section 9.2 and docs/05 section 4 rule 5, *no clock frequency is claimed for this design anywhere*; slack against a 20 ns SDC period, with its corner named, is what is reported.

4.12 Four shapes an error takes here

The project keeps a catalogue of the mistakes it makes, because the shapes repeat and the catalogue is worth more than any single number in it (docs/64 sections 2 and 3; ROADMAP.md section 3). Four bear on hardening. They are findings, not apologies.

A green result read wider than what it looked at. docs/41 section 6.6 lists eight instances by document and section and docs/43 section 10 makes it ten; docs/58 section 11 counts nine over a list that drops two of those and adds one of its own, so the two documents disagree about the tally and not about the shape. The canonical case is the netlist census: it counts flip-flops and asserts nothing about placement, and nothing about whether the replicas are *correct*. The fix is never to widen the claim; it is to build a second instrument at the level the first cannot see, with a mutation that makes it fail — which is why the cone census, the shipped-netlist guard, the equivalence check and the gate-level campaign all exist. One instance is the shape at its purest: a mutation harness reported a pass because a build failure produced no FAIL lines to parse.

A measured delta attributed to the wrong cause. The register file's timing cost is the clearest case: the number moved, the explanation was wrong, and only looking at the paths found it. The clock-gate diagnosis of section 4.7 is a second, and docs/61's 4.4 % estimate — an earlier absolute area carried onto a new denominator, measured at +4.9406 % by docs/62 section 1 — is a third, where the method was the error rather than the arithmetic. The defence is a measured threshold: docs/44 section 5.2 put this flow's resolution at about 0.4 ns at synthesis, and docs/70 measured it after layout with a control pair that changes nothing a designer would call a change — it moves hold by 0.0198 ns and setup by 0.5031 ns. Neither is instrument noise: one netlist laid out twice reports byte-identically, so both bound the flow's sensitivity to a *changed* netlist (docs/71 section 7).

One change leaking in from several places. A delta is only a delta if the two runs were asked the same question, and the answer is a paired counterfactual on identical draws.

`docs/55` section 8.3 had to refuse a 19 % improvement because its control stratum moved on 15 of 100 records; `docs/58` reports its delta because a field-by-field diff over the strata it does not touch found **0 of 198 records changed**, and `docs/69` found the same zero over 296. `docs/44` re-ran the entire core campaign and found the two 1,400-record files *byte-identical*, which is stronger than “the rates agree”, since two records moving in opposite directions would satisfy that too. The same discipline forced the isolation builds of `docs/43` section 5.4: with both the window and the budget armed, both counters have fired by the end of a run that reset itself, and a report reading them would have credited the window with a catch it did not make.

The synthesiser deleting redundancy. Section 4.10. The hazard is that *everything else stays green*: the collapsed design satisfies every property of the composition proof, passes every functional test, and produces bit-identical fault-injection output. Redundancy is the one property whose evidence cannot live in the source, because the source is precisely what the mapper is entitled to rewrite.

4.13 What is still open

No radiation data exists for this process. Neither open PDK publishes total-ionising-dose or single-event figures, no silicon of this SoC exists, and no beam time has been taken (`paper/main.tex`, abstract; `docs/79` section 3). Every reliability number here is a seeded fault-injection measurement on RTL or a placed netlist, conditional on an upset having landed inside a measured window, and the project has no upset rate λ (`docs/67` section 6.2). **Every campaign is single-bit and flip-flop only**, with no multi-bit strike and no transient in combinational logic [25] — a larger gap for a code than for a replicated bank, since a code’s defence *is* combinational logic, the CLINT’s exposed codec alone being 528 cells against a 22-bit majority gate (`docs/58` section 11). **Addresses are unprotected everywhere** (`docs/67` section 9; `docs/43` section 6.6), and **no campaign has reached any peripheral** — 1,003 to 1,051 of the placed design’s 5,873 flip-flops are in a block none has injected into, the fault counters entire (`docs/60` section 9.10). **The ROM’s contents are not protected in a flight part**: on this PDK the boot ROM is an SRAM that powers up undefined and nothing in the design can write it, so its check macros are check bits nobody can initialise, and the flight answer is a mask ROM whose bit is a via and has no stored-upset mechanism (`docs/68` section 3). **And the vendor macros cannot be signed off on this PDK version** [10], so neither can the die that carries them (`docs/67` section 11).

Chapter 5

Verification

5.1 Five layers, and the failure they all share

The programme has five layers: cocotb simulation of blocks and of the whole system; *guards* that read the tree without simulating it; SymbiYosys property sets over blocks; an instruction-set proof of the management core against riscv-formal; and fault-injection campaigns at register-transfer and gate level. They are five and not one because each can be green about something it never looked at. docs/41 section 6.6 says this repository has been bitten eight times by a green check read wider than what it examined; docs/78 section 3.2 records the third such instance inside this project’s own guards — the first in a guard written to prevent the shape.

Four rules govern reporting. Every assert set ships with cover obligations, and a target with passing asserts and failing covers is *red* (docs/09 section B.1); a bounded result is never called proven and carries its depth (docs/09 C.4 item 6). Timing is outside formal scope, OpenSTA being the separate authority (docs/09 C.4 item 2) — and **no clock frequency is claimed for this design at all**, by docs/05 section 4 rule 5, which docs/53 section 9.4 applies to both numbers a layout has reached. Formal proves mechanisms correct, fault injection measures architectural sensitivity [19], and only beam data speaks to cross-sections [6] (docs/09 C.4 item 4). Where a count here supersedes one standing in a document, both are given (docs/64).

5.2 Simulation: the cocotb suites

Counted in the tree on 15 September 2026: **33 cocotb test modules carrying 413 test functions** — hw/tb 10 modules and 166 functions, hw/soc/tb/cocotb 22 and 242, tt/test 1 and 5 — driven by 30 makefiles, all of which scripts/run_cocotb.sh reaches. That count is a floor on what a run reports: several makefiles elaborate one module at several parameter points. The last recorded whole-tree run is verification-log.tsv’s row of 2026-09-11 at 587fdd8: **462 passing, 0 failing, 0 suites not clean**. Three are skipped by default: one rewrites hw/tb/fi_campaign_results.json and so dirties the frozen tree scripts/verify.sh watches — hw/rtl, hw/tb, tt, formal, hw/openlane — and the two gate-level suites need a hardening run’s artefacts (scripts/run_cocotb.sh).

The runner is itself an instrument with a repair history. It parses each suite’s results XML rather than trusting the make exit code, because a suite can pass every test and then segfault in Icarus teardown. And it **refuses to run twice at once**, because counting is by “the XMLs written since this marker”, which orders by time and not owner: on 2026-09-08 two overlapping runs recorded **558 and 624 cocotb tests for a 401-test repository**, and both landed in the log looking like measurements. Those rows still stand, superseded by a later row saying so, because

a log that quietly loses its own bad measurements is not a log.

It also carries a **completeness check**: a `test_*.py` named by no makefile and by no table is an error when added, rather than a suite that quietly never runs. On 2026-09-11 three modules did not run. All three were green, all three were invisible, and an edit re-opening the defect two of them exist to catch would have passed every check in the tree.

5.2.1 What only simulation can see

The block suites drive a module’s ports against its contract: `test_soc_npu` carries 35 test functions and `test_soc_boot` 18. Four are campaigns rather than tests: the watchdog’s carries 220 injections inside three functions and the register file’s 124 inside six (`docs/44` section 8.3), and `test_soc_boot_fi` and `test_soc_clint_fi` inject into the boot block and into the time base’s 72 stored bits, 64 data and 8 check.

What simulation sees that no other layer does is **a whole machine executing a program**. `docs/38` section 7 runs a self-checking C program on the sv2v output of Ibex under Icarus, with no SystemVerilog in the flow: eleven checks over fetch, the ALU, six of the eight RV32M operations including the ISA’s divide-by-zero results, loads and stores, branches, recursion, compressed code, CSR access, traps, and memory protection — a locked read-only region permitting a load and faulting a store. All eleven pass in 131,694 cycles. On the hardened system the run is 22 checks and 185,443 cycles, and `docs/43` section 9.1 reports it cycle-identical to the run before the register file was substituted — one of the arguments `docs/44` section 9 gathers under *an expectation is not a measurement* before re-running the campaign anyway.

Its limits are one workload, one seed, one parameter point — and at gate level a tool limit: the IHP cell models [10] drive their outputs from the negative-timing-check outputs of `$setuphold` and `$recrem`, which Icarus 12 does not implement, so every flip-flop output sits at x and the design reads back zeros. That is 1 of 5 passing on the shuttle suite (`docs/15` section 5.4, whose run that section marks stale, being against a superseded netlist) and was 21 of 21 failing in `hw/tb` (`docs/24` section 3.1): both read like a dead netlist and neither is one. The gate-level makefile refuses to run under Icarus 12, and the shuttle suite, being frozen and generated, has that check applied on its behalf by the runner.

5.3 Guards: the checks that do not simulate

A guard is a pytest that reads the tree — RTL text, a generated netlist, a flow script, a document — and asserts a property of it. On 15 September 2026 pytest collects **527 tests across 25 files** under `sw/tests`, superseding `verification-log.tsv`’s 2026-09-11 row of 509.

Run whole that evening it reports **527 passed, 0 failed, in 706 seconds**. An earlier run the same day is the more informative one and is kept here rather than discarded, by `docs/64`’s rule: **523 passed and 3 failed in 881 seconds**, and all three were the suite catching its own tree mid-edit — a formal-count guard and a document-link guard whose documents acquired the recount to 83 tasks and the index row for `docs/82` while the run was in flight, and a guard forbidding a second copy of the frozen accelerator source, which tripped on a SymbiYosys scratch copy and was repaired at 20:08 the same day. That repair is this section’s principle: a directory is a solver run directory when it *contains sby’s own config.sby*, because those directories are called `<job>_bmc`, `soc_npu`, `regfile_scrub` and the next will invent another spelling.

Guards exist because the cocotb suite and the formal job check *behaviour*, and neither can notice that a constant was copied instead of generated, or that the frozen directory has been written to. The boot guards state the family’s placement rule — a guard belongs at the stage that can satisfy

Table 5.1: The mutation only a connectivity census can see (docs/75 section 4.3).

Check	intact	mutant
Flip-flops per replica, by instance path	29 / 29 / 29	29 / 29 / 29
Total flip-flops in the block	132	132
Fan-in cone per voter input	29 / 29 / 29	29 / 29 / 29
Union of the three cones	87	58
Flip-flops shared by two inputs	0	29

it: `soc_boot.sby` proves the boot counter is not writable and says nothing about that block’s reset being the *system* reset, which is what makes it count boots.

5.3.1 The principle: check the property, not the name

`scripts/spdx_check.py` once reported itself as licence-tagged while carrying no header. It searched the first 4,000 characters for the identifier string in any context — and its own docstring and regular expression both contain it, so **the checker read its own prose as its own licence**. A tag now counts only on a comment line in the first twelve.

The third instance is a guard failing at guarding; the second, the verification recorder’s glob, is below. The public-mirror generator had to rewrite one sentence in a CI workflow before publishing it, and refused to run if that sentence changed — anchored on the *filename*. When the file was deleted the guard did not fire: it was skipped, and the generator wrote 511 files as though nothing were owed. docs/78 section 3.2: **a check that cannot fail when the thing it checks is absent is not a check**. The same rule chose the anchor in `test_soc_shipped_netlist_guard.py`: a replica is identified by the fan-in cone of *the voter’s own input pin*, because where the voter reads one replica twice the other bank’s output wire is still driven — it is simply not what the voter reads.

5.3.2 Two guards that caught something

The voter wired to one replica twice. A test changes one argument in a scratch copy of `soc_wdog.v`; Table 5.1 is what each instrument then says. A `keep` attribute holds the unread replica’s flip-flops in the netlist [5], so the mutant has three intact banks whose vote is $\text{maj}(q_a, q_a, q_c) = q_a$ — **an upset in replica A is not masked at all**. Every count in this repository passes on it, every functional test passes, every proof passes, and the cone census fails — not an instrument that undercounted, but one that counted the right number of the wrong thing.

A fault line that carried a bus signal. `clkgate_wake.sby` proves a transformation of the accelerator’s clock-gate enable from two side conditions, one of which (its A2) is that the slow half of that enable is a function of the block’s state alone — so it cannot move while the state is frozen. A proof cannot discharge its own assumption, so `test_soc_clkgate_guard.py` does: it elaborates and flattens the module, walks the enable’s fan-in cone cut at every sequential cell, and fails if any input port is reachable. It found `C_INJ_OVF`, the one fault line carrying `pse1_i`, which no reading of the source had noticed.

5.3.3 What guards cannot do

They compare port *lists* and not port *meanings* (docs/43section 3 names that an accepted risk), and the netlist guards read git-ignored build products, so on a fresh clone they skip — and say that a skip is evidence of nothing. One failure was the *counting* layer’s own: until 2026-09-09 scripts/verify.sh enumerated formal results with a two-level glob over two directories, and riscv-formal’s jobs sit four to six levels down elsewhere, so the glob reached none and the log recorded formal_pass=118, formal_other=0 for a tree with 35 non-passing formal jobs in it. Enumeration is now every directory holding both a config.sby and a logfile.txt; the counts became 425 and 35, and formal-dispositions.tsv carries all 35 with a reason and a date: **6 FAIL, 3 ERROR, 26 STOPPED**.

5.4 The SymbiYosys property sets

Counted on 15 September 2026: **28 property sets carrying 137 tasks** — 8 sets and 54 tasks under formal/ for the frozen pilot, 20 and 83 under hw/soc/formal/. docs/63section 2 states “87 sby tasks across 16 property sets”; true when written, superseded here, both given because the document still says it. All 54 pilot task directories carry a PASS marker. All 83 SoC tasks have a workdir and 79 of them a recorded verdict — **76 PASS, 2 TIMEOUT, 1 UNKNOWN** — and four have none: two stopped by hand, two still running. Table 5.2 lists them all.

Table 5.2: Every SymbiYosys job under hw/soc/formal/, read from the job headers and that directory’s Makefile. Engines are smtbmc yices unless named; verdicts are those recorded on 15 September 2026.

Job	What it binds	Tasks and depths	Recorded
soc_bus	soc_bus.v, F1–F9; decode proved against the generated map	bmc 24, prove 12, cover 24	3 PASS
soc_apb_bridge	soc_apb_bridge.v, A1–A11, clauses of AMBA 3 APB	bmc 24, prove 12, cover 24	3 PASS
soc_apb_wb	soc_apb_wb.v, I1, W1–W8, P1–P3, D1–D2, clauses of Wishbone B3 and APB	bmc 24, prove 12, cover 24	3 PASS, 2026-09-15
soc_clint	soc_clint.v, C1–C4, P1–P3, E1–E2 (the SECDED time base)	bmc 20, prove 10, cover 20	3 PASS
soc_wdog	soc_wdog.v over three replica banks and a voter; W1–W5, W7–W9	bmc 40, prove 15, prove_w0 15, cover 60	4 PASS
soc_wdog_tmr	three replicas under one voter, with a free fault at the voter	seven prove tasks at 4, bmc 20, cover 8; boolector, abc pdr for _abc	9 PASS
soc_busstat	soc_busstat.v, B1–B6	bmc 20, prove 10, prove_w16 10, cover 24	4 PASS
soc_gpio	soc_gpio.v, P1–P7	bmc 20, prove 8, prove_w16 8, cover 20	4 PASS

Job	What it binds	Tasks and depths	Recorded
soc_qspi	soc_qspi.v, Q1–Q8	bmc 40, prove 12, prove_full 12, cover 110	4 PASS
soc_scrub	soc_scrub.v, C1–C7	bmc 20, prove 10, prove_w16 10, cover 24	4 PASS
soc_boot	soc_boot.v, D1–D10, at two counter widths and at HARDEN = 0	bmc 20, prove 10, prove_full 10, prove_h0 10, cover 24	5 PASS
soc_mem_ecc	soc_mem_ecc.v over a four-row array, M1–M6	bmc 10, prove 4, cover 14	3 PASS
byte_secdec	the (16, 8) byte shortening of the codec, four per RAM row	four tasks at 4; abc pdr for prove_abc	4 PASS
regfile_secdec	the (40, 32) shortening the register file stores	four tasks at 4; abc pdr for prove_abc	4 PASS
soc_npu_ser	soc_npu_ser.v, the transport’s P, H and D properties, at two rates	bmc 30, prove 16, cover 220, prove_h3 16	4 PASS
soc_npu	soc_npu.v with its whole submodule closure; H5 over all reachable states	bmc 30, prove 16, cover 120	3 PASS, 2026-09-14
clkgate_wake	clkgate_wake.v, an abstraction; T1, T2, W1–W4, and the same tasks with A2 dropped	bmc 24, prove 12, cover 24, each also in an _upset group	6 PASS
clkgate_wake_gnt	clkgate_wake_gnt.v, the wakefulness-qualified grant; G1–G3, T2	bmc 24, prove 12, cover 24	3 PASS, 2026-09-15
regfile_scrub	ibex_regfile_secdec.v at SCRUB = 1, <i>real</i> codec; R1–R4	prove 40, bmc 40, cover 40, prove_pdr, bmc_btor; yices, abc pdr, boolector	cover PASS; prove_pdr and bmc_btor TIMEOUT at their bounds; prove and bmc KILLED by hand after 12h
regfile_scrub_abs	the same wrapper, codec abstracted to secdec_identity.v	prove 40, bmc 40, cover 40, prove12 at 12, prove_pdr unbounded; abc pdr for it	prove_pdr PASS (44s), cover PASS, prove12 UNKNOWN; prove and bmc still running

Four design rules are visible there. **Properties live beside the design**, in a `_props.v` file included at the end of the module body under ‘`ifdef FORMAL`’, so the fault-injection flows never see them. **A narrow parameter point serves the covers and a second task the shipped width**: `soc_busstat` is proved at a counter width of 4 and again at 16, because the saturation covers are the point of the job and a counter needing 65,535 increments cannot be driven to its top in a bounded model. Four other jobs do the same. **A copied parameter is guarded by reading the RTL back**: `soc_wdog_tmr.sby` rebuilds the composition `soc_wdog.v` builds, because a fault model needs free inputs at the voter, and a makefile target reads the copied masks back out of the RTL. The boot block instantiates the same composition at a 23-bit word, which is why the job is proved at five widths — 4, 5, 21, 22 and 23: a width it is not proved at is a width it does not cover.

Engine diversity is a task, not a footnote (docs/09 C.4 item 7). And the jobs state what they do *not* prove: every property in `soc_wdog_tmr` stays true of a design in which the three

banks were hashed into one, the anti-merge argument belonging to the netlist; and nothing in `soc_mem_ecc`'s harness injects a fault, since a formal harness that deposits into a row is proving a property of the harness.

The `all` target runs 18 of the 20 jobs. `regfile_scrub` is excluded on purpose — every engine stalls on the codec's parity network, and a target that never returns is not a regression test — and `clkgate_wake_gnt.sby` is not named in the makefile at all, so its three recorded passes are reproducible only by invoking `sby` directly.

5.5 riscv-formal on the management core

`docs/09` part B track 1 named `riscv-formal` in August 2026 and nothing was run with it until `docs/63`: the bring-up, in two phases — the stock Ibex, then the same proofs against the SECDED-protected register file the system ships.

5.5.1 Binding it, which is where the work was

`docs/09` B.1 recorded that Ibex exposes trace ports. True of the SystemVerilog; the Verilog-2005 this system is built from contains **zero** occurrences of `rvfi`, the ports sitting inside an `'ifdef` the conversion does not set. A *second* tree was therefore generated rather than the first one changed: 34 files against 34, 16,770 lines against 15,873, 132 `rvfi` occurrences against 0, zero converter errors in both.

Then the finding that decided the harness's shape: **Ibex's trace block reads sixteen signals out of its submodules by hierarchical reference**, up to three levels deep. The converter carries them through verbatim, and Yosys's Verilog-2005 front end treats every dotted name as an implicitly declared net and stops, with no flag to change it. The pinned toolchain's SystemVerilog front end resolves all sixteen, at the cost of an assumption: **the front end that elaborates the core in this proof is not the one that elaborates it in the build**. The only corroboration is `docs/38` section 5, which measured the two on the same source at 0.89 % of area and two flip-flops apart.

5.5.2 The ledger, and the vacuity control

Nine assumptions are ledgered (`docs/63` section 4.1), and the generated jobs carry the switchable ones in plain text so a `grep` lists what was assumed per check. A1 and A2 are the bus contract; A3 ties bus errors low, at the stated cost that nothing here says anything about the core on a bus error while the fabric raises one on every unmapped address; A4 and A5 keep the memory-protection unit transparent so the privilege-blind instruction models can be compared at all, at the cost that **none of this proves protection enforcement**; A6 bounds memory latency; A7 substitutes a transparent clock gate; A8 excludes debug entry; A9 is an adapter. Interrupts are *not* assumed away: all five lines are free every cycle.

Vacuity control matters more here than anywhere else. Every instruction check assumes the instruction retires at the check cycle; if the depth is too small that assumption is unsatisfiable, there is no trace, and **the check passes having examined nothing**. The sequential divider takes about 37 cycles, so a divide check at the default depth of 20 would be a vacuous pass. The configuration is therefore expanded *twice*, into a bounded-check set and a cover set, and both are run; the solver front end reports PREUNSAT for unsatisfiable assumptions, which `sby` maps to ERROR; and the cover obligation is stronger still, requiring the instruction to retire *and not trap*. **79 of 79 cover jobs pass**, at 1,122 job-seconds against the bounded set's 7,790. Only two of `riscv-formal`'s checkers carry covers at all, so for the other seven consistency checks the anti-vacuity argument

Table 5.3: riscv-formal on Ibex, both phases, at the depths of docs/63section 8.1 and under its section 4.1’s nine assumptions. The two job sets differ only in which file supplies the register module.

	Phase 1, stock	Phase 2, SECDED
bmc, 79 jobs	70 PASS, 3 FAIL, 6 stopped	69 PASS, 3 FAIL, 6 stopped, 1 ERROR
cover, 79 jobs	79 PASS	79 PASS
Instruction checks (70)	62 PASS, 2 FAIL, 6 stopped	identical
Consistency checks (9)	8 PASS, 1 FAIL	7 PASS, 1 FAIL, 1 ERROR
Wall clock at -j8	50 min 52 s	54 min 45 s
CPU as attributed	9,076 s	14,027 s ($\times 1.55$)
Cover set, job-seconds	1,122	2,154 ($\times 1.92$)

is PREUNSAT plus the `cover` job’s own obligation that two instructions retire, and nothing else (docs/63section 5).

5.5.3 What closed, and what did not

Table 5.3 is both phases beside each other.

What is now proved that was not: for **62 of the 70 instructions of RV32IMC**, on every execution the core can reach from reset within that check’s depth, the instruction’s effect on the architectural state — destination register and value, source register numbers, next PC, memory address, byte masks, store data, whether it traps — agrees with riscv-formal’s model of the ISA. Add eight consistency properties over the retirement stream, and a bounded *proof* that the core never has more than two bus requests in flight on either port.

The phase-2 ERROR carries a correction of 2026-09-09: the cell first read TIMEOUT, and the job’s log says DONE (ERROR, rc=16) after a signal at 1 h 41 m 26 s on a configuration with no timeout option. **TIMEOUT means a bound was set and reached, so the measurement is complete; ERROR from an outside kill means no bound was set and nothing was measured.**

5.5.4 Five defects, and the one in the specification

None is in Ibex’s execution of an instruction. Three are in the harness — an assumption that guarded two CSR ranges and needed five; a bus response permitted in the grant cycle, whose counterexample is a *real* reachable deadlock, so a zero-latency memory hangs this core; and an assertion bounding the data port at one outstanding request when a misaligned access makes it two. The fourth is a mismatch between Ibex’s trace and riscv-formal’s definition of it, worked around by A9.

The fifth is in the *specification*. A division check failed on $-347 \div 1,075,056,129$: Ibex answers 0, correct for signed division truncating toward zero, and riscv-formal’s model answers 3, which is $4,294,966,949$ divided by $1,075,056,129$ — *the same bit pattern read as unsigned*. The remainder check failed the same way. The cause is Verilog signedness propagation in three lines of upstream’s model, checked in a second tool rather than argued from the language reference: Icarus Verilog evaluates that expression on those operands to 3, and the same division on its own self-determined wire to 0. Phase 2 reproduced it on fresh operands with negative *divisors*.

Correcting the specification made the proof *harder*: against the defective model the checks failed in 852 s and 731 s, finding one counterexample being a satisfiability question, while against corrected models they had proved nothing after 40 min 43 s and 48 min 52 s. And docs/63section 6 states the residual risk: a model wrong in the same direction as the core would be invisible to the whole exercise.

Table 5.4: `reg_ch0` against check depth on both register files (`docs/63` section 18.4; its section 20 for the depth-25 cells). The stock column is not monotonic because these checks assert at *one* cycle, so a deeper check is a different problem and not a superset.

Check cycle	Stock register file	SECDED register file	Ratio
12	5 s	6 s	1.2
15	14 s	15 s	1.1
18	31 s	353 s	11.4
21	66 s	TIMEOUT at 14,400 s	> 218
23	411 s	TIMEOUT at 14,400 s	> 35
25	189 s alone, 399 s in the run	TIMEOUT at 14,400 s, six engines	> 36

5.5.5 The M extension, and the register file

All eight instructions of the M extension have no formal result in this project. Six were stopped after 35 to 40 minutes each in phase 1; `docs/63` section 22, added 2026-09-15, re-ran all six on both cores under a second solver with a 7,200 s bound in the job, so the status is the tool’s own word — and **none of the twelve closes**. The covers are the finding: all six pass, at 1,388 to 10,999 seconds against 9 to 73 under the first solver, so a change of back end is not the route out. **The delta between the two phases over the M extension is undefined, not clean:** if the substituted register file changed a multiply-high result, this experiment would not know.

`reg_ch0` is the check phase 2 exists for. Two free constants — which register, and which instruction in the retirement stream — let the solver pick both, and it asserts that the value the core reads out is the value the last instruction to write it wrote. Between write and read the substituted file encodes the word to a 40-bit codeword, holds it, may scrub it, and corrects it on the way out. [Table 5.4](#) is what that cost.

The cliff sits between check cycle 15 and 18. Six engines were given the depth-25 case under a four-hour bound and every one reached it — but the control matters more than the six rows, because on the *stock* file only two of those six close the check at all, and the two that matter both time out. `docs/63` section 17.2 states the consequence in both directions: nothing there is evidence that the substitution preserves the property at cycle 25, and nothing is evidence that it breaks it. What does close is cycles 12, 15 and 18 — deeper than a write and a read, shallower than one scrub pass, which is 31 cycles when the core is idle.

`docs/63` section 23, added 2026-09-15, takes that document’s second and third routes. Route 2 asks only the register file: `regfile_scrub.sby` binds the real module with its real codec and states R1 (a word written and not rewritten reads back at every later cycle while the scrub walks), R2 (the pointer visits every register 1–31 and never 0), R3 (scrub and core never contend for the write port) and R4 (a fault-free file raises no error). **Route 2 alone does not close, and the reason is the codec.** Cover passes at depth 40 with all 31 visits reached; bounded check and induction sit at step 4, the first cycle at which R1 examines a stored word, because the solver must relate the encoder’s eight parity trees to the decoder’s through thirty-two scrub encoders on one storage, and no pinned solver carries Gaussian elimination.

Route 3 on top of route 2 closes it, unbounded. `regfile_scrub_abs.sby` is the same wrapper with the codec replaced by stand-ins: same module names and ports, a degenerate code whose one check bit says “the word is non-zero”. Under it `abc pdr` proves R1–R4 for every reachable state in **44 seconds**. A property holding for every codec satisfying $\text{dec}(\text{enc}(x)) = x$ holds for the real one, whose $\text{dec}(\text{enc}(x)) = x$ is `regfile_secDED.sby`’s separate theorem [14]; what it does not cover is any property depending on the code’s *distance*, and none is claimed. `prove12 UNKNOWN` means

Table 5.5: The fault-injection campaigns of record. The register-transfer campaigns run each injection twice, armed and held off.

Campaign	Level	Injections	Simulations	What it isolates
docs/42	RTL	1,300	2,600	the unhardened core, 13 strata weighted by 2,198 bits
docs/43 A	RTL	1,400	2,800	the SECDDED register file, same draws and image
docs/43 AW	RTL	1,400	2,800	the watchdog’s cadence contract on top of A
docs/44	RTL	1,400	2,800	the fast-correction path and the fault-report port
docs/46	RTL	812	1,624	the window, on a partitioned supervisor workload
docs/74	gate	1,424	1,739	docs/43’s records on the sign-off netlist
docs/75	gate	174	4 netlists	the reset on a corrected upset

k -induction at $k = 12$ fails its induction step on R4, the fault-free file’s no-error claim, which needs an inductive invariant the wrapper does not state, while the unbounded proof succeeds.

The cost is docs/63section 21: **304,049 core-seconds, 84.5 core-hours, 300 verdicts** — about 92 % of that solver time producing no verdict at all, almost all on one check on one register file.

5.6 Fault injection

Seven campaigns are of record, five at register-transfer level and two at gate level; [Table 5.5](#) is the list.

5.6.1 At register-transfer level

The campaigns share one instrument and one classifier, and each draw is seeded per stratum and index, which makes a re-run a *paired* comparison. That is checked rather than assumed (docs/43section 8.1): the workload compiles to a byte-identical ROM image and the clean run is the same 18,682 cycles with the same signature.

The register file is the result the hardening was bought for. With the watchdog held off — no backstop at all — its `regfile` stratum goes from 78 masked, 0 corrected, 3 silently corrupt, 15 detected and 4 hung in 100 draws to 6, 94, 0, 0 and 0; the new stratum covering the protection’s own 253 flip-flops gives 4, 96, 0, 0, 0, with **zero uncorrectable syndromes in 1,400 injections**. Design-weighted, silent corruption falls from $2.8\% \pm 1.6$ to $1.3\% \pm 0.4$ and hangs from 2.1 % to 0.3 % — with two cautions: the *denominator* changed, the new stratum taking 10.3 % of the weight at a rate of zero, and 0 of 100 is a rate below about 3.7 %, not a rate of zero [16].

docs/44section 9 re-ran that campaign on a design with a rebuilt register read path and got the strongest null result the harness can express: the two 1,400-record files are **byte-identical**, and the two logs differ in one line, the output path. That is stronger than “the rates agree”, because an aggregate delta of zero is consistent with two records moving in opposite directions. No record moved.

docs/46’s campaign decided a feature. The windowed watchdog rejected a kick as too early on 9 of 812 injections, and the seven records that left the core alive were replayed verbatim against a build with the window off. **Two are genuine catches** — wrong answers in the multiplier’s state register that nothing else announced — and **two of the four spurious escalations are the window’s and two are not**. Benefit 2 of 812, 0.25 % [0.07–0.89]; cost 2 of 728, 0.27 % [0.08–1.00]: the intervals overlap almost completely, and both are two draws at one site each. docs/46section 9.1 therefore calls the mechanism conditional and the condition not worth taking. What tips it is that benefit and cost are the same size and this campaign cannot separate them;

behind that is feasibility — the same supervisor dispatched work-triggered admits no window at any timeout, and docs/09 has not chosen between the two disciplines.

5.6.2 At gate level

docs/74 is the first gate-level simulation of the whole system here, replaying the campaign of record one record at a time on the sign-off layout's netlist — md5 2e43d08f . . . , 10,831,575 bytes, 5,873 sequential cells, six vendor macros.

The 1,400 records were first replayed on the current RTL, to ask whether the RTL had moved under the design: **0 of 1,400 changed**, so whatever the netlist campaign says differently is a difference between two levels and not two RTLs [4].

Of the 1,400, **1,175 have a netlist twin**, identified by a per-cycle trace of both designs, by the elaborated design's alias names, or both. Of those, **1,158 classify identically**, and of the 1,129 like-for-like records — excluding 46 injections into a flip-flop synthesis made the twin of two registers at once — **1,126 are identical**. The 17 disagreements are two structures: 15 into the instruction register an optimisation pass stored once for two copies, where **the netlist is worse on all fifteen** — the silicon has one register where the design has two; and 2 into the multiplier's re-encoded state, where it is *better* on both, the upset that silently corrupted a product at register-transfer level putting the one-hot sequencer into a state the program's own self-check notices.

The controls are where the campaign earns those numbers. Four records were run at both levels with the whole mapped flip-flop state written per cycle, and compared over the 964 flip-flops the trace identifies: **0 of 964 differing anywhere in 18,681 cycles**. That control also caught an injection-instant race, which cost the first pass of 450 records, and a controller state bit whose netlist flip-flop carries the register-transfer name but toggles while that bit is constant.

In the gates that would be manufactured the register file corrected 171 of 180 upsets — the same 171, record for record, 0 uncorrectable — and the watchdog's three replica banks voted out 174 of 174.

5.6.3 The finding only the gate level could produce

And in every one of those 174 the system reset itself one clock after the upset. The watchdog's reset request was a combinational decode of five bits of the *voted* word, two of whose three replicas present an exclusive-or tree over all 29 of their flip-flops — and the top level hung the reset synchroniser off it asynchronously, under a comment asserting the signal was registered in that clock domain. It was not. The netlist says it is a glitch three ways: the settled decode is correct on all 174 records; the same deposit at register-transfer level does not reset; and of the five asynchronous-reset fan-in sets, four are driven by nothing or by registered signals and the fifth by 25 flip-flops of the three replica banks.

The repair, W9, is one flip-flop: the same predicate computed from the pre-storage word instead of the voted one, and-ed with the decode. **And no proof, no simulation and no register-transfer campaign in this repository can tell the two designs apart**, because they are the same function of the same state on every cycle — soc_wdog.sby proves that as W9a by *k*-induction at depth 15. The evidence came from elsewhere: on the mapped netlist a satisfiability check shows no input assignment asserts the reset request while the added flip-flop holds zero, and two mutations that would make that vacuous both fail it; and the campaign was re-run on the unrepaired and repaired layouts *and* on both designs' synthesis netlists [1]. **Resets per corrected upset: 174 of 174 before, 0 of 174 after, on both pairs.**

A cocotb test for W9 cannot fail on the design W9 replaced; what it catches is W9 implemented

wrongly, and a flip-flop registered off the decode rather than the pre-storage function fails exactly one of that suite’s 15 tests.

5.7 The cross-check, and where the layers meet

`docs/09target #5` says the formal result and the force/release campaign “must agree”, any disagreement being a blocker. Nothing compared them until 2026-09-14: the comparison sits across two directories and neither pass owned it. `scripts/formal_fi_crosscheck.py` is the comparison.

`formal/lif_ctrl.sby`’s `bmc_safe` task starts a trace from an *illegal* state encoding and proves the design reaches its safe state — not redundant with the induction proof, because the inductive invariant “the encoding is always legal” makes the recovery antecedent unreachable in the induction step. The campaigns inject into exactly that encoding, so every such record is an empirical instance of the theorem’s antecedent, and the theorem forbids one outcome class: silent corruption. Run on the tree today it compares **24 injections across two arms — 12 gate-level and 12 register-transfer, all classified DETECTED**. `sw/tests/test_artefact_digests.py` makes it run: a script nobody runs is not a gate.

The script prints what it does not do, which is half the item: it does not check that the netlist the gate-level arm injected into is the one the proof elaborated. That is an equivalence obligation, and what answers it is **three modules, not a design**. Until 2026-09-09 there was no equivalence check here of any kind; `formal/eqy/` now proves `tmr_voter` at `WIDTH = 55` and both `SECDED` codecs equivalent between the source and the pilot sign-off run’s own netlist, with two negative controls in the default target because a checker that has never returned red is not a checker (`docs/79` section 2). Whole-design equivalence was attempted separately, reached about half the partitions, and is not claimed (`docs/79` section 3) — so the netlist this campaign injected into has never been formally related to the design, which is `docs/09` B.2 layer 3.

5.8 What each layer sees, and where the blind spots are

A proof sees all reachable states, which a simulation cannot. The accelerator’s H5 invariant — its input strobe is exactly one state of the event engine, equal and not merely implied — had been checked by one `cocotb` test and one textual guard; `docs/55` section 9.3 named the gap and `docs/56` section 9.4 again declined to close it, before `soc_npu.sby` was written.

A simulation sees a whole machine running a program, which a proof cannot. The proofs are per block; nothing proves the composition, and the cycle-identical whole-system run is the only evidence that the blocks fit together.

A guard sees the netlist and the tree, which neither can. Every property of the watchdog’s job stays true on a design whose three replica banks the synthesiser has hashed into one — one bank answering three times — and the only check that sees it counts flip-flops in the mapped netlist, 80 instead of 102 (`docs/41` sections 9.4 and 6.3; `docs/55` section 9.3 states the same thing as “all twenty formal tasks stayed green” and attributes it to `docs/43` section 9.4, which does not carry the claim).

A campaign sees what an upset costs this workload, and the gate level sees what the register-transfer level cannot. The proofs say the mechanisms are correct; the campaign says the failure class a hardening was bought for went from 3 silent corruptions and 4 hangs in 100 draws to 0 and 0.

The blind spots that remain:

- **Equivalence covers three modules and not a design:** outside `tmr_voter` and the two

codecs, the register-transfer design and the mapped netlist have never been formally related, and the front-end split of §5.5 adds a second place where one would help.

- **No CSR is checked**, riscv-formal’s CSR checks binding to ports Ibex does not carry — so nothing is said about the trap vector, exception PC, cause or protection registers, which is what the intended software architecture rests on.
- **Memory-protection enforcement is not proved**; the evidence is one simulated test. **The M extension has no formal result**, under two solvers at two stated bounds.
- **Nothing injects a single-event transient**: every campaign forces or deposits into storage, the register file’s decoder alone is 528 combinational cells, and a processor is mostly not flip-flops [25].
- **No timing**, so a fault depending on a setup or hold miss is invisible — and the layout the campaign ran on misses its setup constraint (docs/70 section 6).
- **One workload, one seed, one parameter point**, so records could be replayed; and **no rate**.

The transient and the equivalence obligation are the two that would change what the rest of this chapter means. Neither is scheduled.

Chapter 6

Physical design

The layout is the run `s83romecc5`, completed 2026-09-13: the eight-macro build, carrying the two check macros that hold the boot ROM’s error-correcting code (`docs/67` section 11), and a layout that meets hold at all three corners. Its own `final/metrics.json` is the source for every unattributed number below; a figure from a superseded layout is marked with its date, the rule `docs/64` sets.

No clock frequency is claimed for this design. The flow is given a 20 ns period as a constraint, and every timing result here is a slack against it at a named corner. An early layout document derived a frequency from its own worst slack; `docs/53` section 9.2 refused to publish that figure or any other, on three grounds — it is a number a run reached rather than a requirement, it is a measured non-closure, and the project’s other part carries a different declared clock — and every physical document since reports slack against 20 ns and nothing else.

6.1 The flow, and what is pinned

The chain is rootless: no container, no system package, no root. Each binary is fronted by a shim that establishes only its own `LD_LIBRARY_PATH`, because the OpenROAD build ships a newer glibc that segfaults every other host binary if it leaks into the ambient environment (`docs/12` section 2.1). [Table 6.1](#) is the pin list [15].

The flow does not synthesise, and that is a decision. The runner hands LibreLane a netlist as an initial state and skips `Yosys.Synthesis`, because both hierarchy modes are wrong for this design in opposite directions. Under `flatten`, Yosys honours the `keep_hierarchy` attribute the triple-modular register bank carries as an anti-merge defence [5] and `Checker.YosysUnmappedCells` stops the run; under `deferred_flatten`, a constant driving the core’s `cheriot_enable_i` never folds, and the same RTL comes out as 37,895 cells of 523,809.6444 μm^2 against 27,565 of 396,177.6420 μm^2 , and its worst slow-corner path is 13.8442 ns worse, measured before any placement exists at all (`docs/47` sections 6.2 and 6.3). The project’s own script holds `keep_hierarchy` through optimisation and `abc` and releases it before the final `flatten`.

The flow is deterministic, and that is measured. Two runs of the same netlist give metrics files with 199 keys and zero differing, and byte-identical DEF, ODB, netlist, SDC, LEF, Liberty and SPICE. The only seed anywhere in it is `detailed_route -or_seed 42`, hard-coded in LibreLane’s `drt.tcl`, and resuming the router at ten threads instead of this machine’s twenty gives a byte-identical database (`docs/71` sections 3, 5 and 8). *Every difference between two layouts here is therefore a difference the netlists caused.*

Table 6.1: The pinned toolchain (`docs/12` section 2.2, recorded 2026-08-25 in this environment). The PDK commit [10] is not chosen: every LibreLane [24] release ships `pdk_hashes.yaml` naming the commit it was tested against, and the runner reads that pin out of the installed wheel and refuses to run against anything else.

Component	Version
LibreLane	v3.0.5
Ciel (PDK manager)	v2.6.1
Python	3.12.3
OpenROAD	26Q1-2938-g0e2d771c5e
Yosys	0.67 (2d1509d1b)
KLayout	0.30.9
Magic	8.3.678
Netgen	1.5.272
IHP-Open-PDK	c4b8b4e5e7a05f375cca3815d51b3a37721fbf5c
Standard cells	sg13g2_stdcell, 84 Liberty cells

Table 6.2: The eight macros, from the run’s own `resolved.json` and the vendor LEF `SIZE` statements. All eight are `FIXED`.

Instance	Master	$W \times H$, μm	Origin, μm	Orient
<code>u_ram...u_b0</code>	2048x64	784.48×626.70	(120.00, 60.48)	FS
<code>u_ram...u_b1</code>	2048x64	784.48×626.70	(944.64, 60.48)	FS
<code>u_ram...u_b2</code>	2048x64	784.48×626.70	(120.00, 1387.26)	N
<code>u_ram...u_b3</code>	2048x64	784.48×626.70	(944.64, 1387.26)	N
<code>u_rom...u_b0</code>	1024x32	416.64×336.46	(1769.28, 347.76)	FS
<code>u_rom...u_b1</code>	1024x32	416.64×336.46	(1769.28, 1387.26)	N
<code>u_rom...u_c0</code>	512x16	236.80×191.34	(1769.28, 60.48)	FS
<code>u_rom...u_c1</code>	512x16	236.80×191.34	(1769.28, 1821.96)	N

6.2 The floorplan

The die is $2306.40 \times 2075.22 \mu\text{m}$, reported as $4,786,290 \mu\text{m}^2$ and exactly $4,786,287.408 \mu\text{m}^2$ as the product of the DEF’s own `DIEAREA` corners (`docs/59` section 4.1). The core runs from (60.00, 45.36) to (2246.40, 2029.86) μm : $2186.40 \times 1984.50 \mu\text{m}$ and $4,338,910 \mu\text{m}^2$. Alignment is asserted rather than arranged — 2,186.40 is exactly 4,555 sites of $0.48 \mu\text{m}$ and 1,984.50 exactly 525 rows of $3.78 \mu\text{m}$, and every macro origin is an integer number of sites and rows from the core’s lower left corner (`docs/47` section 6.1) — and `hw/soc/pnr/floorplan.py` refuses to emit a floorplan that has lost the property, reading macro dimensions from the installed PDK’s LEF rather than from a transcript.

Figure 6.1 and table 6.2 are the arrangement, and **it is forced rather than chosen: the forcing constraint is pin geometry**. Every `RM_IHPSG13` part puts all of its signal pins on one edge, at local $y = 0$, on Metal2 and Metal3 — 352 on the 2048×64 and 190 on the 1024×32 (`docs/47` section 6.1) — and a macro whose pin edge faces another macro is a macro the router reaches the long way round. The lower row is therefore `FS`, which mirrors about X and lifts its pins to its *top*; the upper row is `N`, which leaves its pins at its *bottom*; and both face the $700.08 \mu\text{m}$ channel between them. Rotating a hard macro is not an option this project can check, for the reason section 6.7 gives.

The two ROM check macros fit without growing the die, and why they sat unplaced for weeks is worth more than the placement. The open item read that there was $60.48 \mu\text{m}$ beside each ROM and the check part is 236.80 wide — true, and a measurement of the wrong axis.

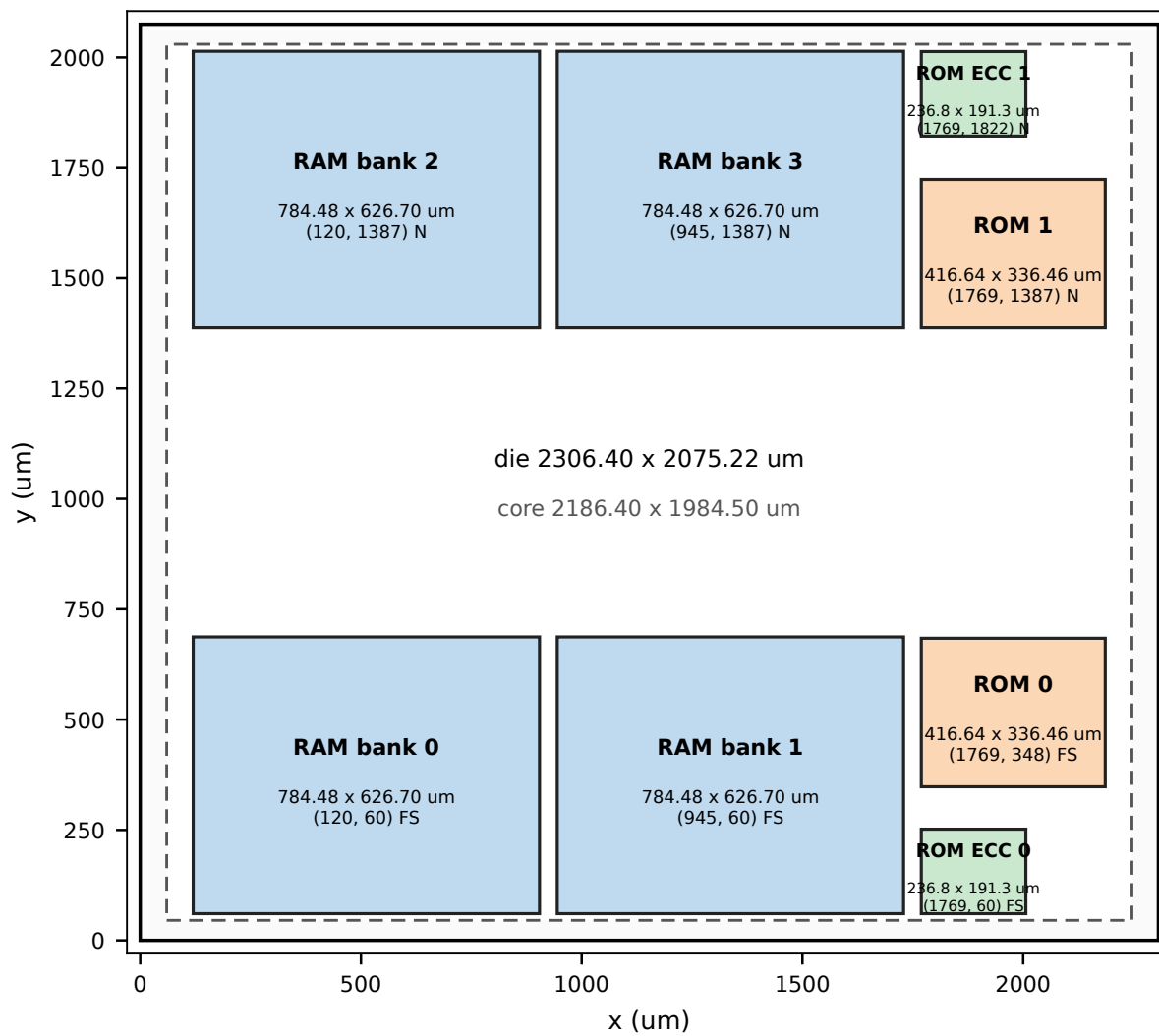


Figure 6.1: The die outline, the core area and the eight vendor macros with their sizes, origins and orientations, drawn from the sign-off run’s own DEF by `thesis/figures/make_layout_figures.py`. Two macro rows face a 700.08 μm standard-cell channel; the third column carries the boot ROM and, in the height the shorter ROM macro leaves over in each row, its two check macros. These eight are the only instances on the die whose position a person chose.

A macro row is as tall as the RAM, 626.70 μm ; the ROM is 336.46; the third column therefore carries 290.24 μm of unused height per row and the check macro is 191.34 tall. The real obstacle was pins: the bottom row is FS, so its ROM's pins are at its top and a check macro in the pocket *above* it would sit across every wire leaving them. The bottom ROM is raised to the top of its own band, to $y = 347.76$, and the check macro takes the blind space beneath; clearances are 95.94 μm and 98.24 μm with a 10 μm halo, computed from the LEF size and row pitch and asserted, never written down as a coordinate (docs/67 section 11). The die, the core and the four RAM origins do not move, which keeps measurements on the earlier six-macro floorplan comparable with these.

The die has named zones, and they are the vocabulary the rest of this chapter uses, as hw/soc/pnr/clint_region.py defines them after docs/72 section 6.4: the *channel* is $687.18 \leq y < 1387.26$; the *upper* and *lower pockets* are the areas at $x \geq 1769.28$ above each ROM, each including the strip beside it at that height; the *strip* is $x \geq 2185.92$. One caveat comes with the definition, because the figures inherit it: those constants are the six-macro floorplan's, in which the lower ROM sat at $y = 60.48$. Here it is at 347.76 with the check macro beneath, so the zone predicate and the “lower pocket” outline in fig. 6.4 describe the earlier geometry, and the zone census quoted below is a measurement on that earlier layout.

Forced is not untested. Two alternatives were hardened — a channel of 82 rows instead of 185, and a die 19.8% narrower with the ROMs as islands — and both are worse at every stage measured, by 0.84 ns and 1.70 ns at post-clock-tree and, for the one that routes, 10.09 ns after global routing on 44.1% more wirelength. Shrinking the die spread the logic rather than compacting it: the tight variant exiled 7,857 of 27,558 cells into the ROM pockets against 739 (docs/48 sections 4 and 5).

6.3 Where the logic went

Figure 6.2 answers “where is the processor, where is the memory” for a die carrying 168,592 instances, and it must be read with its method in hand.

Exactly one cell on this die could be coloured by its own name. Synthesis runs synth-flatten and then abc, so of the 168,592 instance names in the DEF only 1,140 carry a dot and 632 of those are the RAM's; colouring only those would colour a few per cent of the die and put the processor in the wrong place, because the register file keeps its name and the arithmetic unit does not. Attribution is therefore by connectivity, in three passes: Yosys keeps the hierarchical name of any net it did not create, so every net named after a top-level instance votes for that block; each anonymous cell takes the block most of its pins' nets vote for; and cells still unattributed take the majority block of the cells they share a net with, twice. Clock buffers, ties, fill and decap are recognised by name or master and form their own classes (thesis/figures/make_layout_figures.py).

The error bar is printed rather than hidden, and it is larger than the headline figure suggests. Of the 60,694 non-fill instances, thesis/figures/layout_attribution.json records 1 attributed by its own name, 643 named nets producing 19,790 attributions by vote, 1,075 classified as clock tree, and 5,916 — 9.75% — never resolved, drawn grey and counted in the legend. The remaining 33,912, 55.9% of the non-fill instances, came from the two rounds of neighbour propagation (arithmetic on that file's own fields). These maps show where the placer put each block's logic with most of the attribution from the weakest of the three passes; the per-block counts are an attribution with that error bar on them, not a census.

Figure 6.3 separates what fig. 6.2 interleaves. The core's 23,942 attributed instances take the upper half of the channel; the accelerator's 15,686 take the lower half and the left margin. The RAM control and ECC block's 1,510 instances are not a blob but a thin band along the facing edges of both macro rows — it is the logic that fans out to four macros' pins, and the placer put it

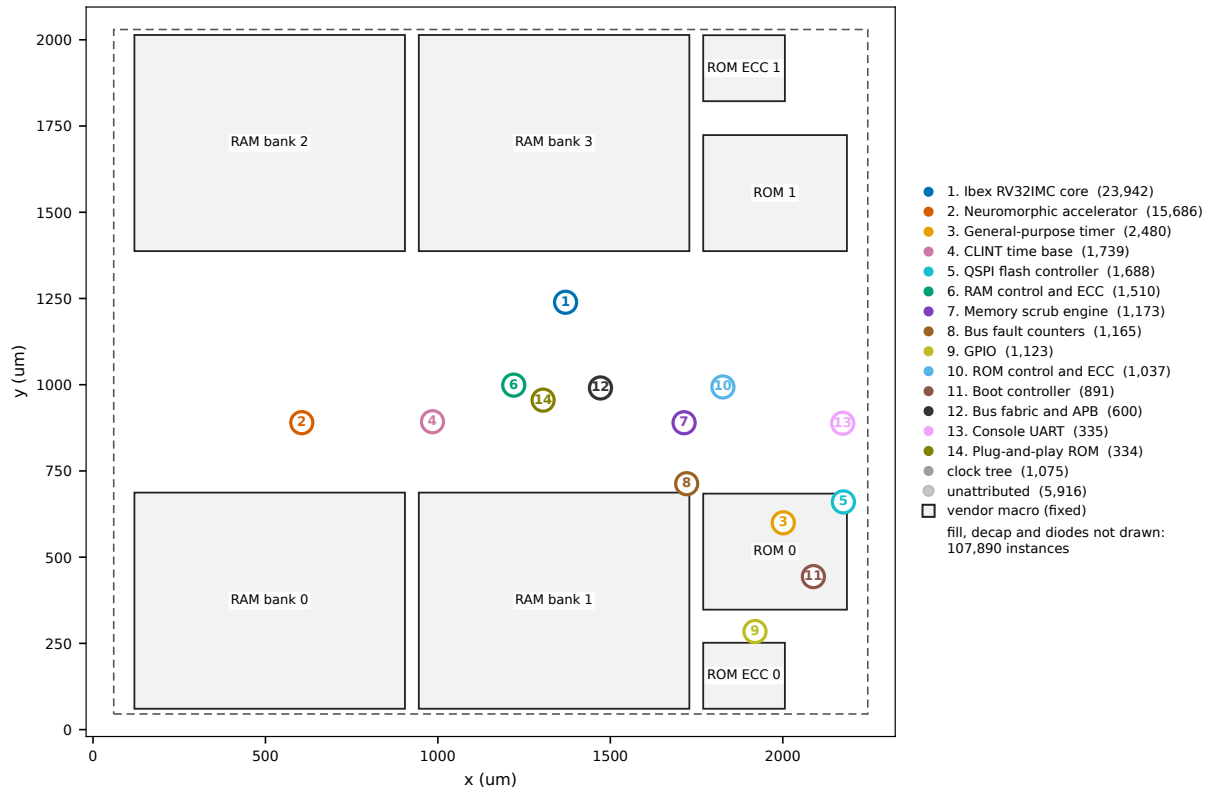


Figure 6.2: Every standard cell in the sign-off layout, coloured by the block it was attributed to, with the macros drawn, a numbered centroid per block and per-block counts in the legend; the 107,890 fill, decap and diode instances are not drawn. Apart from the macros, nothing here was told where to go. The attribution method and its 9.75 % unresolved remainder are described in the text; a centroid is a mean and can land on a macro, which is why markers 3 and 11 sit inside ROM 0.

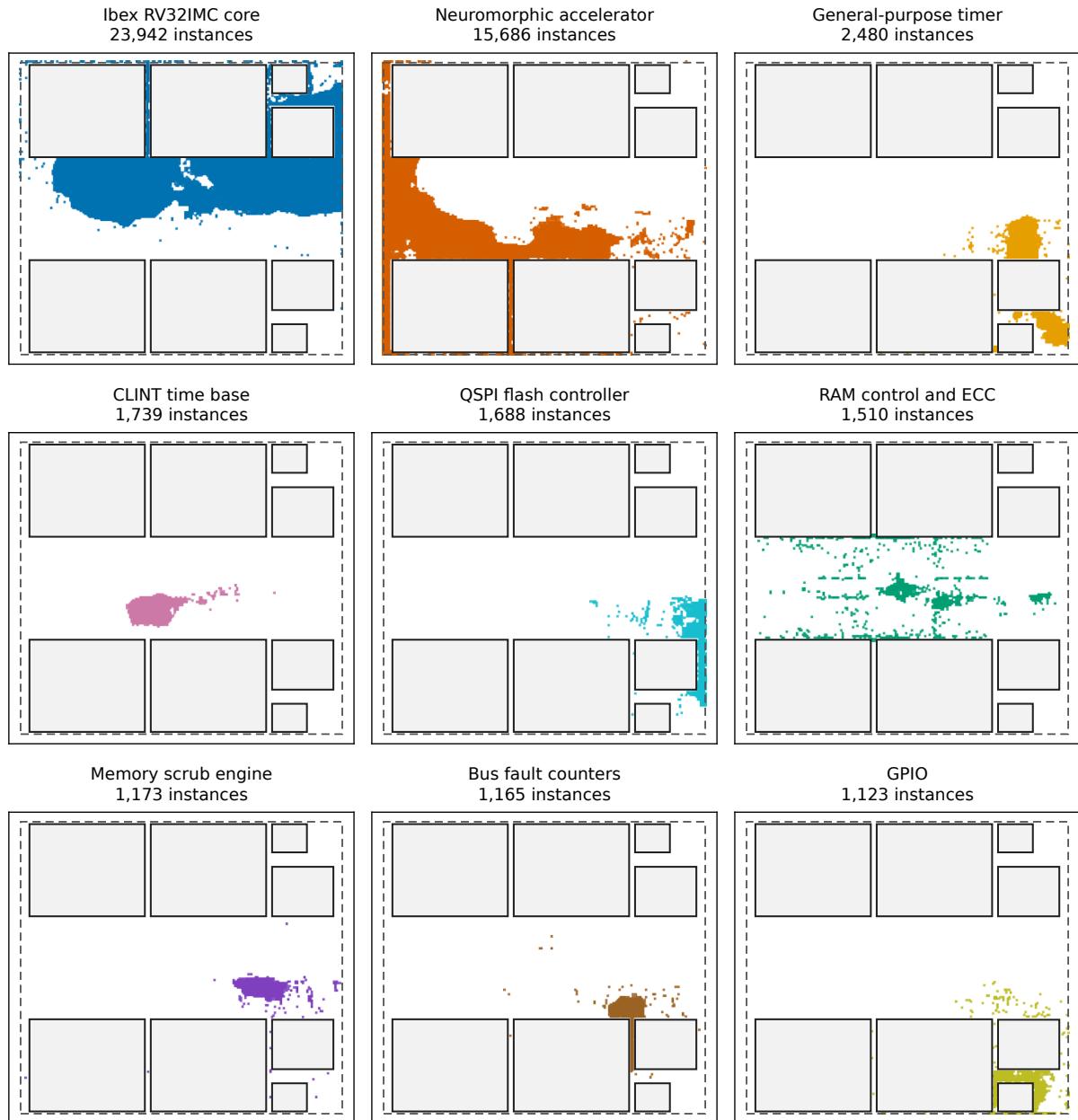


Figure 6.3: The nine most populous blocks, each against the whole die in grey. The core fills the upper channel and reaches into the upper pocket; the accelerator fills the lower channel and the left margin; the peripherals cluster at the right, beside and below the ROM macros; the RAM's control and error-correcting logic is a thin band on both macro rows' pin edges.

Table 6.3: The instance census, from the run’s own `final/metrics.json`. Instance utilisation counts standard cells and macros against the core area; standard-cell utilisation counts standard cells against the core area less the macros.

Quantity	Value
Instances, total	168,592
vendor macros	8
standard cells, excluding fill	64,349
fill cells	104,235
antenna cells (of which diodes)	1,575 (1,258)
Sequential cells	5,998
Integrated clock-gating cells	3
Timing-repair buffers	13,089
inserted for setup / for hold	369 / 11
Clock buffers / clock inverters	815 / 309
Standard-cell area, μm^2	933,010
Macro area, μm^2	2,337,520
Rows / sites	1,517 / 1,003,595
Instance utilisation	0.7538
Standard-cell utilisation	0.4662
Legalisation displacement, mean / max, μm	0.075 / 20.04

on the pins. The CLINT’s 1,739 are the tightest cluster on the die, one compact blob mid-channel, which matters in [section 6.6.1](#); everything with an external interface sits in the third column, in the pockets and the strip beside the ROMs, about as far from the RAM as the core box allows.

[Table 6.3](#) is the census. The macros are 48.8 % of the die area, and the placeable area left inside the core — the core less the macros, which is the denominator the flow’s own utilisation metric uses — is 2,001,390 μm^2 , of which the standard cells fill 46.6 %. That denominator is generous: on the six-macro floorplan `docs/59` section 11 measured the union of the DEF’s ROW statements at 7.66 % less than it, the macro halos and the site grid taking the difference. The first layout of this SoC put 396,177.6420 μm^2 of cells into 2,092,010.95 μm^2 (`docs/48` section 4.1), 18.9 %, and `docs/47` section 8.4 named that sparseness as the probable cause of what layout was costing in timing. It was not: what has since filled the die is the design.

[Figure 6.4](#) is the same placement as a density field, and what it shows is where the logic went: *the channel is not where all of it is*. That is not what the timing turns on, and the difference was measured rather than assumed — on this same run, wire is 1.9 % of a violating path at the median and 3.0 % at the most wire-bound of 1,000, over a median of 78 gate stages, and the CLINT closure the pocket story was about sits 1,802 cells of 1,802 in the channel with none in either pocket (`docs/83` sections 1, 2 and 3). The zone census on the six-macro floorplan puts 45,619 standard cells in the channel, 2,222 in the upper pocket, 3,568 in the lower pocket, 441 in the strip and 436 elsewhere, against a baseline run of the same floorplan at 45,966 / 2,208 / 3,049 / 424 / 173. Those runs differ by 466 cells of netlist, and the extra cells did not go into the channel — it holds 347 *fewer*, and the pockets, the strip and everything outside them 813 more. A net with one pin in the upper pocket and one in the channel crosses a ROM macro, and 140 nets did so in one run and 134 in the other (`docs/72` section 6.4).

6.4 Timing

The flow analyses three corners. It names them exactly, and so does this chapter: `nom_slow_1p08V_125C`, `nom_typ_1p20V_25C` and `nom_fast_1p32V_m40C`. Sign-off timing is on extracted para-

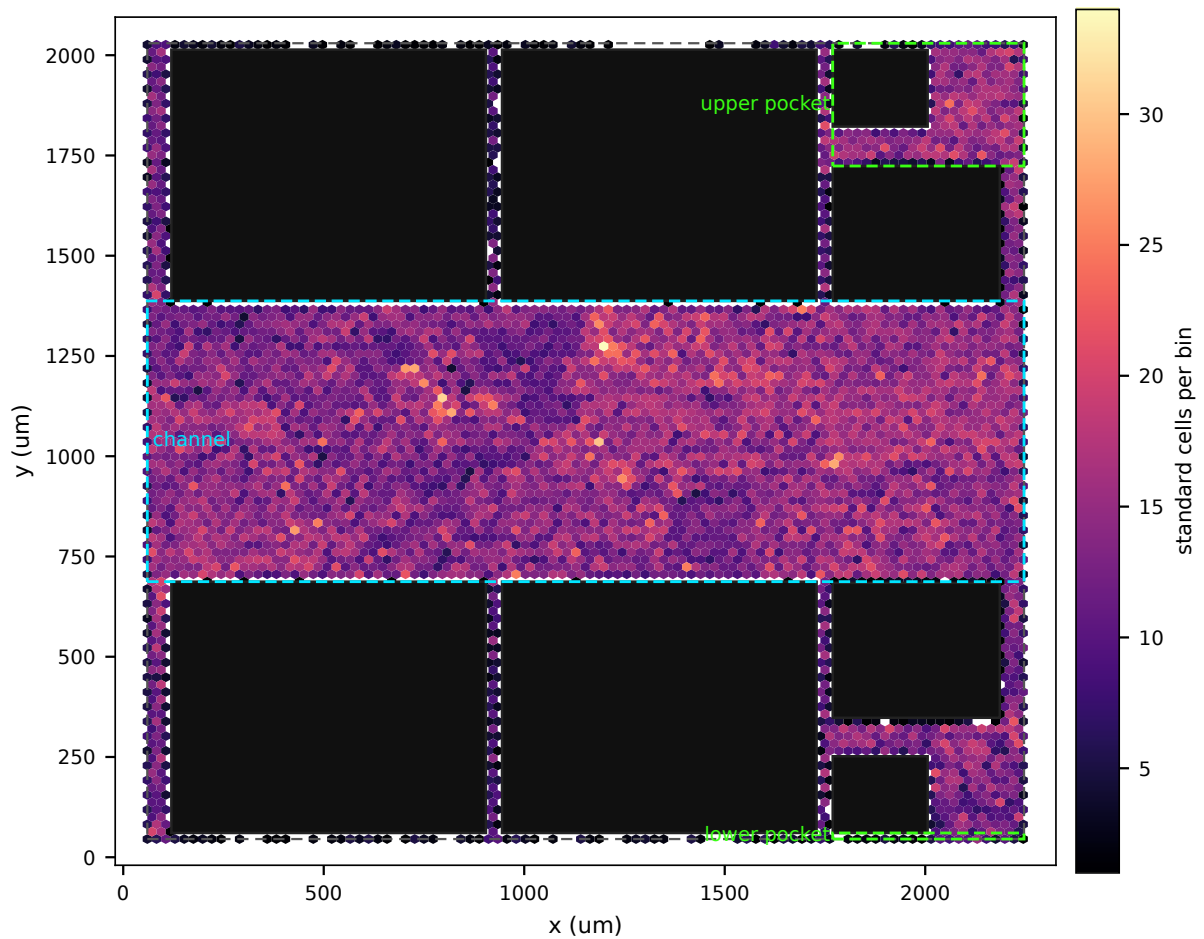


Figure 6.4: Hexbin density of the non-fill standard cells, macros in black, channel and pocket zones outlined. The channel is populated almost uniformly and the rest of the logic is in the pockets and the strip at the right. The “lower pocket” rectangle is drawn from the six-macro floorplan’s constants, in which the lower ROM sat at $y = 60.48$; in this layout the check macro stands there, so that outline no longer marks a pocket.

Table 6.4: Sign-off timing of `s83romecc5`: three corners, extracted parasitics, 20 ns period (the run’s `step-49 summary.rpt` and its `final/metrics.json`). No frequency is derived from any of it.

Corner	Setup WS ns	Vio	Setup TNS ns	Hold WS ns	Vio	Slew vio
<code>nom_slow_1p08V_125C</code>	−7.5758	3,529	−14,735.65	+0.4261	0	533
<code>nom_typ_1p20V_25C</code>	+2.6396	0	0	+0.1769	0	77
<code>nom_fast_1p32V_m40C</code>	+8.3630	0	0	+0.0296	0	20

sitics, at a 20 ns period, with a 5 % derate, 0.25 ns of clock uncertainty and clock-reconvergence pessimism removal. `PNR_CORNERS` is the typical and slow pair, so the fast corner is not *optimised* against during the flow — but all four corner checkers see all three, which is what [table 6.4](#) depends on.

Hold is met at all three corners; setup misses at the slow corner by 7.5758 ns on 3,529 endpoints. There are also 533 max-slew violations at the slow corner and 98 max-cap violations at the fast, and the flow exits non-zero naming them — which it does only because four keys were set before the first run rather than audited after it (`docs/47` section 7). Shipped, `SETUP_VIOLATION_CORNERS` falls through to the PDK’s `["*typ*"]`, so setup would have been examined at the typical corner alone, where this design passes; and the max-cap and max-slew checkers’ step classes override their corner lists to `[""]`, which is truthy, so the list filters to empty, no corner matches, and the step *warns* instead of failing. All four are `["*"]` here. A fifth key is a type rather than a value: the derate ships as the integer 5 and `base.sdc` computes `expr 5 / 100` in Tcl, which is 0, so nothing is derated while the log says otherwise; the configuration sets 5.0, and `resolved.json` cannot audit it because it flattens the float back — a trap that has now bitten in four places (`docs/72` section 7.1).

Two metrics the flow judges with nothing are reported here rather than left in a file. `Checker.WireLength` has no threshold in either PDK and so warns and skips, on a layout whose longest routed net is 4,292.89 µm. And the Classic flow has no max-fanout checker at all, on a layout carrying 589 such violations against a constraint of 10 and a library declaring `default_max_fanout` : 8.

6.4.1 The worst path

`hw/soc/pnr/violator_census.py` resolves all 3,529 violating endpoints against the run’s own netlist and finds six launch points behind them. One is the register file’s read address bit `raddr_b_i[4]`, which launches 2,865 of them and carries −13,195.08 ns of total negative slack. The others are 360 endpoints from a RAM macro’s `A_DOUT[31]`, 154 from `raddr_b_i[2]`, 101 from the accelerator’s axon register, 38 from a launch point the census cannot resolve to a public name, and 11 from the CLINT’s `mtime` codeword register.

The worst is a *register-file read address into the register file’s own error-correcting encoder*, ending at the check bit `g_rf_flops[2].g_chk.rf_chk_q[6]` of that file’s single-error-correcting double-error-detecting code. It decomposes into 2.492291 ns of launch clock — through the core’s integrated clock gate, which is in the launch path — and a data path of 27.0731 ns over **77 driver stages, of which 26.290 ns is cell delay and 0.7827 ns is net delay**, arriving at 29.565346 ns against a required 21.989546 ns, for −7.575801 ns of slack (`step-49 max.rpt`, slow corner). Seventeen of the 77 stages are `sg13g2_xnor2_1`, which is the encoder; eighteen are buffers or inverters, and fifteen of those carry the `rebuffer` or `fanout` names the repair steps give what they insert.

Two stages cost 5.701 ns between them, and why neither was repaired is a property

of the library rather than of the resizer. One `sg13g2_nand4_1` drives 216.9 fF to a single load for 3.034 ns at a 3.839 ns transition; one `sg13g2_xnor2_1` drives 283.6 fF for 2.667 ns at 3.627 ns. Both loads are *legal*: the slow-corner Liberty declares `default_max_capacitance : 0.3` and both sit under it, so `repair_design` never looks at them, and the design has no tighter limit because the capacitance constraint is optional in LibreLane, neither PDK ships a value, and the flow-written SDC contains no `set_max_capacitance` at all. Neither cell can be made larger: `nand4`, `xnor2`, `a22oi`, `o21ai`, `nand3` and `a221oi` ship in *exactly one drive strength*. Both transitions exceed the library's `default_max_transition` of 2.5074 ns, which is what the slow corner's 533 max-slew violations are. Tightening the capacitance limit was measured to sign-off on an earlier layout and cost 1.6525 ns, because `repair_design` is a global instrument and the problem is local: it buffered 4,163 nets instead of 127 and left the targeted repair step 615 gate resizes instead of 1,620 (`docs/48` sections 6 and 7).

6.4.2 The macro read return

The second-largest violating launch group is the memory. The 2048×64 macro's `A_CLK`→`A_DOUT` arc is 9.2941 ns at the slow corner, and it is a function of macro *depth*: across the eighteen single-port parts it runs from 5.2678 ns at 512×8 to 9.6611 ns at 8192×32 while density improves in the same direction, a 4.3933 ns trade end to end. The part chosen is the smallest byte-writable 64 KiB build available, the denser 8192×32 having no per-bit write mask (`docs/47` sections 4.1 and 4.2).

On this layout the arc launches 360 violating endpoints, worst -4.1587 ns, with -864.88 ns behind them. On the earlier six-macro layout the same structure was the *binding* path, and the report charged 9.5277 ns to the arc and 13.1261 ns to the 26 standard-cell stages after it, of which only 0.2481 ns was net delay (`docs/48` section 6.2, superseded 2026-09-13). The conclusion drawn there holds here: *the wires are not slow, the wires are heavy* — the delay is in cells driving capacitance, not in the metal.

6.4.3 Where the resizer stopped

The flow's two post-global-route optimisation stages default to `False` in LibreLane v3.0.5, and nothing in any report says they did not run: the only evidence is one flow-log line. On the first layout of this SoC turning them on was worth 4.30 ns (`docs/47` section 8.5). Both are set here.

What the setup resizer then does was instrumented rather than guessed at — on the six-macro boot-hardened layout that preceded this one, not on `s83romecc5`, so the figures below are that layout's. The step was re-executed on its own saved state with OpenROAD's `repair_setup` trace on, reproducing the flow's step to the byte — 53 metrics, zero differing — and printing why the loop stopped. No configured limit bound; what ended it was OpenROAD's own rule abandoning `repair_timing` when total negative slack fails to improve by the fix-rate threshold in two consecutive hundred-iteration windows — 0.02% here, the source's 0.01% doubled once past iteration 1,000. It chased its worst endpoint for 257 iterations, from -12.260 ns to -7.990 ns; took up nineteen more and reverted every change on seventeen of them after 51 non-improving passes each; and ended that phase having visited **20 of 3,365 endpoints**. A second phase with a reduced move set visited 156 more and found 0.146 ns. The stage that defeated the first drove 180 fF from the channel at (1911.84, 1224.72) into the pocket at (1973.76, 1776.60) — 62 μ m across and 552 μ m up, over a 336 μ m macro, at a 4.69 ns transition and 3.73 ns on one gate that the resizer did size up, from `nor4_1` to `nor4_2`, without recovering it (`docs/72` sections 3.2, 6.3 and 7).

6.5 Hold, antennas, and 29.6 ps

Hold is met at every corner and the fast-corner margin is 0.0296 ns, on a small local path: an unused intermediate-value register in the core’s execute block, through one inverter and one `sg13g2_a22oi_1` and back to its own *D* pin, arriving at 1.353502 ns against a required 1.323898 ns (step-49 `min.rpt`, fast corner). The flow inserted 11 hold buffers in the whole design. `docs/50` section 16 calls this the only layout in the project that meets hold; several earlier six-macro runs also report zero hold violations at all three corners in their own `final/metrics.json`, so what is scarce here is the margin and not the fact.

That 29.6 ps is the scarcest quantity on this die, and two experiments have now been spent against it (section 6.6.2 is the second). The first is the antenna repair. `s83romecc5` leaves two antenna violations — one net into a RAM macro’s `A_ADDR[4]` on Metal3 at $2.26\times$ the ratio limit, one core net at $1.22\times$ — out of 538 net and 729 pin violations the default repair had already cleared. Raising the repair from OpenROAD’s default three iterations and 10 % margin to eight and 25 clears them: run `s83ant` reports zero nets and zero pins. It also inserts 60 more antenna cells — 1,635 against 1,575, of which 1,318 against 1,258 are diodes — and **loses hold**: -0.4351 ns on 55 endpoints, 38 of them at the fast corner. That is a bad trade, because a hold violation is a functional failure at any period where an antenna violation over a ratio limit is a reliability risk to be argued about.

And it is structural rather than tunable. Run `s83ant2` repeats the repair at five iterations and a 15 % margin and lands on *exactly* the same layout: 168,737 instances, the same 1,635 antenna cells, hold -0.4351 ns on the same 55 endpoints, every corner identical to four decimals. Both settings converge on the same 60 diodes, because the repair stops when it is clean and the default three iterations had simply stopped early. Those 60 diodes are the price of the last two violations and their load is what breaks hold; the remaining lever is the resizer’s hold margin rather than the repair’s aggression, and nobody has pulled it (`docs/67` section 11).

6.6 Two placement experiments

6.6.1 The CLINT cluster probe

`docs/72` section 6.4 found both layouts of a pair splitting the CLINT’s 32-bit response register across the upper ROM macro, with the binding stage a one-drive gate driving a macro-crossing net, and asked for one probe: put the CLINT’s read multiplexer and its response register on one side of that macro, with *the crossing count on the worst path* as the metric rather than the slack alone. Both forms of the probe ran on the same six-macro boot-hardened layout as that section’s resizer trace, not on `s83romecc5`.

Deriving the cell set is most of the work, because the design is flattened before `abc` and only net names survive. `clint_region.py` seeds the set with every cell driving a net named `u_clint.*` or `s_rdata_clint[*]`, then takes the fixpoint of “a cell belongs if every load of every one of its outputs already belongs, and no output touches a port or a macro pin”. A plain backward cone does not work: it runs through the fabric’s address multiplexer, the core’s arithmetic unit and the register file’s read multiplexer and returns 4,036 cells, because after `abc` that is one combinational cloud. The exclusive rule gives 743 and stops at the first net a second block also reads.

Two mechanisms were tried. A placement *region* — OpenROAD’s `dbRegion` and `dbGroup`, which no LibreLane key reaches, written into the flow’s own saved database between two steps — fences the set into a box in the channel under the ROM and **does not route**: the fence takes $38,102\ \mu\text{m}^2$ of placeable area out of a channel that has none to give, 3,076 cells spread into the macro gaps, and the global router ends with 420 units of overflow. The flow’s own manual-placement key,

which moves the same cells to the same coordinates and holds every other cell at the baseline's placement, routes and reaches sign-off — and **the crossing count on the register-file read went from 2 to 4, not to 0.**

That failure is the finding. The response registers and the exclusive part of their multiplexer moved; the decode that selects them and the `mtime` error-correcting decoder did not, because the exclusive rule does not reach them; so every net between the two halves now crosses the macro. The worst path moved into the CLINT's time base — the codeword register through its decoder and back, four crossings, three stages over 300 fF — at -13.9924 ns at sign-off against -9.1085 ns for the baseline, with the typical corner failing for the first time. **The crossing is a property of where the cut falls, not of which cells are moved:** the response registers, the fabric decode that selects them and the time base that feeds them have to be on one side of the macro together. The probe also exposed a defect found only after both layouts had used the tool — the expression reading macro definitions out of the LEF was unanchored, and the PDK's property-definitions block also contains the word `MACRO`, so a match starting there swallowed the file's first five macros. On the corrected tool the rule's answer is 999 cells, and its union with the block's register-to-register closure — the set that captures the whole co-placed cluster — is 1,830 (`docs/73` sections 1, 3, 5 and 7).

6.6.2 The memory read register

The newest measurement in the project, added 2026-09-15, carries the sharpest methodological lesson. Registering the macro read return — so the standard-cell stages after `A_DOUT` no longer share a period with the arc — is a synthesis option, and it had been priced twice on designs that no longer existed. `docs/50` section 16 prices it on the design that does: one synthesis, one layout of that netlist through the whole flow with the same configuration and initial state, against `s83romecc5` as the other arm.

The mechanism works exactly as predicted. The macro-read launch group falls from 360 violating endpoints to 17, its worst slack from -4.1587 ns to -2.4798 ns and its total negative slack from -864.88 ns to -28.87 ns ; none of the seventeen survivors is on the bus at all, and sixteen of them capture into the scrub engine's counter. The design's total setup violation improves by 21.9%, from $-14,735.65\text{ ns}$ to $-11,509.24\text{ ns}$, the largest single-change improvement any layout here has produced.

And it does not make the part. The worst path is unchanged in kind and 0.1636 ns worse in value — still the register file's read address into its own encoder, which the read register was never going to touch, lying between the register file and the peripherals rather than behind a macro. And *hold at the fast corner goes negative*: 0.0296 ns becomes -0.2961 ns . Sixty-one new flip-flops and 683 new instances, placed by the same tool under the same constraints, spend 325.7 ps of a 29.6 ps margin. The option stays at its default, and the experiment establishes the price of turning it on with the hold margin as it is today, not a recommendation to turn it on.

The first attempt at that run measured nothing, and it is kept for that reason. It was launched with the option set in the environment and completed all 67 steps — and placed the *baseline* netlist, because the runner takes its netlist from `SYN_NETLIST`, which defaults to the baseline path, and the synthesis knob does not reach it; a `grep` for the option's generate-block name on that run's own netlist returns zero. Its numbers measure nothing, and the run directory is left in place with that sentence as its label. Nothing in the flow said so, because the layout is a perfectly good layout of the netlist it was given: this is the failure mode the project names *the measurement was taken on the wrong design*.

6.7 Geometric sign-off

The sign-off run’s own configuration gates all four deck families off, for a measured reason rather than a schedule. A trial harden containing one RM_IHPSG13 macro produced 1,106,478 Magic DRC error boxes — every one inside the macro footprint and none outside — 2,316 KLayout DRC errors and 359 Netgen LVS errors, and docs/12section 8 records that as a NO-GO on that macro family for this PDK version, tied to an upstream PDK issue. It is a property of the PDK, not of this design. The decks were therefore run separately on the sign-off layout, and all four now have a result on the eight-macro build (docs/67 section 11):

- **Detailed routing.** 0 DRC errors, 0 disconnected pins, 0 power-grid violations on either net, 0 illegal overlaps; 62,767 signal nets and 2 special nets routed with 486,886 vias, all single-cut, over 5,746,921 μm of wire.
- **Netgen LVS.** *Circuits match uniquely*, 263 symmetries, all seven counters zero — with the vendor macro a black box on both sides, so this signs off the connectivity *into* all eight macros’ pins and says nothing about their internals. Supplying the vendor CDL instead makes the run fail, 64,901 netlist nets against 63,155 layout nets, for the bus-delimiter and global-name disagreement the single-macro trial measured as its 359 errors.
- **KLayout DRC.** 11,048 markers, **0 outside the vendor macro hierarchy**, on 204 cells of which all 204 are the macros’ decoders and internal cells.
- **Magic DRC**, macros entering as LEF abstracts: 170 error boxes, 20 inside a macro footprint and 150 outside — and *every one of the 150 lies within $0.5\mu\text{m}$ of a macro edge*, none further and none in open routing area. A LEF obstruction blanket is one shape more than $10\mu\text{m}$ wide, so ordinary routing beside it violates a wide-metal spacing rule it would not violate against the macro’s real geometry; M3.f alone is 102 of the 150. Four of the twenty inside are the check macro’s pin rectangles for A_DOUT[14] and A_DOUT[15], the two outputs a 512×16 row does not use when it holds two words’ seven check bits, so nothing is routed to them.
- **XOR.** Zero differing shapes, on the antenna-repaired layout, 2026-09-14 — the first XOR result in the project on anything carrying the ROM’s check macros.

This is what “clean where the design drew” means, and the phrase is doing precise work. It does not mean the die is manufacturable. It means that in every deck run, each marker has been attributed either to vendor geometry or to the interaction between that geometry’s abstract and the design’s routing beside it, and none to a shape this design’s flow drew in open area [2]. The total is not available, because no open deck can check these macros at the pinned PDK version. One check that did not run is recorded with the ones that did: the detailed router skips a cut-enclosure rule the PDK states in a form this OpenROAD build does not implement — ten warnings over seven layer names, identical layer for layer in the hardened and baseline runs — so it is the PDK’s and every layout has inherited it.

6.7.1 What the KLayout deck is objecting to

Because that deck was named as the possible exit from the NO-GO, its 2,316 was examined in full, and two caveats on the number were found that the report prints nowhere.

The headline double-counts by construction. Two rules, Sdiod.d and Sdiod.e, are evaluated over the same derived layer and flag the identical 1,022 polygons: intersection 1,022, symmetric difference 0. On the eight-macro layout the structure recurs at another scale, 4,464 and 4,464

with exact set equality, so the 11,048 markers are 6,584 distinct shapes. **And the deep-mode count is a property of the deck rather than of the layout:** the same deck in flat mode on the single-macro file returns 122,814 against its 2,316, a factor of 53.0. Any figure quoted from it has to name the deck and the mode; normalised, the two decks are a factor of 4 to 9 apart rather than three orders of magnitude, and they *corroborate* each other on the contact-enclosure rule.

The scope matters as much as the count. A deck that fires three rules could be a deck that looked at three; this one declared **173 rule categories in 47 groups** — M1 to M5, V1 to V4, the top-metal and MIM rules, the well, active, gate, seal and fill families — and evaluated all of them over the whole 113 MB layout [13]. Three fired, all inside vendor geometry.

And the count is a property of the macro's structure rather than of the memory the design asked for, which a sweep of all twenty-seven parts settled: the 1024×8 , $\times 16$ and $\times 32$ parts are identical at 2,672 markers across a fourfold difference in stored bits, the 256×16 , $\times 32$ and $\times 48$ at 2,148, the 64-bit parts add exactly 206 at three different depths, depth moves the count monotonically — the row decoder — and a second port roughly doubles it. A design that halves its word width buys no reduction whatever. The deck swap is therefore not the exit: the deck the flow runs is the PDK's main set, which its own documentation calls verified and intended for foundry precheck, and the deck that returns zero [1] is the extra set, which the same documentation calls not verified or tested [10]. DRC was never the binding constraint anyway — a design that passes DRC and fails LVS is not signed off (docs/54 sections 3 and 7).

6.8 Power

The sign-off run writes a `power.rpt` per corner, and docs/67 section 11 reads the typical corner's as 56.41 mW in total; the metrics file's stored split, rounded here to four significant figures, is 32.78 mW internal, 23.56 mW switching and 0.06900 mW leakage. **That is a flow estimate at a default activity assumption and must be quoted as one:** the default was pinned on an earlier layout by reproducing that layout's published figure to every digit with `set_power_activity -input -activity 0.1 -duty 0.5`, so it is the power of a design in which every top-level input and every register output toggles 0.1 times per clock period and spends half its time at 1 (docs/57 section 2).

Measured instead at the toggle rates of a real program — on that earlier six-macro layout, which does not contain the accelerator at all (docs/57 section 3.2), so superseded for this design — the same recipe gives 33.890 mW computing and 5.539 mW waiting at the typical corner, the waiting window being 57.6 % of the run (docs/57 section 5.2). docs/67 section 11 states plainly that no power number in the corpus has been re-measured on `s83romecc5`. **Idle is 16.3 % of busy, and the reason is a clock gate that had been in the netlist since the core was brought up and that no document had ever mentioned.** It is the real PDK cell, bound deliberately because the core's upstream substitute is a modelling stand-in, and it covers 2,464 of that design's 3,085 flip-flops; on the measured workload it is open 42.4 % of the time, with two clock transitions per cycle when open and 0.0004 when shut (docs/57 section 4). The sign-off layout carries three such cells (table 6.3) where that one carried one; the second and third are the subject of docs/76 and docs/77.

By group, at the typical corner and measured activity, busy against idle in milliwatts: combinational 13.880 against 1.672; sequential 10.749 against 2.414; the clock tree 3.828 against 1.281; the six macros 5.433 against 0.173, a ratio of $31.4\times$ which is the macros' own enable pin doing what it exists for (docs/57 section 7.1). **The clock tree is the least gateable thing in the design and the smallest of the four,** and what it costs to wait is three free-running counters — 231 flip-flops carrying 99.98 % of the idle switching between them.

IR drop is reported as 0.0103 V worst on the supply net and 0.0441781 V on the ground net, which

`docs/67` section 11 states as 0.86 % and 3.68 %. **Only the step's connectivity result is used here:** the supply-location file is unset, so its voltages have no supply position behind them. What it catches is an unpowered macro, a real risk in this flow — the Metal4-to-TopMetal1 connect in `hw/soc/pnr/pdn_macro.tcl` is the one file without which the macros have no power and the flow does not notice for thirty-five steps (`docs/47` sections 2 and 9).

6.9 What is open

Setup does not close ([section 6.4](#)): -7.5758 ns on 3,529 endpoints at the slow corner, launched by the register file's read address that `docs/50` section 16 finds unchanged in kind by the one change that most improves the total. Four leads have been worked through and none recovers it — the hierarchy mode and both post-global-route stages were banked before the first number existed, two alternative floorplans are worse and one does not route at all, and the capacitance constraint was measured to sign-off and costs more than it buys. The one unpriced mechanism left is a hold-repair pass aimed at the fast corner that would let the read register be turned on. The other, a registered request phase in the bus fabric, was built and priced on 2026-09-16: it takes the design's maximum combinational depth from 81 levels to 69 and its median from 39 to 20, costs 24.89 % of the workload's cycle count and $7,618.97 \mu\text{m}^2$, and does not route — its layout `s89rr1` stops at global routing with 387 overflowing tiles, 353 of them on Metal5, against zero for this one (`docs/84` sections 1 and 11).

The macros cannot be signed off, and therefore neither can the die. Every macro area here is a LEF SIZE statement and every macro timing figure a Liberty arc, and both describe a part no open deck can check at the pinned PDK version.

The redundancy is in the netlist and not in the placement [1]. Nothing in this flow requests physical separation between triple-modular replicas and nothing supplies it by default: on the frozen pilot's sign-off layout — the run `docs/34` superseded on 2026-08-31, whose DEF hash is identical to the current frozen one's — the median centre-to-centre distance between corresponding bits of different replicas is $27.52 \mu\text{m}$ against $113.47 \mu\text{m}$ over all cross-replica pairs, a $4.12\times$ concentration, with 49 of the 9,075 cross-replica pairs abutting, and the same picture appears on this SoC across three triplicated structures and two hardens (`docs/79` sections 1.2 to 1.6). The likely mechanism is wirelength minimisation to a shared voter; no seed and no objective function was varied, so it is evidenced and not established. **And no campaign has been re-run on this layout:** every fault-injection number in the project describes an earlier six-macro build.

One citation disagreement belongs here rather than in a footnote. `docs/50` section 16 attributes the antenna-versus-hold trade to `docs/79`, which is about replica placement and formal equivalence and contains no antenna or hold measurement; the trade is measured in `docs/67` section 11, and the metrics quoted above are read directly from `s83romecc5` and `s83ant`.

Chapter 7

The measurements, together

The preceding chapters each carried their own numbers. This one puts them on one page, states what each of them is evidence *of*, and then reports the four experiments that were run while this document was being written, because they are the newest measurements in the project and two of them change what an earlier chapter would otherwise have said.

7.1 The state of the part, on one page

[Table 7.1](#) is the whole design in numbers. Every row names the document that owns the measurement; where a figure has been superseded, the superseding document is named rather than the figure silently replaced, which is the rule `docs/64` sets for this project.

Two rows deserve emphasis because they are the ones a reader is most likely to misread.

The worst setup slack is negative and that is not a defect report. No frequency is claimed for this design (`docs/53`, `docs/05` rule 5); the constraint the flow was given is a period, and the slack is measured against it. A negative worst slack at the slow corner says what period this layout would *not* make at that corner, and the honest statement of the result is the number and the corner together, never a megahertz figure derived from either.

The hold margin is 29.6 ps and everything competes for it. That single number is the reason two separate experiments in this project ended the same way: a change with a clearly positive effect of its own spends more hold margin than the design has. The antenna repair does it (`docs/67` section 11), and so does the memory read register ([section 7.2.4](#)). A design that meets hold by 29.6 ps is a design in which the next improvement must be paid for out of hold.

7.2 Four experiments, 14–15 September 2026

7.2.1 The event engine’s invariant, proved over all reachable states

`docs/56` section 9.4 had named a gap and declined to close it twice: the largest register-transfer file in the system-on-chip carried no formal property set, and the one-line invariant on which its event mechanism rests — that the strobe presenting an event to a neuron is exactly the one state of the event-engine state machine, neither implied by it nor implying it — was checked by one simulation test and one textual guard rather than over all reachable states.

It is now a property set, `hw/soc/formal/soc_npu.sby`, and the invariant holds: bounded model checking to depth 30, k -induction at depth 16, and a cover showing the state is reachable, all pass.

Table 7.1: Headline measurements of the design as of 15 September 2026. Each row is measured, not estimated, and names its source. The layout rows are all from the sign-off run `s83romecc5`, the only layout in the project that meets hold at every corner.

What	Value	Source
<i>Process and floorplan</i>		
Technology	IHP SG13G2, 130 nm, open PDK	docs/12
Die	2306.40 × 2075.22 μm	final DEF
Core area	(60.00, 45.36) to (2246.40, 2029.86) μm	clint_region.py
Vendor macros	8: four 2048 × 64 RAM, two 1024 × 32 ROM, two 512 × 16 ROM ECC	docs/67
Instances placed	168,592	metrics.json
Instance utilisation	0.7538 (standard cells 0.4662)	metrics.json
Flip-flops	5,998	final netlist
<i>Timing, three corners, no frequency claimed</i>		
Worst hold slack, <code>nom_fast_1p32V_m40C</code>	+0.0296 ns	metrics.json
Worst hold slack, <code>nom_typ_1p20V_25C</code>	+0.1769 ns	metrics.json
Worst hold slack, <code>nom_slow_1p08V_125C</code>	+0.4261 ns	metrics.json
Worst setup slack, slow corner	−7.5758 ns	metrics.json
Setup TNS, slow corner	−14,735.65 ns over 3,529 endpoints	violator_census.py
Dominant violating launch point	the register file’s read address	docs/83
Dominant violating capture class	the register file and the rest of Ibex, 2,088 of 3,529 endpoints	docs/83
<i>Geometric sign-off</i>		
Route DRC errors	0	metrics.json
Antenna violating nets	2, and clearing them breaks hold	docs/79
LVS	matches uniquely, macros as black boxes	docs/67
XOR	0 differing shapes	docs/67
<i>Where the logic sits (fig. 6.3)</i>		
Ibex core and its register file	23,942 instances, upper channel	this document
Neuromorphic accelerator	15,686 instances, lower channel and left edge	this document
Fill, decap and antenna diodes	107,890 instances, not drawn in the maps	this document
Cells not attributable by connectivity	5,916, 9.75 % of the logic	this document

Two details of how it was made to work are worth carrying, because they are the kind of thing that costs a day and appears in no manual. The properties are *included into* the module under a formal guard, not read as a separate compilation unit, so that the state machine’s signals are in scope natively; a hierarchical reference from an outside wrapper into a generate scope silently declares a fresh free wire instead, which is how an earlier attempt on a different block produced a counterexample that the register-transfer code cannot produce. And the eight submodules in the instantiation closure are read *without* the formal flag, so that the property includes three of them carry — and the assumptions inside those — stay out of this model.

7.2.2 The register file with its storage and its scrub

`docs/63` section 18 had measured that the register-file consistency check of the RISC-V formal framework proves to check cycle 18 on the core with the error-correcting register file substituted and times out at 21, 23 and 25 under six engines at four hours each. That document ranked three routes out and took none of them.

Two were taken here, and the result is a clean separation of what is hard from what is merely large.

Route 2 asks only the register file: a wrapper that writes a word, waits while the scrub engine walks, and reads it back, with three more properties for the scrub’s coverage, its arbitration against the core and its silence when no fault is present. Cover passes — the scrub visits all 31 architectural registers. Bounded model checking and k -induction do not: four engines, including a bit-level model checker and an IC3 implementation, sit at step 4 and two of them report `TIMEOUT` at 10,800 seconds. Step 4 is the first cycle at which the read-back property examines a stored word, so the solver is being asked to relate eight parity trees at the write encoder to eight at the read decoder through thirty-two more encoders in the scrub path.

Route 3 removes exactly that. With the codec replaced by a degenerate stand-in of the same interface, an IC3 engine proves all four properties *unbounded* in 44 seconds. The codec’s own correctness is a separate theorem that another job in the same directory already proves, so the composition is sound: a property of the wrapper that holds for every codec satisfying decode-of-encode-is-identity holds for the real one.

The abstraction also taught the wrapper’s hidden assumptions. The first stand-in was the identity code, and bounded model checking failed at step 3 with every read bit inverted. The cause is in the read path’s correction mask, which flips every bit whose syndrome matches its column of the parity-check matrix: with an all-zero column set and a zero syndrome, that is every bit. The register file therefore assumes that every column of its parity-check matrix is non-zero and that the encoding of the all-zero word is zero — both true of the real code by construction, and both now written down where the next person to abstract this block will find them.

7.2.3 The multiply and divide checks, bounded rather than stopped

`docs/63` section 8.4 reported six instruction checks stopped without a verdict after 35 to 40 minutes each, and section 11 of the same document listed a different solver as the first route out. That route was taken: all six checks were run again on both the stock and the substituted core under a second bit-blasting solver, each with an explicit time bound written into the job so that the status the tool records is its own word.

All twelve report `TIMEOUT` at 7,200 seconds. None closes.

The interesting number is not in the checks but in the witnesses. The same six cover jobs, which the original solver reached in 9 to 73 seconds, take 1,388 to 10,999 seconds under the new one — between roughly twenty and one hundred and fifty times slower — while still passing. Whatever

eventually closes a sequential multiplier equivalence, it is not a change of satisfiability back end: two of them have now been given the problem and the second is worse at both halves of it.

7.2.4 The memory read register, measured on the design that exists

`docs/50` priced a response register on the memory read path twice, on two designs that no longer exist, and five later documents each recorded that the measurement was owed on the design that does. It was taken: one synthesis with the register enabled, one full layout of that netlist, one whole-system simulation.

The mechanism works exactly as `docs/50` predicted. The macro read return stops being a violating launch group: 360 violating endpoints become 17, their total violation falls from -864.88 to -28.87 ns, and the design's total setup violation improves by 21.9%, the largest single-change improvement any layout in this project has produced. The seventeen that remain do not reach the bus at all; sixteen of them end in the scrub engine's counter.

And it does not make the part. The worst path is unchanged in kind and 0.16 ns worse in value, because it was never behind a macro: it is the register file's read address into the register file's own error-correcting encoder, whose structural answer is not the fabric's at all. The fabric captures 54 of the 3,529 violating endpoints and the time base 138, against 1,224 in the register file (`docs/83` section 4). Worse, the 61 new flip-flops and 683 new instances cost 325.7 ps of hold margin at the fast corner, and the design has 29.6 ps. The register therefore stays at its default of disabled, and what the experiment establishes is its price at today's hold margin rather than a recommendation.

7.2.5 A run that measured nothing, kept on purpose

The first attempt at the experiment above completed all 67 flow steps and produced a layout of the *baseline* netlist. The synthesis knob was set in the environment; the place-and-route script takes its netlist from a different variable and had defaulted to the unmodified one. Nothing in the flow objected, because the layout was a perfectly good layout of the netlist it was given.

The run directory is kept, labelled, and its numbers are in no table. It belongs in this chapter because it is the project's most common failure shape in its purest form: not a wrong measurement, but a correct measurement of the wrong thing. The countermeasure that would have caught it in seconds — count the mechanism in the netlist before believing the layout — is the same one the redundancy guards apply, for the same reason.

Chapter 8

What exists, what does not, and what comes next

8.1 What exists

A radiation-hardened neuromorphic system-on-chip for the IHP SG13G2 130nm open process design kit, in register-transfer form and in layout, built end to end with open tools.

In register-transfer form: a 32-bit RISC-V application core with a single-error-correcting register file and a scrub engine that walks it; a spiking accelerator with its event engine, its address-event queues and its serial attachment to the core; a bus fabric with a real peripheral bus behind it; a time base, a general-purpose timer and a three-stage watchdog; memories behind error-correcting codes; a boot path that is hardened rather than assumed; and triple-modular redundancy where the design decided it was worth its area.

In layout: a placed and routed die of $2306.40 \times 2075.22 \mu\text{m}$ carrying eight vendor memory macros and 168,592 instances, with zero routing design-rule errors, a layout versus schematic comparison that matches uniquely, a zero-difference geometric comparison against the extracted view, and positive hold slack at all three corners. Every geometric error the sign-off decks do report lies inside a vendor macro or within half a micrometre of one: the design is clean where the design drew.

And, throughout, an instrumentation layer that is part of the result rather than scaffolding around it: guards that count redundancy in the netlist because the source cannot be trusted to survive synthesis, formal property sets that state one-line invariants over all reachable states, fault-injection campaigns at both register-transfer and gate level, and a cross-check that asks whether an any-state theorem and a campaign agree about the same block.

8.2 What does not exist, stated plainly

No silicon of this system-on-chip. A frozen slice of the accelerator was prepared for a multi-project shuttle and is pinned by hash; the submission itself is an owner decision with a deadline and a cost, and nothing in this document should be read as saying the part has been fabricated.

No frequency. The project adopted a rule early — report periods, slacks and corner names, never a megahertz figure — and has kept it through every layout. A reader who wants a speed grade will not find one here, and that is deliberate.

No mesh. The multi-node link that would make this a neuromorphic *fabric* rather than a single accelerator is specified down to its 24-bit link word and its per-node counters, and no register-

transfer code implements any of it. By the project's own ranking it is the highest-leverage unbuilt item in the design.

No closed multiplier proof. Six instruction checks in the formal programme have no verdict on either core, under two solvers, at a stated bound of two hours each. The absence is bounded and named; it is not a pass.

No verdict for the register-file consistency check on the substituted core at its required depth. What exists instead is a stronger statement about a smaller thing — the register file and its scrub proved unbounded with the codec abstracted — and a located reason for the original difficulty.

No flight heritage, no qualification campaign, and no claim that any of the hardening in this design has been validated by irradiation. Every hardening claim in this document is a claim about a mechanism, its cost, and what an injected-fault campaign observed in simulation.

8.3 The open items, ranked by what they would settle

1. **The mesh link and a second node.** The largest unbuilt thing, and the one that changes what the part is rather than how well it does what it already does.
2. **A registered request phase in the fabric.** The register-file read into the peripheral space is the dominant violating path in every layout this project has produced, and no placement experiment has moved it, because it is a structure and not a placement. It is the one change that would change the worst number in [table 7.1](#), and it is now priced: docs/84 takes the design's median combinational depth from 39 levels to 20 and its maximum from 81 to 69, for 24.89% of the workload's cycle count and 1.08% of its area, and the layout of it stops at global routing with 387 overflowing tiles, 353 of them on one metal layer. The default stays off for that reason rather than for want of a price.
3. **Hold margin as a first-class budget.** Two separate improvements have now been refused because they cost more hold than the design has. The next layout study worth running is not another improvement but a hold-repair pass aimed at the fast corner, to find out whether the 29.6 ps is structural or tunable.
4. **Irradiation.** Everything in [chapter 4](#) is a mechanism with a simulated campaign behind it. The step that would turn it into evidence about silicon is a beam, and it is not a step this project can take alone.
5. **The remaining peripherals.** A second serial port, the synchronous and two-wire interfaces, and the space-oriented link and bus codecs are specified and unbuilt; two of them — the two-wire interface and the bus codec, both Wishbone-classic cores with an 8-bit data path — are now unblocked by the one bridge that exists and is proved, and none of them is on the critical path of anything above.

8.4 What this document is

A record, not a report. Its distinguishing habit is the one stated in the introduction and applied in every chapter since: a measurement that is superseded is marked with its date and left standing, because the history of a number is evidence about the number. A check that did not finish is reported in the word its tool used. A result whose scope is narrower than its headline says so in the same paragraph as the headline. A run that measured the wrong thing is kept and labelled rather than deleted, because the failure shape is more instructive than the result would have been.

That habit has a cost — this document is longer and less quotable than a datasheet — and one benefit that seems worth it: everything in it can be checked, and the places where it is weakest are the places it says so itself.

Bibliography

- [1] H. M. Akbulut. Triplicated in the netlist, adjacent in the die: Instrumenting a redundancy claim across an open 130 nm flow. <https://github.com/Melihakbulut221/nssoc>, 2026. [V]. Companion paper. Carries the fault-injection campaign that produced bit-identical output on a defective design and its repair, named in section 4.
- [2] H. M. Akbulut. What the deck checked: The design-side residual of an open-PDK sign-off run. <https://github.com/Melihakbulut221/nssoc>, 2026. [V]. Companion paper, same design and same run trees. Carries the deck accounting this paper’s sign-off table summarises.
- [3] E. Cheng, S. Mirkhani, L. G. Szafaryn, C.-Y. Cher, H. Cho, K. Skadron, M. R. Stan, K. Lilja, J. A. Abraham, P. Bose, and S. Mitra. Tolerating soft errors in processor cores using CLEAR. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 37(9):1839–1852, 2018. DOI 10.1109/TCAD.2017.2752705. [V], DOI [P]; arXiv:1709.09921. 586 cross-layer resilience combinations against cost and residual SDC.
- [4] H. Cho, S. Mirkhani, C.-Y. Cher, J. A. Abraham, and S. Mitra. Quantitative evaluation of soft error injection techniques for robust system design. In *50th ACM/EDAC/IEEE Design Automation Conference (DAC)*, 2013. DOI 10.1145/2463209.2488859. [V]. The outcome taxonomy this paper’s classes map onto, and the measurement of what a higher abstraction level costs.
- [5] ESA Microelectronics Section. Functional triple modular redundancy (FTMR). Technical Report FPGA-003-01, European Space Agency, 2000. [V] as a technical note, not peer reviewed. Documents the use of synthesis-preservation attributes to stop the tool removing redundancy.
- [6] European Cooperation for Space Standardization. Space product assurance — radiation hardness assurance: EEE components. Technical Report ECSS-Q-ST-60-15C, ESA-ESTEC, 2012. [V]. Rev.1 issued 2025. The standard whose evidence tiers this work does not reach.
- [7] C. Frenkel, M. Lefebvre, J.-D. Legat, and D. Bol. A 0.086-mm² 12.7-pj/sop 64k-synapse 256-neuron online-learning digital spiking neuromorphic processor in 28-nm CMOS. *IEEE Transactions on Biomedical Circuits and Systems*, 2019. DOI 10.1109/TBCAS.2018.2880425. [V]. ODIN, the substrate Nijssink et al. irradiated.
- [8] Frontgrade Gaisler. GR801: Radiation-hardened neuromorphic processor. <https://www.gaisler.com/products/gr801>, 2026. [V]. Product brief, status "under development", released April 2026; NOEL-V RV64GC with a licensed BrainChip Akida accelerator on ST 28 nm FDSOI. The commercial reference this design positions against and does not compete with.
- [9] M. Y. Hsiao. A class of optimal minimum odd-weight-column SEC-DED codes. *IBM Journal of Research and Development*, 14(4):395–401, 1970. DOI 10.1147/rd.144.0395. [V].

- [10] IHP GmbH. IHP-Open-PDK: Open source PDK for the IHP SG13G2 130 nm BiCMOS technology. <https://github.com/IHP-GmbH/IHP-Open-PDK>, 2026. [V]. The process design kit under test. Deck contents, the main-versus-maximal rule-set distinction and its README wording were read from the installed commit c4b8b4e5e7a05f375cca3815d51b3a37721fbf5c.
- [11] X. Iturbe, B. Venu, E. Ozer, J.-L. Poupat, G. Gimenez, and H.-U. Zurek. The Arm triple core lock-step (TCLS) processor. *ACM Transactions on Computer Systems*, 2019. DOI 10.1145/3323917. [V]; the extended author list is [P].
- [12] J. Johnson and M. Wirthlin. Voter insertion algorithms for FPGA designs using triple modular redundancy. In *ACM/SIGDA International Symposium on Field Programmable Gate Arrays (FPGA)*, 2010. DOI 10.1145/1723112.1723154. [V].
- [13] Andreas Krinke, Robert Fischbach, and Jens Lienig. Layout verification using open-source software. In *Proceedings of the 2024 International Symposium on Physical Design, ISPD '24*, pages 137–142. ACM, 2024. [V]. Title, full author list, venue, year and page range retrieved from Crossref, [api.crossref.org/works/ 10.1145/3626184.3635289](https://api.crossref.org/works/10.1145/3626184.3635289), 2026-09-14. Tool-side: what a generated deck can express for a commercial PDK. This paper is design-side and the two compose.
- [14] A. Kumar. Pragmatic formal verification of sequential error detection and correction codes (ECCs) used in safety-critical design. In *Design and Verification Conference (DVCon) U.S.*, 2023. [V]; arXiv:2404.18270. Formally proving an ECC is routine industrial practice.
- [15] C. Lamb and S. Zacchiroli. Reproducible builds: Increasing the integrity of software supply chains. *IEEE Software*, 39(2):62–70, 2022. DOI 10.1109/MS.2021.3073045. [V]; arXiv:2104.06020. The framework citation for hash pinning.
- [16] R. Leveugle, A. Calvez, P. Maistri, and P. Vanhauwaert. Statistical fault injection: Quantified error and confidence. In *Design, Automation and Test in Europe (DATE)*, 2009. DOI 10.1109/DATE.2009.5090716. [P], DOI reconstructed. How many faults must be injected for a stated margin.
- [17] S. Mitra, N. R. Saxena, and E. J. McCluskey. A design diversity metric and analysis of redundant systems. *IEEE Transactions on Computers*, 51(5), 2002. DOI 10.1109/TC.2002.1004589. [P], DOI reconstructed. The citation against identical replication: identical replicas have diversity near zero against correlated physical disturbances.
- [18] F. Modaresi, M. Guthaus, and J. K. Eshraghian. OpenSpike: An OpenRAM SNN accelerator. arXiv:2302.01015, 2023. [V]. Fully open EDA and open PDK, taped out in SkyWater 130 nm at roughly thirty times this design’s scale. An SNN on an open PDK is not a contribution.
- [19] S. S. Mukherjee, C. Weaver, J. Emer, S. K. Reinhardt, and T. Austin. A systematic methodology to compute the architectural vulnerability factors for a high-performance microprocessor. In *36th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO-36)*, 2003. [V]; the author list beyond the first two is [P]. Origin of AVF, ACE/un-ACE, SDC and DUE.
- [20] J. Naoukin, M. Isik, and K. Tiwari. A survey examining neuromorphic architecture in space and challenges from radiation. arXiv:2311.15006, 2023. [V].
- [21] W. Nijsink, B. Endres Forlin, A. Yousefzadeh, and M. Ottavi. Shooting neutrons at neurons: Radiation testing of a spiking neural network on flash-based FPGAs. arXiv:2605.00030, 2026. [V]. Beam data for an SNN; the evidence tier this paper explicitly does not reach.

-
- [22] A. M. Saleh, J. J. Serrano, and J. H. Patel. Reliability of scrubbing recovery techniques for memory systems. *IEEE Transactions on Reliability*, 39(1):114–122, 1990. DOI 10.1109/24.52622. [P], DOI reconstructed from the Xplore document number.
- [23] D. A. Santos, A. M. P. Mattos, D. R. Melo, and L. Dillo. Enhancing fault awareness and reliability of a fault-tolerant RISC-V system-on-chip. *Electronics*, 12(12):2557, 2023. DOI 10.3390/electronics12122557. [V]. The closest published work to this paper’s angle: its stated contribution is extending fault OBSERVABILITY.
- [24] M. Shalan and T. Edwards. Building OpenLANE: A 130nm OpenROAD-based tapeout-proven flow. In *IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2020. DOI 10.1145/3400302.3415735. [V].
- [25] P. Shivakumar, M. Kistler, S. W. Keckler, D. Burger, and L. Alvisi. Modeling the effect of technology trends on the soft error rate of combinational logic. In *International Conference on Dependable Systems and Networks (DSN)*, pages 389–398, 2002. [V], DOI not established. Logical, electrical and latching-window masking.